



Dissertation

Multilevel Monte Carlo Method for Models on Mixed Meshes

Study programme:

P0788D270007 – Applied Sciences in Engineering

Author:

Ing. Martin Špetlík

Supervisor:

doc. Mgr. Jan Březina, Ph.D.

Liberec 2026

Declaration

I hereby certify, I, myself, have written my dissertation as an original and primary work using the literature listed below and consulting it with my thesis supervisor and my thesis counsellor.

I acknowledge that my dissertation is fully governed by Act No. 121/2000 Coll., the Copyright Act, in particular Article 60 – School Work.

I acknowledge that the Technical University of Liberec does not infringe my copyrights by using my dissertation for internal purposes of the Technical University of Liberec.

I am aware of my obligation to inform the Technical University of Liberec on having used or granted license to use the results of my dissertation; in such a case the Technical University of Liberec may require reimbursement of the costs incurred for creating the result up to their actual amount.

At the same time, I honestly declare that the text of the printed version of my dissertation is identical with the text of the electronic version uploaded into the IS STAG.

I acknowledge that the Technical University of Liberec will make my dissertation public in accordance with paragraph 47b of Act No. 111/1998 Coll., on Higher Education Institutions and on Amendment to Other Acts (the Higher Education Act), as amended.

I am aware of the consequences which may under the Higher Education Act result from a breach of this declaration.

26. 1. 2026

Ing. Martin Špetlík

Acknowledgements

I would like to express my deepest gratitude to my supervisor, doc. Mgr. Jan Březina, Ph.D., for his invaluable guidance, inspiring discussions, and continuous support throughout my doctoral studies. His insightful advice, encouragement, and the opportunity to participate in his research projects have been instrumental in the successful completion of this dissertation.

I am also sincerely grateful to Dr. Eric Laloy for his generous hospitality and expert guidance during my research stay at the Belgian Nuclear Research Center (SCK CEN).

Víceúrovňová Metoda Monte Carlo pro Modely na Smíšených Sítích

Abstrakt

Modelování procesů podzemních vod v rozpukaných krystalických horninách je zásadní pro hodnocení bezpečnosti hlubinných úložišť radioaktivního odpadu. Modely typu discrete fracture-matrix (DFM) umožňují explicitní reprezentaci puklinových sítí i okolní horninové matrice, jejich vysoká výpočetní náročnost však omezuje využití v úlohách kvantifikace nejistot, které vyžadují velký počet vyhodnocení modelu. Tato disertační práce představuje víceškálový přístup, který integruje víceúrovňovou metodu Monte Carlo (MLMC), numerickou homogenizaci a náhradní modelování založené na hlubokém učení s cílem umožnit efektivní odhady statistik. Metoda MLMC je využita ke snížení výpočetních nákladů ve srovnání se standardní metodou Monte Carlo, zatímco homogenizace slouží k vyjádření vlivu hydraulické vodivosti puklin, které jsou pod rozlišovací schopností modelu. Pro snížení vysoké výpočetní náročnosti konvenční numerické homogenizace je zaveden náhradní model založený na hlubokém učení, který přímo predikuje ekvivalentní tenzory hydraulické vodivosti z voxelizovaných trojrozměrných oblastí reprezentujících hydraulické vodivosti matrice a puklin. Náhradní modely jsou trénovány na datech generovaných pomocí DFM simulací a systematicky hodnoceny napříč širokým spektrem konfigurací puklinových sítí, kontrastů vodivosti puklina-matrice a parametrů matrice. Výsledky prokazují vysokou predikční přesnost, přičemž ve většině testovacích scénářů platí $\text{NRMSE} < 0,22$, a výrazné výpočetní zrychlení přesahující faktor 100 při inferenci na GPU. Praktický dopad použití náhradních modelů je hodnocen na dvou makroskopických problémech. V obou případech navržený přístup zachovává přesnost sledovaných veličin ve srovnání s použitím numerické homogenizace. Nakonec jsou porovnány MLMC odhady zahrnující homogenizaci pomocí náhradních modelů se standardními Monte Carlo odhady, dosaženo je snížených výpočetních nákladů o více než 90% při zachování statistické přesnosti odhadů.

Klíčová slova: víceúrovňová metoda Monte Carlo, DFM modely, surrogate založený na hlubokém učení, konvoluční neuronové sítě, numerická homogenizace

Multilevel Monte Carlo Method for Models on Mixed Meshes

Abstract

Modeling groundwater processes in fractured crystalline rock is essential for the safety assessment of deep geological repositories of radioactive waste. Discrete fracture–matrix (DFM) models allow for explicit representation of fracture networks and the surrounding rock matrix, but their computational cost limits their applicability in uncertainty quantification workflows that require large numbers of model evaluations. This dissertation proposes a multiscale computational framework that integrates the multilevel Monte Carlo (MLMC) method, numerical homogenization, and deep learning–based surrogate modeling to enable efficient statistical estimation. MLMC is employed to reduce computational cost relative to the standard Monte Carlo method, while homogenization is used to upscale the hydraulic conductivity effects of sub-resolution fractures. To overcome the computational bottleneck associated with repeated homogenization, a deep learning surrogate is developed to directly predict equivalent hydraulic conductivity tensors from voxelized three-dimensional representations of fracture and matrix hydraulic conductivities. The surrogates are trained on datasets generated from DFM simulations and systematically evaluated over a broad range of fracture network configurations, fracture–matrix conductivity contrasts, and matrix parameters. The results demonstrate strong predictive accuracy, with NRMSE < 0.22 in most test scenarios, and substantial computational gains exceeding a factor of 100 when inference is performed on a GPU. The practical performance of surrogate-based upscaling is demonstrated in two macroscale groundwater flow applications, where the surrogate approach preserves the accuracy of quantities of interest compared to conventional numerical homogenization. Finally, MLMC estimators incorporating surrogate-based homogenization are compared with standard Monte Carlo estimators, yielding computational cost reductions exceeding 90% while maintaining reliable statistical accuracy.

Keywords: multilevel Monte Carlo method, discrete fracture–matrix model, deep learning surrogate, convolutional neural networks, numerical homogenization

Contents

List of abbreviations	11
1 Introduction	12
2 State of the Art	14
2.1 Monte Carlo Methods in Groundwater Modeling	14
2.2 Numerical Homogenization of Hydraulic Conductivity	15
2.3 Surrogates in Groundwater Modeling	15
2.3.1 Deep Learning Surrogate	16
3 Multiscale Models of Fractured Medium	18
3.1 Discrete Fracture Network	18
3.2 Spatially Correlated Random Field on the Matrix	20
3.3 Discrete Fracture–Matrix Model	21
3.3.1 Constraint Problem	21
3.3.2 Anisotropy Problem – Numerical Homogenization	21
4 Monte Carlo Methods	23
4.1 Standard Monte Carlo Method	23
4.2 Multilevel Monte Carlo Method	24
4.2.1 Properties of MLMC Method	25
4.2.2 Optimal Number of Samples	27
4.3 MLMC Software Library	29
4.3.1 Sampling Algorithm	29
4.4 MLMC Method for DFM Models	30
4.4.1 Upscaling Procedure	31
4.4.2 MLMC Level Transition	33
5 Deep Learning Surrogate for Numerical Homogenization	34
5.1 Surrogate Training	34
5.1.1 Maximum Likelihood Estimation	35
5.1.2 Loss Function as MLE	37
5.1.3 Regularization	39
5.1.4 Activation Functions	39
5.1.5 Surrogate Accuracy Assessment	40
5.2 Multilayer Perceptrons	41
5.3 Convolutional Neural Networks	41

5.4	Graph Convolutional Neural Networks	42
6	Datasets	46
6.1	Dataset Configuration	46
6.2	Voxelization	47
6.3	Datasets Analysis	48
6.4	Datasets Preprocessing	49
6.4.1	Sample-wise Physical Scaling	49
6.4.2	Statistical Normalization for Training	50
7	Surrogate Prediction Accuracy Assessment	52
7.1	Surrogate Architecture	52
7.2	Loss Comparison	53
7.3	Prediction Accuracy on Test Datasets	54
7.4	Impact of SRF Correlation Length on Prediction Accuracy	56
7.5	Prediction Accuracy for Different SRF Means and Covariances	59
7.6	Impact of Number of Fractures on Prediction Accuracy	66
7.7	Prediction Accuracy Across Datasets with Varying DFN Configurations	68
7.8	Prediction Accuracy with Varying Homogenization Block Sizes	71
7.9	Computational Cost Reduction by Surrogates	73
7.10	Utilization of Upscaled Equivalent Hydraulic Conductivity Tensors	74
7.10.1	Constraint Problem	75
7.10.2	Anisotropy Problem	76
7.10.3	Comparison to Upscaling without Homogenization	77
8	Results on MLMC with Surrogate	80
8.1	Multilevel Monte Carlo Estimator Validity	82
8.2	Constraint Problem	85
8.2.1	Reference and Standard Monte Carlo	85
8.2.2	MLMC Estimators	85
8.2.3	Performance Assessment of 3LMC-2	88
8.3	Anisotropy Problem	90
8.3.1	Reference and Standard Monte Carlo	90
8.3.2	MLMC Estimators	91
8.3.3	Performance Assessment of 3LMC-2	95
9	Conclusions	98

List of Figures

4.1	Comparison of the fine DFM model fracture network $\mathcal{F}_{h,D}$, with $h \leq 5$ (left), and the corresponding coarse DFM model fracture network $\mathcal{F}_{H,D}$, with $H \leq 10$ (right).	32
4.2	The fine DFM model ($h \leq 5$) and corresponding coarse DFM model ($H \leq 10$) with homogenized hydraulic conductivity tensor component K_{xx}	32
6.1	Illustration of the input random field of K_{xx} defined on an unstructured mesh and its voxelized representation.	48
6.2	Distributions of $\mathbf{K}^{eq}$ component k_{xx} for Dataset $\mathcal{A}$, shown for different matrix correlation lengths $\lambda \in \{0, 10, 25\}$	50
6.3	Distributions of $\mathbf{K}^{eq}$ components for datasets of different K_f/K_m	51
7.1	The prediction accuracy of the trained surrogates for the components of $\mathbf{K}^{eq}$ evaluated on test datasets.	55
7.2	Effect of the SRF correlation length λ on the prediction accuracy of the surrogates.	57
7.3	Prediction accuracy of Surrogate A for the diagonal and off-diagonal components of the equivalent hydraulic conductivity tensor $\mathbf{K}^{eq}$, evaluated on test datasets with $\lambda \in \{0, 10\}$	58
7.4	Prediction accuracy of $\mathbf{K}^{eq}$ diagonal components when using Surrogate A, Surrogate B, and Surrogate C to predict on data with $\boldsymbol{\mu}$, $\boldsymbol{\mu}_4$, and $\boldsymbol{\mu}_5$	63
7.5	Distributions of $\mathbf{K}_{xx}^{eq}$ divided by the average matrix hydraulic conductivity $\overline{\mathbf{X}}_j^m$. Datasets with $\boldsymbol{\mu}$, $\boldsymbol{\mu}_4$, and $\boldsymbol{\mu}_5$ are compared.	65
7.6	Impact of the number of fractures (P_{30}) on the surrogate's prediction accuracy.	67
7.7	Impact of the number of fractures (P_{30}) on the surrogate's prediction accuracy for the diagonal and off-diagonal components of $\mathbf{K}^{eq}$	67
7.8	The prediction accuracy of the trained surrogates for the diagonal and off-diagonal components of $\mathbf{K}^{eq}$ evaluated on test datasets with different DFNs.	70
7.9	Prediction accuracy on data with different block sizes $b \in \{10, 20, 30\}$, reference $b = 15$. Constant fine mesh step $h = 5$ and coarse mesh step $H = 10$	72

7.10	Prediction accuracy on data with different block sizes $b \in \{10, 20, 30\}$, reference $b = 15$. Constant ratio between b , H , and h	73
7.11	Comparison of computational time between numerical homogenization (C_H) and its surrogate counterpart.	74
7.12	Speedup obtained by replacing numerical homogenization with the surrogate model.	74
7.13	Accuracy comparison of the outflow Y from macroscale DFM models of the Constraint problem.	78
7.14	Accuracy comparison of the k_{xx} component of K_{UP}^{eq} from macroscale DFM models of the Anisotropy problem, domain $\Omega_{UP} = (60, 60, 60)$. .	78
7.15	Accuracy comparison of the k_{xx} component of K_{UP}^{eq} from macroscale DFM models of the Anisotropy problem, domain $\Omega_{UP} = (15, 15, 15)$. .	79
8.1	Comparison of distributions of estimated moments obtained by approach (A) and approach (B).	84
8.2	Level variances V_l for the Constraint problem.	87
8.3	Level computational cost C_l for the Constraint problem.	88
8.4	Normalized runtime breakdown for the Constraint problem using three-level MLMC estimators.	89
8.5	Variance and computational cost per level of 3LMC-2 variants for the Constraint problem.	90
8.6	Level variances V_l for the Anisotropy problem.	94
8.7	Level computational cost C_l for the Anisotropy problem.	94
8.8	Normalized runtime breakdown for the Anisotropy problem using three-level MLMC estimators.	96
8.9	Variance and computational cost per level of 3LMC for the Anisotropy problem.	96

List of Tables

6.1	Discrete fracture network parameters.	47
7.1	The architecture of the surrogate.	53
7.2	Prediction accuracy (R^2) of surrogates trained with mean squared error (MSE) and squared Mahalanobis distance losses.	54
7.3	Prediction accuracy (NRMSE) of surrogates evaluated on test datasets generated with different covariance matrices Σ_i of the spatial random field (SRF).	61
7.4	Prediction accuracy (NRMSE) of surrogates evaluated on test datasets generated with different mean vectors μ_i of the spatial random field (SRF).	64
7.5	Test DFN configurations.	68
7.6	Comparison of the NRMSE for the Constraint problem outflow Y obtained from coarse DFM models for different fracture-to-matrix conductivity ratios K_f/K_m . $\Omega_{UP} = (60, 60, 60)$	75
7.7	Comparison of the NRMSE for the Constraint problem outflow Y obtained from coarse DFM models for different fracture-to-matrix conductivity ratios K_f/K_m . $\Omega_{UP} = (15, 15, 15)$	76
7.8	Comparison of the NRMSE of the components of the upscaled equivalent hydraulic conductivity tensor $\mathbf{K}_{UP}^{eq}$ obtained from coarse DFM models for different fracture-to-matrix conductivity ratios K_f/K_m . $\Omega_{UP} = (60, 60, 60)$	77
7.9	Comparison of the NRMSE of the components of the upscaled equivalent hydraulic conductivity tensor $\mathbf{K}_{UP}^{eq}$ obtained from coarse DFM models for different fracture-to-matrix conductivity ratios K_f/K_m . $\Omega_{UP} = (15, 15, 15)$	77
8.1	Overview of all multilevel Monte Carlo (MLMC) methods investigated.	82
8.2	Computational cost of MLMC estimators for the Constraint problem.	86
8.3	Moment estimates and corresponding standard errors of MLMC estimators for the Constraint problem.	87
8.4	Computational cost of 3LMC-2 variants for the Constraint problem.	89
8.5	Computational cost of MLMC estimators for the Anisotropy problem.	91
8.6	Moment estimates and corresponding standard errors of MLMC estimators for the Anisotropy problem.	92
8.7	Computational cost of 3LMC-2 variants for the Anisotropy problem.	95

List of abbreviations

CNN	convolutional neural network
DFM	discrete fracture–matrix
DFN	discrete fracture network
DL	deep learning
DNN	deep neural network
DNS	direct numerical simulation
ECM	equivalent continuum medium
GCNN	graph convolutional neural network
MC	Monte Carlo
MLE	maximum likelihood estimation
MLMC	multilevel Monte Carlo
MLP	multilayer perceptron
MSE	mean squared error
NRMSE	normalized root mean squared error
PDE	partial differential equation
SRF	spatial random field
QoI	quantity of interest

1 Introduction

The modeling of groundwater processes is regarded as essential for the assessment of the long-term safety of deep geological repositories for radioactive waste, as noted in [1]. The planned repository in the Czech Republic is expected to be situated in crystalline rock, where the limited intergranular pore space causes groundwater to flow primarily through fracture networks [14]. Direct numerical simulation (DNS) of such highly heterogeneous medium is usually computationally infeasible, particularly when flow is coupled with thermal, chemical, or mechanical processes. A common remedy is to upscale the numerous small-scale fractures into an equivalent hydraulic conductivity tensor. A comprehensive review of hydraulic conductivity upscaling techniques is provided in [109]. However, the fractal nature of fracture networks, discussed in detail in [95], provides no clear scale-separation threshold to guide this upscaling. To address this challenge, a discrete fracture–matrix (DFM) [16] approach is adopted for representing the bedrock environment and relevant processes. DFM models combine discrete fracture networks (DFN) with an equivalent continuum medium (ECM) representation that accounts for the effects of fractures below the resolution scale.

Given the geological uncertainty, the multilevel Monte Carlo (MLMC) method [43] could provide a computationally efficient framework for estimating statistics from DFM models. The MLMC method relies on constructing a hierarchy of correlated fine- and coarse-grid solution pairs. Through this hybrid description, an arbitrary fracture-size cutoff can be imposed, enabling a smooth transition from full DNS to coarser continuum-only models. When combined with fast and robust numerical homogenization [11], the resulting coarse model yields a low-cost, low-fidelity approximation that enables efficient uncertainty quantification under limited subsurface data. The direct application of the MLMC method to DFM models poses a major challenge: coarse models lack the resolution to capture the influence of small-scale fractures. Their effect must therefore be efficiently and accurately homogenized into the ECM properties of the coarse DFM model. To accelerate numerical homogenization, a surrogate is employed within the MLMC framework.

To the best of the author’s knowledge, this dissertation presents the first application of MLMC to DFM models. The central contribution is the development of a surrogate designed to predict equivalent hydraulic conductivity tensors from 3D DFM models. Tensorial spatial random fields (SRFs) are used for matrix conductivities, while fracture conductivities are permitted to vary with fracture size and aperture. The influence of the matrix-to-fracture conductivity ratio on the distribution of the equivalent hydraulic conductivity tensor, as well as on the predictive

power of the trained surrogates, is investigated. Accuracy is further evaluated on unseen DFM data with varying SRF parameters, fracture counts, and configurations. Surrogate predictions are compared with numerical homogenization results in two macroscale problems: groundwater flow through a cubic volume (Constraint problem) and, the upscaling of the hydraulic conductivity tensor (Anisotropy problem). Although the surrogate is developed as a component of the MLMC framework for DFM models, it is also intended to be applicable as a standalone tool. The trained surrogates are subsequently employed within the proposed MLMC framework for DFM models. The resulting MLMC estimators are analyzed in detail, and their properties are compared against those of the standard Monte Carlo method. The developed code can be found in the MLMC-DFM repository [118].

The thesis addresses two main interrelated objectives:

- **Surrogate for numerical homogenization:** A surrogate model to predict equivalent hydraulic conductivity tensors directly from DFM representations, thereby reducing computational cost.
- **MLMC method for DFM models:** An MLMC framework for DFM models. Within this framework, the computationally expensive homogenization step is replaced by the surrogate model to enable efficient transitions between fine and coarse resolution levels.

The dissertation is structured as follows. Chapter 2 reviews state-of-the-art approaches in Monte Carlo methods for groundwater modeling, numerical homogenization of hydraulic conductivity, and surrogate modeling in hydrogeological applications. In Chapter 3, multiscale DFM models are introduced as a conjunction of the discrete fracture network and equivalent continuum medium approaches. The adopted homogenization procedure for equivalent conductivity tensors is detailed, along with two macroscale test problems. Chapter 4 presents Monte Carlo methods, their multilevel extension (i.e., MLMC), and relevant theoretical foundations. The implemented sampling algorithm, which forms part of the developed MLMC software library [27], is described, and the MLMC strategy for DFM models is formulated. Chapter 5 introduces the deep learning surrogate developed for numerical homogenization. The chapter also presents the training objectives, accuracy metrics, and neural network architectures considered, including graph neural networks explored during the early stages of the Ph.D. study. In Chapter 6, the datasets generated for surrogate training are described, including DFN and matrix parameter configurations, together with preprocessing steps. Chapter 7 outlines the surrogate architecture and evaluates its predictive performance for two loss functions and different DFM configurations - including variations in SRF correlation length, mean, and covariance, as well as different DFN settings. The comparison of upscaled hydraulic conductivity tensors obtained from numerical homogenization and surrogate predictions is also provided for two macroscale problems. Chapter 8 reports the results of MLMC-DFM estimators that employ the trained surrogates for upscaling across levels. Particular emphasis is placed on the investigation of computational efficiency. Finally, Chapter 9 concludes the dissertation.

2 State of the Art

This chapter reviews the current state of research relevant to the approaches adopted in this dissertation. First, Monte Carlo methods are discussed in the context of groundwater flow and transport modeling. Second, numerical homogenization techniques are reviewed, with emphasis on the determination of equivalent hydraulic conductivity in fractured rock media. Finally, surrogate modeling is addressed, with particular focus on deep learning surrogates that can approximate expensive numerical homogenization computations.

2.1 Monte Carlo Methods in Groundwater Modeling

Monte Carlo (MC) methods have been applied across many scientific fields, including groundwater modeling. The first applications to flow in porous medium date back to the early 1960s [116]. More recently, Anderson et al. [7] identified the MC method as a powerful tool for advanced uncertainty analysis and highlighted its wide range of groundwater applications. These include groundwater flow modeling [61, 113, 60], contaminant transport [57], and groundwater recharge estimation [13]. Despite advances in computing power, the need to generate large numbers of MC samples remains a major limitation. To address this, Heinrich [55] and Giles [42, 43] developed the MLMC method, which has enabled more affordable applications in, for instance, groundwater flow [34, 76] and contaminant transport modeling [66, 79, 121].

Several pioneering studies have applied the MLMC method to fracture models. Berrone et al. [17] investigated flow through a discrete fracture network (DFN) composed of rectangular fractures with random sizes. Randomness in fracture geometry led to disconnected networks, causing sharp variations in flux intensity and resulting in non-smooth quantities of interest (QoIs). As the authors note, this lack of smoothness limits the applicability of advanced stochastic strategies, but the MLMC method remains effective. In their approach, MLMC levels were defined by computational meshes of increasing resolution. Pieraccini et al. [87] extended this work to more complex DFNs with disc-shaped fractures and random properties. In their study, fine and coarse models shared the same fracture geometry, differing only in mesh resolution. To date, no publications have introduced procedures for systematically coarsening fractures across MLMC levels, and DFM models have not been coupled with the MLMC method. Most studies also pay little attention to the

software used for implementations of the MLMC method. Typically, each research group develops its own code. While general-purpose packages such as MLMCPy [58] and `pymcmc` [39] exist, they lack essential features such as cluster computing support, efficient sample storage, and robust post-processing capabilities. To overcome these limitations, the in-house Python library [27] has been developed to provide the required functionality (see Section 4.3).

2.2 Numerical Homogenization of Hydraulic Conductivity

Homogenization techniques typically employ a representative elementary volume (REV) to compute the effective hydraulic conductivity of a fractured medium [11]. However, for fracture networks in rocks, the REV concept is not applicable due to their fractal structure [23]. In such cases, the homogenized hydraulic conductivity forms a heterogeneous random field with a correlation length bounded from below. Numerical homogenization can be employed in a block-averaged sense, where the resulting properties might vary depending on the block size, see [32]. Under this interpretation, the terminology is commonly shifted from effective to equivalent properties [96, 91]. Although a distinction between homogenization and upscaling has been made in some studies [38], the terms are used interchangeably in this work. Comprehensive reviews of approaches for determining equivalent hydraulic conductivity in fractured porous medium can be found in [92, 91, 96]

Numerical homogenization has also been investigated specifically in the context of DFM models. Bogdanov et al. [21] considered a cubic rock matrix containing two-dimensional polygonal fractures of uniform size, with the matrix assumed isotropic and homogeneous. Extending earlier work by Koudina et al. [64], they modeled Darcy flow using distinct permeabilities for fractures and matrix, discretized with a tetrahedral mesh. Their results highlighted the influence of fracture density and matrix conductivity on the upscaled tensor. In subsequent work [22], they incorporated fracture sizes distributed according to a power law. Lang et al. [68] determined equivalent conductivity tensors for both 2D and 3D DFM models on unstructured grids, using a pressure gradient with flux averaging. Other notable studies include [69, 12], which further explore numerical homogenization strategies in fracture networks.

2.3 Surrogates in Groundwater Modeling

A surrogate model, also referred to as a metamodel or emulator, is a simplified mathematical function that approximates the dominant behavior of more complex functions, which may be implicit or require evaluation through simulations or experiments. While the primary motivation for employing surrogates is to reduce the computational cost of expensive models, they can also alleviate numerical instabilities, identify non-influential parameters, and improve the interpretability of the

original model (see Asher et al. [10]).

A wide range of surrogate modeling techniques has been developed, each with distinct strengths and trade-offs depending on the application. Polynomial regression is simple and interpretable but limited to smooth, low-dimensional problems. Gaussian process regression (kriging) provides uncertainty quantification and performs well on small datasets, making it a popular choice in geostatistics [41]. Radial basis function (RBF) models yield smooth approximations and scale well in moderate dimensions [28]. Artificial neural networks (ANNs), including deep learning models, are particularly effective for capturing highly nonlinear relationships in high-dimensional data, especially when large training datasets are available [101]. For a more detailed overview of surrogate methods and applications, see Koziel and Leifsson [65].

Surrogate modeling has been widely applied in groundwater and hydrogeological studies, with comprehensive reviews available in the literature [10, 90]. As outlined by Robinson [94] and Asher et al. [10], surrogates can be broadly categorized into three classes: multi-fidelity surrogates (e.g., multigrid and multiscale finite element methods), projection-based surrogates (e.g., proper orthogonal decomposition, Karhunen–Loève expansion), and data-driven surrogates (e.g., RBFs, support vector regression, neural networks). This dissertation focuses on data-driven surrogates. Unlike physics-based approaches, they do not rely on detailed domain knowledge of the modeled process but instead require sufficient input–output data pairs from simulations or measurements. Despite their advantages, data-driven surrogates are not without limitations. As noted by Asher et al. [10], they may be prone to overfitting, convergence to local minima, or difficulties in capturing complex heterogeneities. Recent advances in deep learning architectures, however, have mitigated many of these issues. Deep neural networks (DNNs) have shown remarkable ability to generalize across high-dimensional input spaces and to capture intricate nonlinear relationships [9]. Furthermore, DNNs can accommodate larger datasets and more parameters than traditional surrogate approaches. These advantages motivate the adoption of deep learning-based surrogates in this dissertation.

2.3.1 Deep Learning Surrogate

Deep learning (DL) [45] is a subset of machine learning that employs deep neural network architectures to learn complex representations from data. A comprehensive review of DL is beyond the scope of this work; therefore, the focus is placed on DL techniques most relevant to hydrogeological modeling.

The potential of DL in hydrology has been extensively reviewed by Marcais and de Dreuzy [74], who highlight its applications in prediction, model calibration, and model reduction, while also pointing to challenges such as data preprocessing and the difficulty of capturing spatial heterogeneity. Shen [99] further discusses DL for water modeling, summarizing fundamental concepts and commonly used deep neural network (DNN) architectures, such as multilayer perceptrons, convolutional neural networks, or recurrent neural networks. In recent years, DNNs have been successfully applied to a wide range of groundwater-related tasks, including the pre-

diction of groundwater levels [112], groundwater quality [51, 33, 4], groundwater flow [47, 98, 67, 102], and contaminant transport [114, 121]. These studies demonstrate the increasing relevance of DL-based approaches for modeling complex, nonlinear processes in hydrogeology.

In this dissertation, DNNs are employed to solve regression problems through supervised learning [45], where the neural network learns a mapping between input and output data based on sample pairs obtained from model runs. The input data are defined on an unstructured mesh: 3D elements represent the bulk material, and 2D elements represent fractures, with spatially correlated random fields in the bulk. To capture spatial relationships, convolutional neural networks (CNNs) would be a natural choice. However, CNNs cannot be directly applied to unstructured meshes [111]. Several approaches have been proposed to address this limitation. A basic method applies 1D CNNs to vectors of spatial random field (SRF) values from the mesh, though this has achieved limited success [54]. Wang et al. [108] proposed a novel approach tailored to random fields described by the spectral representation method or the Karhunen–Loève expansion. Another strategy interpolates unstructured data onto a regular grid, enabling CNN use but introducing interpolation errors and often requiring a large number of grid points. Alternatively, the SRF structure can be represented as an undirected graph [103] and processed using graph convolutional neural networks (GCNNs) [49]. Both CNNs [105, 72] and GCNNs [82, 20] have been successfully employed in hydrogeological studies.

Supervised CNNs are widely used for predicting hydraulic conductivity and permeability. For example, 2D CNNs have been applied to images of porous medium to predict permeability or porosity [30, 5, 107]. More realistic 3D CNNs are increasingly popular [89, 56, 86, 104, 70]. Hong et al. [56], for instance, trained four 3D CNNs on binary pore geometry images to predict permeabilities along three axes and the mean permeability. Recently, hybrid CNN-transformer models have also been introduced [75]. CNNs have also been used to upscale DFM models [53, 8], but these studies assumed constant hydraulic conductivity for both matrix and fractures. Such assumptions limit applicability in settings where surrogates must be integrated into stochastic MLMC simulations with variable conductivities. Beyond CNNs, other deep learning models have been proposed for predicting equivalent permeability tensors, including multilayer perceptrons [100, 84], GCNNs [31], and generative adversarial networks (GANs) [40]. After initial experiments with GCNNs, I chose to employ CNNs as the key part of a surrogate model for numerical homogenization in this dissertation. Details of the used deep learning-based surrogate are presented in Chapter 5.

From a software perspective, TensorFlow [2] and PyTorch [85] are the most widely used DL frameworks. For graph neural networks (GNNs), specialized libraries such as the Deep Graph Library (DGL) [106] and Spektral [46] provide advanced tools for modeling graph-structured data, with additional options summarized by Zhou et al. [117, Table B.6].

3 Multiscale Models of Fractured Medium

The primary aim of this chapter is the introduction of homogenization problems, which are later replaced by surrogates. A secondary aim is the contextualization of these homogenization problems within a multiscale description of fractured medium. A simple DFN stochastic model of the distribution of discrete fractures is introduced first. Subsequently, an ECM description of the rock mass by a random hydraulic conductivity tensor field is presented. These descriptions are then combined by means of a DFM approach (see [16] for various descriptions of fractured rock). Finally, homogenization and two macroscale test problems are introduced.

3.1 Discrete Fracture Network

As Darcy flow in crystalline rock is primarily attributed to fractures, discrete fracture network (DFN) models are commonly employed: [73, 24, 36]. The characteristics of fractures in DFNs can include (1) length, (2) aperture, (3) orientation, or (4) density (as well as connectivity), all of which are regarded as random variables.

1. Based on evidence from nature, it is commonly assumed that fracture lengths (f_s) follow a power-law distribution [23]:

$$f_s \sim Cr^{-\alpha}, \quad C = \frac{1 - \alpha}{\bar{r}^{1-\alpha} - \underline{r}^{1-\alpha}}, \quad (3.1)$$

where α represents the characteristic exponent, while $\underline{r}$ and $\bar{r}$ denote the minimum and maximum fracture size.

2. The aperture of a fracture plays a crucial role in determining its hydraulic conductivity. According to Bonnet et al. [23], various distributions such as log-normal, power-law, normal, or Lévy-stable have been utilized to describe the aperture distribution. The correlation between length and aperture has not been observed in all porous media; see [95]. However, for simplification purposes, some studies [52, 68] adopt a constant aperture for all fractures. In the present work, a linear relationship between fracture aperture and fracture size is assumed: $\delta = af_s$.
3. The distribution of orientations [3, section 2.5.2] is frequently modeled using the von Mises distribution for 2D cases or the Fisher distribution for 3D cases.

The probability density function for the Fisher distribution is:

$$f(\mathbf{n}; \mathbf{a}, \kappa) = C_3 \exp(\kappa \mathbf{n} \cdot \mathbf{a}), \quad C_3 = \frac{\kappa}{2\pi(\exp(\kappa) - \exp(-\kappa))},$$

where $k > 0$ is the dispersion parameter, $\mathbf{n}$ is the random fracture normal vector. Mean normal vector $\mathbf{a}$ has items:

$$\mathbf{a}_0 = \sin(\theta) \cos(\phi),$$

$$\mathbf{a}_1 = \cos(\theta) \cos(\phi),$$

$$\mathbf{a}_2 = -\sin(\phi),$$

where $\theta \in (0, 2\pi)$ is trend - the direction of the projection of the normal vector onto the horizontal plane (measured from North) and $\phi \in (0, \pi/2)$ is plunge - angle of the normal from the horizontal plane (0° corresponding to horizontal and 90° to vertical orientation)

4. Fracture density can be defined as the number of fractures per unit volume or area. In the three-dimensional case, it is quantified using the parameter P_{30} . An alternative approach defines fracture density as the total length of fracture traces per unit area in two dimensions, or as the total fracture surface area per unit volume of rock in three dimensions.

For this latter approach, a dimensionless fracture density, as described in [3], is used to quantify the degree of fracture connectivity. It represents the number of fracture centers - that is, the total number of fractures - within a given domain. The dimensionless density for the three-dimensional case with two-dimensional fractures is defined as

$$\rho'_{3D} = \rho_{3D} \langle V_{\text{ex}} \rangle, \quad (3.2)$$

where ρ_{3D} is calculated from the number of fractures N_f within the volume domain V and the excluded volume

$$\langle V_{\text{ex}} \rangle = v_{\text{ex}} \langle R_1 R_2^2 + R_1^2 R_2 \rangle. \quad (3.3)$$

Here, v_{ex} depends on the fracture shape (for example, see [77]). The concepts of excluded area and excluded volume originate from percolation theory [95, ch. 3] and represent the area or volume surrounding a fracture in which the center of another fracture cannot be located, ensuring that the two fractures do not intersect.

This work assumes spatially uncorrelated fractures, with their positions determined by a Poisson process. More general models can be found in [95, 71]. Each fracture is assigned an isotropic hydraulic conductivity according to the cubic law:

$$k_f = \frac{g \rho_w}{12 \mu} \delta^2, \quad (3.4)$$

where $g = 9.81 \text{ m/s}^2$ is the gravitational acceleration, $\rho_w = 1000 \text{ kg/m}^3$ is the water density, and $\mu = 8.9 \times 10^{-4} \text{ Pa} \cdot \text{s}$ is the dynamic viscosity.

3.2 Spatially Correlated Random Field on the Matrix

The DFN approach becomes computationally prohibitive when dealing with a large number of fine-scale fractures. The equivalent continuum medium (ECM) approach, as detailed in [48, 63], offers a solution by representing rock hydraulic properties with a random hydraulic conductivity tensor field $\mathbf{K}(x)$. This spatial random field (SRF) can be characterized either implicitly, through the homogenization of a DFN realization (refer to Section 3.3), or explicitly, by describing its point distribution and correlation structure. In the latter case, the conductivity tensor field is represented as:

$$\mathbf{K} = \mathbf{Q}^T \mathbf{\Lambda} \mathbf{Q}, \quad \mathbf{Q} = \begin{pmatrix} S_x & Y_x & N_x \\ S_y & Y_y & N_y \\ S_z & Y_z & N_z \end{pmatrix}, \quad \mathbf{\Lambda} = \begin{pmatrix} k_x & 0 & 0 \\ 0 & k_y & 0 \\ 0 & 0 & k_z \end{pmatrix},$$

where $\mathbf{Q}$ is a uniformly distributed rotation matrix used to model isotropic unresolved fractures. In order to prescribe the rotation with a correlation structure, the following normalized vector fields are used: $\mathbf{R} = \tilde{\mathbf{R}}/\|\tilde{\mathbf{R}}\|_2$ and $\mathbf{N} = \tilde{\mathbf{N}}/\|\tilde{\mathbf{N}}\|_2$, where both $\tilde{\mathbf{R}} = (\tilde{R}_x, \tilde{R}_y)$ and $\tilde{\mathbf{N}} = (\tilde{N}_x, \tilde{N}_y, \tilde{N}_z)$ have components that are independent Gaussian fields with zero mean, unit variance, and a specified correlation length λ . To calculate $\mathbf{Q}$, an arbitrary vector $\mathbf{N}_{xy}$ in the XY -plane perpendicular to $\mathbf{N}$ is first assumed:

$$\mathbf{N}_{xy} = (-N_y, N_x). \quad (3.5)$$

Then $\mathbf{R}$ is projected onto the local frame defined by $\mathbf{T} = \mathbf{N}_{xy}/\|\mathbf{N}_{xy}\|_2$. Parallel and orthogonal components are computed:

$$\mathbf{R}_p = (\mathbf{R} \cdot \mathbf{T})\mathbf{T}, \quad (3.6)$$

$$\mathbf{R}_o = \mathbf{R} - \mathbf{R}_p. \quad (3.7)$$

Then 2D R_o and R_p components are rotated into 3D:

$$\mathbf{S}_{xy} = \mathbf{R}_p + (\cos \theta)\mathbf{R}_o, \quad (3.8)$$

where $\cos \theta = N_z$. The Z-component is calculated as:

$$\mathbf{S}_z = \|\mathbf{R}_o\| \sin \theta, \quad (3.9)$$

where

$$\sin \theta = \begin{cases} -\|\mathbf{N}_{xy}\|, & \text{if } (N_x, N_y) \cdot (R_x, R_y) > 0, \\ \|\mathbf{N}_{xy}\|, & \text{otherwise.} \end{cases} \quad (3.10)$$

The final transformation matrix is composed of three column vectors: $\mathbf{S}$, $\mathbf{Y} = \mathbf{N} \times \mathbf{S}$, and $\mathbf{N}$.

The principal hydraulic conductivities are defined by the lognormal model $(\log k_x, \log k_y, \log k_z) \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, where $\boldsymbol{\mu}$ is the mean vector and $\boldsymbol{\Sigma}$ is the covariance matrix, and the corresponding Gaussian fields are generated with correlation length λ . All fields are sampled using the GSTools library [80].

3.3 Discrete Fracture–Matrix Model

The discrete fracture–matrix (DFM) approach couples a DFN with a surrounding rock matrix represented by a heterogeneous ECM conductivity field. For details and related concepts, see [97, 18, 25]. The description provided in this section is adapted from the previously published article [122]. In 3D, the matrix occupies a volumetric domain Ω_m , while fractures are represented as embedded 2D surfaces Ω_f , forming a mixed-dimensional mesh combining 3D and 2D finite elements. Within Ω_m , the Darcy flow equation is considered:

$$\operatorname{div} \delta_m \mathbf{u}_m = 0, \quad \mathbf{u}_m = -\mathbf{K}_m \nabla h_m, \quad (3.11)$$

where $\mathbf{u}_m(\mathbf{x})$ denotes the Darcy velocity $[m/s]$ and $\mathbf{K}_m(\mathbf{x})$ is the hydraulic conductivity tensor $[m/s]$. The principal unknown is the piezometric head $h_m [m]$. For consistency with the fracture domain equations, $\delta_m = 1$. The corresponding equation on Ω_f is given by:

$$\operatorname{div} \delta_f \mathbf{u}_f = q^+ + q^-, \quad \mathbf{u}_f = -k_f \nabla h_f, \quad (3.12)$$

where $\delta_f(\mathbf{x}) = a_{f_{s_i}} [m]$ is the aperture of fracture i , and $k_f(\mathbf{x}) = k_f^i$ denotes the isotropic hydraulic conductivity of the fracture fill. The sources $q^\pm$ denote outflows from the two aligned boundaries of Ω_m with normal vectors $\mathbf{n}^\pm$, coupling the fracture and matrix domain through Robin-like boundary conditions:

$$-\mathbf{K}_m \nabla h_m \cdot \mathbf{n}^\pm = q^\pm := \frac{k_f}{2\delta} (h_m^\pm - h_f) \quad \text{on } \Omega_f. \quad (3.13)$$

The system is solved using the Flow123d simulator [26], based on a mixed finite element method.

Two benchmark problems are now introduced. The first benchmark is used to assess the macroscale performance of the surrogate models and the proposed MLMC framework. The second benchmark plays a dual role: it is employed both as a macroscale test case and as the microscale numerical homogenization problem, which is subsequently approximated by a surrogate.

3.3.1 Constraint Problem

Fluid flow is examined through a cubic domain $\Omega = (0, D)^3$. Dirichlet boundary conditions are prescribed by setting the pressure head $h = H$ at $x = 0$ and $h = 0$ at $x = D$. No-flow boundary conditions are applied on the remaining boundaries (the Y and Z faces). The model quantity of interest (QoI) is the total outflow Y from the domain. In particular, Y represents the total volumetric flux through the outlet face at $x = D$.

3.3.2 Anisotropy Problem – Numerical Homogenization

This problem is used to compute an equivalent hydraulic conductivity tensor $\mathbf{K}^{eq}$ for a given discrete fracture–matrix configuration and therefore defines the numerical homogenization operation employed in the MLMC framework.

To obtain the full hydraulic conductivity tensor, three flow problems are solved on the cubic domain $\Omega = (0, D)^3$, with Dirichlet boundary conditions prescribed as $P^1 = x$, $P^2 = y$, and $P^3 = z$, respectively. It is noted, that the choice of boundary conditions can systematically influence the resulting equivalent hydraulic conductivity tensor. Investigating the impact of alternative boundary conditions on the homogenized response is therefore left for future work. A mixed formulation with RT_0 finite elements is employed, by which the velocity vector $\mathbf{u}(e, P^j)$ is obtained on element e for each boundary condition P^j , $j = 1, 2, 3$. The corresponding pressure gradient is then computed as $\nabla h(e, P) = -\mathbf{K}_e^{-1} \mathbf{u}(e, P)$. For a vector quantity $\mathbf{v}(e, P)$ defined on elements, a weighted average is employed:

$$\langle \mathbf{v} \rangle^j = \frac{\sum_{e \in \mathcal{T}} |e| \delta_e \mathbf{v}(e, P^j)}{\sum_{e \in \mathcal{T}} |e| \delta_e}, \quad j = 1, 2, 3. \quad (3.14)$$

With this notation, the components of the symmetric equivalent tensor in Voigt notation,

$$\mathbf{K}^{eq} = (k_{xx}, k_{yy}, k_{zz}, k_{yz}, k_{xz}, k_{xy}),$$

are determined by solving the least-squares system

$$- \begin{bmatrix} \mathbf{A}^1 \\ \mathbf{A}^2 \\ \mathbf{A}^3 \end{bmatrix} (\mathbf{K}^{eq})^T = \begin{bmatrix} \langle \mathbf{u} \rangle^1 \\ \langle \mathbf{u} \rangle^2 \\ \langle \mathbf{u} \rangle^3 \end{bmatrix},$$

where each block $\mathbf{A}^j \in \mathbb{R}^{3 \times 6}$ is defined as

$$\mathbf{A}^j = \begin{bmatrix} \langle \nabla h \rangle_x^j & 0 & 0 & 0 & \langle \nabla h \rangle_z^j & \langle \nabla h \rangle_y^j \\ 0 & \langle \nabla h \rangle_y^j & 0 & \langle \nabla h \rangle_z^j & 0 & \langle \nabla h \rangle_x^j \\ 0 & 0 & \langle \nabla h \rangle_z^j & \langle \nabla h \rangle_y^j & \langle \nabla h \rangle_x^j & 0 \end{bmatrix}.$$

4 Monte Carlo Methods

Monte Carlo methods [43] are statistical techniques used to estimate expectations arising from stochastic models. They have been widely applied across diverse scientific fields, including computational finance, uncertainty quantification, physical process modeling, and environmental engineering. In this chapter, the standard Monte Carlo method and its extension, the multilevel Monte Carlo method, are delineated. The MLMC software library is briefly presented, and the MLMC framework for DFM models is outlined.

Throughout the thesis, $\mathbb{E}[\cdot]$ denotes the expectation with respect to the underlying probability measure, while $\mathbb{V}[\cdot]$ denotes the variance. For a sequence of independent and identically distributed realizations $\{X_n\}_{n=1}^N$ of a random variable X , the numerical (sample) mean is denoted by $\langle X \rangle_N = \frac{1}{N} \sum_{n=1}^N X_n$.

4.1 Standard Monte Carlo Method

Let $P(\omega)$ denote the quantity of interest (QoI), where ω represents a realization of random parameters. In the considered setting, $P(\omega)$ is a random variable that is a solution of a partial differential equation (PDE). Although $P(\omega)$ is not computationally accessible, it is feasible to compute its approximation $P_L(\omega)$, for instance, by a finite element method.

In the standard Monte Carlo method, the expected value $\mathbb{E}[P_L(\omega)]$ is estimated by the sample mean of N independent realizations:

$$Z = \langle P_L(\omega) \rangle_N = \frac{1}{N} \sum_{n=1}^N P_L(\omega_n).$$

Since $\mathbb{E}[Z] = \mathbb{E}[P_L]$, the mean squared error (MSE) of the estimator Z can be decomposed as:

$$\mathbb{E}[(Z - \mathbb{E}[P])^2] = \mathbb{V}[Z] + (\mathbb{E}[Z] - \mathbb{E}[P])^2 \leq \varepsilon^2, \quad (4.1)$$

where ε is the accuracy of the estimator, and the approximation error $(\mathbb{E}[Z] - \mathbb{E}[P])^2$ is assumed negligible. By the central limit theorem, Z is asymptotically normally distributed:

$$Z \sim \mathcal{N}\left(\mathbb{E}[P_L], \frac{\mathbb{V}[P_L]}{N}\right). \quad (4.2)$$

To ensure the accuracy ε of the MC estimator, the following condition must be satisfied:

$$\sqrt{\frac{\mathbb{V}[P_L]}{N}} \leq \varepsilon, \quad (4.3)$$

which implies that $N = O(\varepsilon^{-2})$ samples are required for fixed $\mathbb{V}[P_L]$. Such a requirement may become computationally prohibitive, especially for models involving PDE solvers. To mitigate this drawback, the multilevel Monte Carlo method was developed [43], based on the principles of the control variate method for variance reduction.

Assume two well-correlated random variables f and g , where $\mathbb{E}[g]$ is known and $\mathbb{E}[f]$ is to be estimated. In such a case, g can be employed as a control variate, and an unbiased estimator for $\mathbb{E}[f]$ can be formed as:

$$\langle f(\omega) - \lambda g(\omega) \rangle_N + \lambda \mathbb{E}[g], \quad (4.4)$$

where the optimal coefficient is given by $\lambda = \text{Cov}(f, g) / \mathbb{V}[g]$ and can be estimated from Monte Carlo samples. The principle of the two-level Monte Carlo method is analogous.

By setting $f = P_L$ and $g = P_{L-1}$, where P_{L-1} is a cheaper and less accurate approximation of P_L , and choosing $\lambda = 1$, motivated by the strong correlation between P_L and P_{L-1} , the relation

$$\mathbb{E}[P_L] = \mathbb{E}[P_{L-1}] + \mathbb{E}[P_L - P_{L-1}]$$

is obtained. The expectation $\mathbb{E}[P_{L-1}]$ is not known a priori and must therefore be estimated. The corresponding two-level unbiased estimator then takes the form

$$\langle P_{L-1}(\omega') \rangle_{N_{L-1}} + \langle P_L(\omega) - P_{L-1}(\omega) \rangle_{N_L}, \quad (4.5)$$

where $P_L(\omega_n)$ and $P_{L-1}(\omega_n)$ denote evaluations of the same random realization ω_n . The difference $P_L(\omega_n) - P_{L-1}(\omega_n)$ is expected to have small variance. Consequently, accuracy can be improved by increasing the number N_{L-1} of inexpensive coarse-model samples, while the number N_L of costly fine-model samples can be kept low.

4.2 Multilevel Monte Carlo Method

The multilevel Monte Carlo method is formulated as an extension of the two-level Monte Carlo method by introducing additional levels of less accurate and computationally cheaper models. Formally, let a sequence $P_1, \dots, P_{L-1}$ be assumed to approximate P_L , with accuracy increasing from P_1 to P_L , but at the cost of higher computational effort. The expected value of P_L can then be expressed as:

$$\mathbb{E}[P_L] = \mathbb{E}[P_1] + \sum_{l=2}^L \mathbb{E}[P_l - P_{l-1}]. \quad (4.6)$$

An MLMC estimator of this expectation is given by

$$Z = \langle P_1(\omega^1) \rangle_{N_1} + \sum_{l=2}^L \langle P_l(\omega^l) - P_{l-1}(\omega^l) \rangle_{N_l}, \quad (4.7)$$

where L denotes the number of levels and N_l the number of simulation samples on level l . While the realizations ω_n^l are independent across samples on level l , dependence is retained within each pair of differences $P_l(\omega_n^l) - P_{l-1}(\omega_n^l)$.

In certain cases, it might be advantageous to use different approximations of a fine sample P_l^f and a coarse sample P_l^c . To ensure a valid MLMC estimator, the condition $\mathbb{E}[P_l^f] = \mathbb{E}[P_l^c]$ must hold. Then the estimator has the following form:

$$Z = \langle P_1^f(\omega^1) \rangle_{N_1} + \sum_{l=2}^L \langle P_l^f(\omega^l) - P_{l-1}^c(\omega^l) \rangle_{N_l}. \quad (4.8)$$

The total computational cost C and variance V of the MLMC estimator (4.8) are of particular interest, and can be expressed as

$$C = \sum_{l=1}^L N_l C_l, \quad (4.9)$$

$$V = \sum_{l=1}^L \frac{V_l}{N_l}, \quad (4.10)$$

where C_1, V_1 denote the cost and variance of P_1 , while C_l, V_l represent the cost and variance of the differences $P_l - P_{l-1}$ for $l > 1$.

4.2.1 Properties of MLMC Method

A rigorous analysis of the MLMC estimator was provided by Giles [43, ch. 2], considering bounds of the mean squared error and a geometric sequence of levels.

Theorem 4.2.1 ([43], MLMC Theorem). Suppose that there exist independent estimators Z_l for $l = 1, 2, \dots, L$ each with expected cost C_l and variance V_l , and positive constants $\alpha, \beta, \gamma, c_1, c_2, c_3$ satisfying

$$\alpha \geq \frac{1}{2} \min(\beta, \gamma),$$

such that

1. $|\mathbb{E}[P_l - P]| \leq c_1 2^{-\alpha l},$
2. $\mathbb{E}[Z_l] = \begin{cases} \mathbb{E}[P_0], & l = 0, \\ \mathbb{E}[P_l - P_{l-1}], & l > 0, \end{cases}$
3. $V_l \leq c_2 N_l^{-1} 2^{-\beta l},$

$$4. C_l \leq c_3 2^{\gamma l}.$$

Then there exists a positive constant $c_4 > 0$ such that for any tolerance $\varepsilon < 1$, one can choose a maximum level L and numbers of samples N_l on each level so that the multilevel estimator

$$\hat{Z} = \sum_{l=1}^L \hat{Z}_l, \quad \text{where} \quad \hat{Z}_l = \langle Z_l \rangle_{N_l},$$

satisfies a mean squared error bound

$$\mathbb{E}[(\hat{Z} - \mathbb{E}[P])^2] < \varepsilon^2$$

with computational cost bounded by

$$\mathbb{E}[C] \leq \begin{cases} c_4 \varepsilon^{-2}, & \beta > \gamma, \\ c_4 \varepsilon^{-2} (\log \varepsilon)^2, & \beta = \gamma, \\ c_4 \varepsilon^{-2-(\gamma-\beta)/\alpha}, & 0 < \beta < \gamma. \end{cases}$$

The relation between β and γ determines the asymptotic computational complexity of the MLMC method:

- If $\beta > \gamma$, the dominant cost comes from the low-accuracy levels, and the complexity is optimal, scaling as $\mathcal{O}(\varepsilon^{-2})$.
- If $\beta < \gamma$, the dominant cost is concentrated on the high-accuracy levels, leading to a worse complexity bound.
- In the case $\beta = \gamma$, the cost and variance reduction are evenly distributed across levels, yielding a complexity of order $\mathcal{O}(\varepsilon^{-2}(\log \varepsilon)^2)$.

Bounds of the form $2^{-\beta l}$ for the variance and $2^{\gamma l}$ for the computational cost provide a convenient parametrization of geometric scaling with respect to the level index. The MLMC theorem itself does not rely on a specific refinement strategy.

Presented Monte Carlo methods can also be used to estimate expectations of quantities derived from the originally investigated QoI. However, unlike standard Monte Carlo, the MLMC method cannot directly estimate quantiles or the full probability distribution. Two main approaches have been proposed to enable MLMC-based estimation of distributions. M. Giles et al. [44] developed a method in which MLMC estimates of indicator functions of the QoI are combined with cubic-spline interpolation. Alternatively, C. Bierig and A. Chernov [19] employed the maximum entropy method, which reconstructs the probability distribution from generalized statistical moments estimated by the MLMC method. In particular, Legendre polynomials are used as generalized statistical moments denoted as $\phi^r(x)$, $r = 1, \dots, R$, where R is the total number of moments. A detailed description of the maximum entropy method is beyond the scope of this work. See [15, 19] for a comprehensive study or [120] for a brief introduction.

4.2.2 Optimal Number of Samples

Determining the number of MLMC samples N_l is a necessary part of the MLMC procedure. There are two fundamental strategies. First, the total computational cost is prescribed, and the variance is minimized. Second, the target variance V_t is prescribed, and the total cost is minimized.

Minimization of Computational Cost under a Variance Constraint

The problem of determining the number of samples N_l at each level $l = 1, \dots, L$ is considered so that the total computational cost is minimized

$$C(N_1, \dots, N_L) = \sum_{l=1}^L N_l C_l, \quad (4.11)$$

subject to a prescribed total variance

$$V = \sum_{l=1}^L \frac{V_l}{N_l}. \quad (4.12)$$

This defines a constrained optimization problem of the form

$$\min_{N_1, \dots, N_L > 0} C(N_1, \dots, N_L) \quad \text{subject to } g(N_1, \dots, N_L) = 0,$$

where the constraint function $g(N_1, \dots, N_L) = \sum_{l=1}^L V_l/N_l - V$. A Lagrange multiplier $\lambda \in \mathbb{R}$ is introduced, leading to the Lagrangian

$$\mathcal{L}(N_1, \dots, N_L, \lambda) = \sum_{l=1}^L N_l C_l + \lambda \left(\sum_{l=1}^L \frac{V_l}{N_l} - V \right). \quad (4.13)$$

The stationarity condition with respect to N_l is then obtained as:

$$\frac{\partial \mathcal{L}}{\partial N_l} = C_l - \lambda \frac{V_l}{N_l^2} = 0, \quad l = 1, \dots, L. \quad (4.14)$$

From this condition, the optimal N_l can be expressed as

$$N_l = \sqrt{\frac{\lambda V_l}{C_l}}. \quad (4.15)$$

Substitution of (4.15) into the constraint (4.12) yields the expression for the Lagrange multiplier,

$$\sum_{l=1}^L \frac{V_l}{\sqrt{\lambda V_l / C_l}} = V \quad \implies \quad \sqrt{\lambda} = \frac{\sum_{l=1}^L \sqrt{V_l C_l}}{V}. \quad (4.16)$$

Finally, the optimal number of samples at each level is given by

$$N_l = \sqrt{\frac{V_l}{C_l} \frac{\sum_{l=1}^L \sqrt{V_l C_l}}{V}}. \quad (4.17)$$

For applications involving multiple statistical moments $r = 1, \dots, R$, let V_l^r denote the variance of the r -th moment contribution on level l :

$$V_l^r = \mathbb{V}[\phi_l^r(x) - \phi_{l-1}^r(x)]. \quad (4.18)$$

In practice, the exact values of V_l^r are unknown, so their estimates $\widehat{V}_l^r$ are used. Applying the same optimization procedure to each moment individually gives

$$\widehat{N}_l^r = \sqrt{\frac{\widehat{V}_l^r}{C_l} \frac{\sum_{l=1}^L \sqrt{\widehat{V}_l^r C_l}}{V_r}}, \quad r = 1, \dots, R, \quad (4.19)$$

where V_r is the target variance for the r -th moment. The final allocation of samples is then chosen as the maximum over all moments:

$$\widehat{N}_l = \max_{r=1, \dots, R} \widehat{N}_l^r. \quad (4.20)$$

Minimization of Total Variance under a Cost Constraint

Alternatively, the total computational cost can be fixed

$$C = \sum_{l=1}^L N_l C_l,$$

while the total variance is minimized

$$V = \sum_{l=1}^L \frac{V_l}{N_l}.$$

Then the Lagrangian

$$\mathcal{L}(N_1, \dots, N_L, \lambda) = \sum_{l=1}^L \frac{V_l}{N_l} + \lambda \left(\sum_{l=1}^L N_l C_l - C \right),$$

with the stationarity condition

$$\frac{\partial \mathcal{L}}{\partial N_l} = -\frac{V_l}{N_l^2} + \lambda C_l = 0$$

yields an analogous solution to the previous case

$$N_l = \frac{C \sqrt{\frac{V_l}{C_l}}}{\sum_{l=1}^L \sqrt{V_l C_l}}. \quad (4.21)$$

4.3 MLMC Software Library

The Python software library MLMC [27] has been gradually developed since my bachelor's studies. The most recently released version, 1.0.3 (Oct 21, 2025), provides users with the following main features:

- User-defined simulation with output QoI of user-defined structure. This approach allows for the computation of derived QoIs within the library (developed during Ph.D. study).
- Simulation samples are scheduled iteratively - according to the MLMC sampling algorithm (see Section 4.3.1). V_l and C_l are continuously estimated to correct the optimal N_l .
- Scheduled samples can run in parallel locally or on clusters with the PBS scheduler system.
- Sample data can be stored in HDF5 files, which is particularly beneficial when working with large datasets. Results are post-processed in small chunks that fit into internal memory.
- In addition to estimating expectations of QoI, there are direct methods for calculating moments or a covariance matrix. Also, the algorithm for probability density function reconstruction using the maximal entropy method is available.

The current version of the library provides the capabilities necessary for conducting reliable MLMC experiments.

4.3.1 Sampling Algorithm

The optimal choice of the sample vector $\mathbf{N} = (N_1, \dots, N_L)$, determined according to the procedure described in Section 4.2.2, can be viewed as a function

$$N^{\text{opt}}(\mathbf{V}, \mathbf{C}),$$

depending on the vector of estimated level variances $\mathbf{V} = (\hat{V}_1, \dots, \hat{V}_L)$ and the vector of level costs $\mathbf{C} = (\hat{C}_1, \dots, \hat{C}_L)$ that are computed as the average sample time of all simulations performed on level l . To approximate the optimal number of samples, an iterative procedure is employed, see Algorithm 1.

The algorithm begins with a prescribed initial vector of scheduled samples, $\mathbf{N}_0^s$, chosen as a geometric sequence decreasing from 100 for $l = 1$ to 10 for $l = L$. The initial target vector $\mathbf{N}_0^t$ is then set to $2\mathbf{N}_0^s$. At each iteration i , additional samples are scheduled up to $\mathbf{N}_i^s$. Once at least half of these samples have been computed, the estimates $\mathbf{V}_i$ and $\mathbf{C}_i$ are updated using all available data. Based on these estimates, a new target vector $\mathbf{N}_{i+1}^t$ is computed, and the scheduled vector is adjusted by adding a fraction ν of the difference between the new and old target vectors:

$$\mathbf{N}_{i+1}^s = \mathbf{N}_i^s + \nu(\mathbf{N}_{i+1}^t - \mathbf{N}_i^s).$$

This process is repeated until the relative difference between the target and scheduled sample vectors is below a prescribed tolerance τ on all levels.

Algoritmus 1: Iterative estimation of MLMC sample numbers	
1	$\mathbf{N}_0^s \leftarrow (100, \dots, 10)$
2	$\mathbf{N}_0^t \leftarrow 2\mathbf{N}_0^s$
3	$i \leftarrow 0$
4	while $N_{i,l}^t - N_{i,l}^s > \tau N_{i,l}^s$ <i>for any</i> l do
5	schedule new samples up to $\mathbf{N}_i^s$
6	wait until half of the samples are completed
7	estimate $\mathbf{V}_i, \mathbf{C}_i$ from collected samples
8	$\mathbf{N}_{i+1}^t \leftarrow N^{\text{opt}}(\mathbf{V}_i, \mathbf{C}_i)$
9	$\mathbf{N}_{i+1}^s \leftarrow \mathbf{N}_i^s + \nu(\mathbf{N}_{i+1}^t - \mathbf{N}_i^s)$
10	$i \leftarrow i + 1$
11	end

4.4 MLMC Method for DFM Models

Although the standard Monte Carlo method can be directly applied to DFM models, the application of the multilevel Monte Carlo method requires a consistent treatment of transitions both between fine and coarse models on the same level and across consecutive levels. The strategy developed to enable MLMC for DFM models is described in this section.

For each level $l > 1$, the coarse models (P_{l-1}^c) employ equivalent hydraulic conductivity tensors obtained through block numerical homogenization of the corresponding fine models (P_l^f). This approach was first introduced and validated for two-dimensional DFM models in [119] and has since been extended to three-dimensional DFM models [122].

Within the MLMC framework, the procedure begins at the finest level $l = L$, where the model P_L^f is defined. Its mesh discretization has a maximum element size h and explicitly resolves fractures $\mathcal{F}_{h,D}$ within the size range (h, D) , where D denotes the side length of the cubic domain (D, D, D) . The influence of fractures smaller than h is represented by a realization of a spatially correlated random field of hydraulic conductivity tensors, denoted by $\mathbf{K}_h$ (see Section 3.2). In contrast, the corresponding coarse model (P_{l-1}^c) employs a coarser mesh with a maximum element size H ($H > h$), which explicitly captures fractures $\mathcal{F}_{H,D}$ within the size range (H, D) . The effect of smaller fractures $\mathcal{F}_{h,H}$, which are no longer explicitly represented, is incorporated into the hydraulic conductivity tensor field $\mathbf{K}_H$, which is determined via numerical homogenization solving the microscale Anisotropy problem described in Section 3.3.2.

4.4.1 Upscaling Procedure

To ensure a consistent transition between the fine and coarse models on level l , the following upscaling steps are performed to construct $\mathbf{K}_H$:

1. Center points $\mathbf{x}_l$ of the homogenization blocks are placed within the domain (D, D, D) , forming a regular grid. The homogenization blocks have size $b \times b \times b$ with an overlap of $b/2$, where $b = 1.5H$ by default.
2. A fracture network and the hydraulic conductivity field $\mathbf{K}_h$ are generated on an extended domain of side length $D + b$ to ensure that the homogenization blocks adequately capture boundary regions.
3. For each block, the intersection of fractures $\mathcal{F}_{h,H}$ with the block is determined. A compatible mesh with maximal element size h is constructed, and the field $\mathbf{K}_h$ is interpolated onto this mesh.
4. An equivalent hydraulic conductivity tensor $\mathbf{K}^{eq}$ is calculated for each block to form the field $\mathbf{K}_H(\mathbf{x}) = P_h(\mathbf{x}, \mathcal{F}_{h,H}, \mathbf{K}_h)$, which is subsequently linearly interpolated onto the unstructured coarse grid.

To ensure smooth compatibility with the surrogate model based on convolutional neural networks, $\mathbf{K}_h$ is interpolated from $\mathbf{K}_G$, a hydraulic conductivity SRF generated on a regular grid over the enlarged domain $D + b$. This setup allows voxelization to be performed directly on $\mathbf{K}_G$, thereby avoiding costly mesh generation during surrogate inference.

Figure 4.1 shows an example of $\mathcal{F}_{h,D}$ (left), illustrating the homogenization blocks of size $b = 15$ and their overlap, alongside the corresponding $\mathcal{F}_{H,D}$ (right) for $D = 60$, $H = 10$, and $h = 5$. Large fractures from the surrounding volume intersecting the domain of interest can produce small fragments, which remain visible even on the coarse DFM mesh. To further demonstrate the homogenization effect, Figure 4.2 shows a slice in the ZY -plane through the cube center. The homogenized representation of $\mathcal{F}_{h,H}$ (left, shown in orange) exhibits reduced variability in hydraulic conductivity in the corresponding coarse model (right).

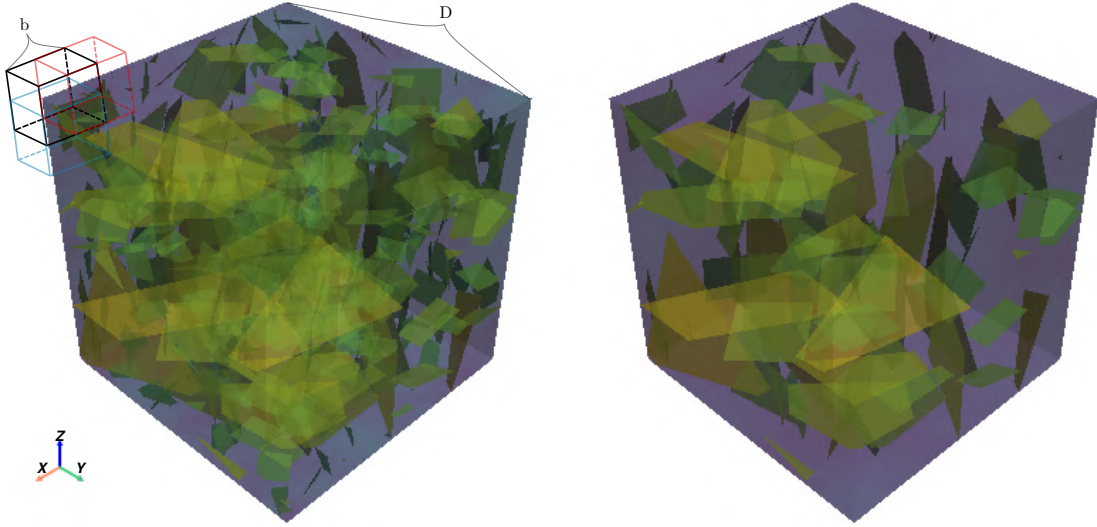


Figure 4.1: Comparison of the fine DFM model fracture network $\mathcal{F}_{h,D}$, with $h \leq 5$ (left), and the corresponding coarse DFM model fracture network $\mathcal{F}_{H,D}$, with $H \leq 10$ (right). The illustration also shows the homogenization blocks of size $b = 15$ and their overlap $b/2$, $D = 60$.

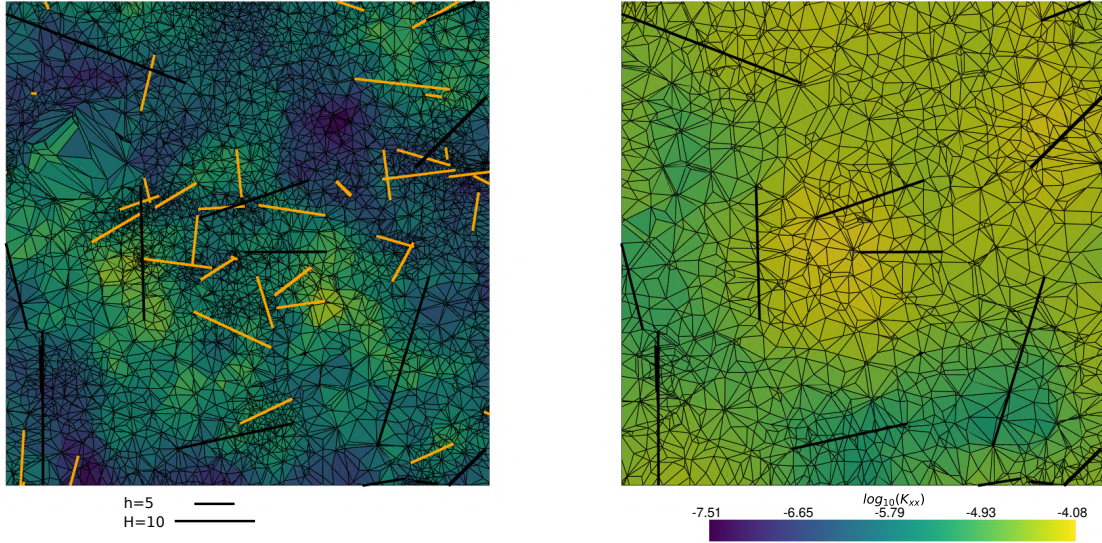


Figure 4.2: **Left:** The fine DFM model ($h \leq 5$). Orange fractures ($\mathcal{F}_{h,H}$) and the hydraulic conductivity tensor field ($\mathbf{K}_h$) are homogenized using overlapping cubic blocks of size $b = 15$. **Right:** Corresponding coarse DFM model ($H \leq 10$) with homogenized hydraulic conductivity tensor component K_{xx} . Notice the reduced range in the upscaled field. Black fractures ($\mathcal{F}_{H,D}$) are not affected by homogenization. Only the K_{xx} component of the hydraulic conductivity tensor is shown.

4.4.2 MLMC Level Transition

In the MLMC framework, fine and coarse discretizations, together with the two-scale models, are employed for each level $l = L, \dots, 2$ as follows:

$$\begin{aligned} P_l^f &= P(h_l, \mathcal{F}_{h_l, D}, \mathbf{K}_{h_l}), \\ P_{l-1}^c &= P(H_l, \mathcal{F}_{H_l, D}, P_{h_l}(\mathcal{F}_{h_l, H_l}, \mathbf{K}_{h_l})), \end{aligned} \quad (4.22)$$

where $H_l = h_{l-1}$. All fracture sets $\mathcal{F}$ and fine tensor fields $\mathbf{K}_{h_l}$ are functions of a common input random state ω .

The upscaling procedure ensures a consistent transition between fine and corresponding coarse samples, starting from the finest level L . In addition to this within-level coupling, the MLMC hierarchy requires the construction of fine-scale models on coarser levels, i.e. P_{l-1}^f . At this stage, fractures smaller than the discretization resolution have already been homogenized, and the hydraulic conductivity field for P_{l-1}^f is therefore generated from the corresponding tensor field $\mathbf{K}_G$, rather than from the original description of the ECM. Two approaches are considered for constructing $\mathbf{K}_G$ on levels $L - 1, \dots, 1$.

- (A) The population $\mathcal{B}_l$ of equivalent hydraulic conductivity tensors $\mathbf{K}_H(\mathbf{x})$ is collected from samples on level l . For practical reasons, the population size is limited to $|\mathcal{B}_l| = 15,000$. The field $\mathbf{K}_G$ for samples on level $l - 1$ is then constructed by drawing tensor values independently from $\mathcal{B}_l$, while retaining the original center points $\mathbf{x}_l$ and extending them as needed to cover the entire homogenization domain on a regular grid. In this approach, no additional spatial correlation structure is imposed; the tensor values are treated as pointwise independent realizations.
- (B) Alternatively, the field $\mathbf{K}_G$ for level $l - 1$ is obtained by rerunning the homogenization procedure for additional samples from level l over a larger domain corresponding to the mesh size of level $l - 1$. With this approach, the condition $\mathbb{E}[P_l^f] = \mathbb{E}[P_l^c]$ is implicitly satisfied, and the samples remain fully independent across levels. However, for multiple levels, this approach requires the SRF to be generated on a sufficiently large domain on the finest level L to accommodate the homogenization block sizes on coarser levels. Moreover, executing the entire hierarchy of homogenization steps across levels is computationally expensive in itself.

For this MLMC-DFM procedure, the sampling algorithm (Algorithm 1) is adjusted to ensure that the populations $\mathcal{B}_l$ are sufficiently large. In particular, the initial scheduled vector $\mathbf{N}_0^s$ is increased according to the number of homogenization blocks used in the given setup. In practice, a total of approximately 10,000 samples in $\mathcal{B}_l$ is considered to be sufficient. The results for MLMC-DFM are presented in Chapter 8.

5 Deep Learning Surrogate for Numerical Homogenization

This chapter focuses on deep learning surrogates for approximating the numerical homogenization procedure. As shown in the previous chapter, obtaining $\mathbf{K}_H(\mathbf{x})$ requires repeated fine-scale finite element simulations over a potentially large number of homogenization blocks. The computational cost of this operation limits the efficiency of MLMC when applied to DFM models. To address this bottleneck, deep learning-based surrogate models are introduced to replace the numerical homogenization step, thereby accelerating the upscaling procedure.

The general surrogate training is described, and an appropriate loss function is derived. Furthermore, the metrics used to assess surrogate accuracy are presented. During the course of my Ph.D. research, the primary focus was on multilayer perceptrons, convolutional, and graph neural network architectures, which are therefore introduced in this chapter.

5.1 Surrogate Training

Let $\mathcal{D} = \{(\mathbf{X}_j, \mathbf{K}_j^{eq})\}_{j=1}^M$ be a dataset of independent and identically distributed samples from the data-generating distribution p_{data} . Here $\mathbf{X}_j \in \mathbb{R}^{6 \times F}$ is a vector of input features (with F representing the number of voxels) and $\mathbf{K}_j^{eq} \in \mathbb{R}^6$ is the vector of components: k_{xx} , k_{yy} , k_{zz} , k_{yz} , k_{xz} , and k_{xy} of the corresponding equivalent hydraulic conductivity tensor $\mathbf{K}^{eq}$. Since supervised learning is carried out, $\mathcal{D}$ is divided into learning (=training) samples $\mathcal{L}$, validation samples $\mathcal{V}$, and test samples $\mathcal{T}$. Although the number of samples in these datasets generally varies, all of them should reasonably cover the distribution p_{data} .

For further use, let $\hat{p}_{data}(\mathbf{X}, \mathbf{K}^{eq})$ denote the empirical data distribution associated with the observed input-output(target) pairs $\{(\mathbf{X}_i, \mathbf{K}_i^{eq})\}_{i=1}^N$. This empirical distribution assigns equal probability mass $1/N$ to each observed sample. The surrogate specifies a conditional probability distribution $p(\mathbf{K}^{eq} | \mathbf{X}, \boldsymbol{\theta})$, where $\boldsymbol{\theta}$ denotes the learnable parameters of the surrogate. This conditional model induces a likelihood function $p(\mathcal{L} | \boldsymbol{\theta})$ for the observed training dataset $\mathcal{L}$.

When $\boldsymbol{\theta}$ is fixed, $p(\mathcal{L} | \boldsymbol{\theta})$ represents a probability distribution. Conversely, when the data $\mathcal{L}$ are fixed, $p(\mathcal{L} | \boldsymbol{\theta})$ is interpreted as the likelihood function. In order to train the surrogate, a point estimate $\hat{\boldsymbol{\theta}}$ of the parameters $\boldsymbol{\theta}$ is computed given the observed data $\mathcal{L}$. The most common approach to obtain this estimate is maximum

likelihood estimation (MLE) [78, p. 107].

5.1.1 Maximum Likelihood Estimation

The maximum likelihood estimate used in neural network training is usually defined in the literature as

$$\hat{\boldsymbol{\theta}} \triangleq \arg \max_{\boldsymbol{\theta}} p(\mathcal{L}|\boldsymbol{\theta}). \quad (5.1)$$

In practice, data $\mathcal{L}$ are provided to a surrogate and the parameters $\boldsymbol{\theta}$ are inferred accordingly: $p(\boldsymbol{\theta}|\mathcal{L})$. The definition of MLE can be reformulated by means of Bayes' theorem:

$$\arg \max_{\boldsymbol{\theta}} p(\boldsymbol{\theta}|\mathcal{L}) = \arg \max_{\boldsymbol{\theta}} \frac{p(\mathcal{L}|\boldsymbol{\theta})p(\boldsymbol{\theta})}{p(\mathcal{L})}. \quad (5.2)$$

Since the probability of the data $p(\mathcal{L})$ is independent of $\boldsymbol{\theta}$, the term $p(\mathcal{L})$ can be omitted without affecting the estimate of $\boldsymbol{\theta}$. In the absence of prior knowledge regarding the parameters $\boldsymbol{\theta}$, the estimate can be considered independent of $p(\boldsymbol{\theta})$. A uniform prior $p(\boldsymbol{\theta}) \propto 1$ is thereby assumed; for details, see [78, p. 109]. The resulting formulation is

$$\arg \max_{\boldsymbol{\theta}} p(\boldsymbol{\theta}|\mathcal{L}) = \arg \max_{\boldsymbol{\theta}} p(\mathcal{L}|\boldsymbol{\theta}). \quad (5.3)$$

Under the assumption of i.i.d. data, the likelihood can be factorized as a product of conditional probabilities. Since the input features $\mathbf{X}_i$ are treated as given and are not parameterized by $\boldsymbol{\theta}$, the likelihood depends on $\boldsymbol{\theta}$ only through the conditional distribution $p(\mathbf{K}^{eq} | \mathbf{X}, \boldsymbol{\theta})$:

$$\hat{\boldsymbol{\theta}} \triangleq \arg \max_{\boldsymbol{\theta}} p(\mathcal{L}|\boldsymbol{\theta}) = \arg \max_{\boldsymbol{\theta}} \prod_{i=1}^N p(\mathbf{K}_i^{eq} | \mathbf{X}_i, \boldsymbol{\theta}). \quad (5.4)$$

For practical reasons, the direct computation of such products is typically avoided, as finite-precision representations of small numbers in computers lead to numerical instability. To circumvent this, the logarithm is applied, which does not affect the position of the desired maximum value:

$$\hat{\boldsymbol{\theta}} = \arg \max_{\boldsymbol{\theta}} \log p(\mathcal{L}|\boldsymbol{\theta}) = \arg \max_{\boldsymbol{\theta}} \sum_{i=1}^N \log p(\mathbf{K}_i^{eq} | \mathbf{X}_i, \boldsymbol{\theta}). \quad (5.5)$$

Since optimization algorithms are generally formulated as minimization procedures, the negative log-likelihood is commonly employed:

$$\hat{\boldsymbol{\theta}} = \arg \min_{\boldsymbol{\theta}} -\log p(\mathcal{L}|\boldsymbol{\theta}) = \arg \min_{\boldsymbol{\theta}} \sum_{i=1}^N -\log p(\mathbf{K}_i^{eq} | \mathbf{X}_i, \boldsymbol{\theta}). \quad (5.6)$$

MLE from the Perspective of Information Theory

To gain more intuition about why maximum likelihood estimation (MLE) is the natural choice, it is useful to look at it through the lens of information theory. For this, three central concepts are recalled [45]:

- **Entropy:** For a random variable $Z \sim p(z)$, the entropy is defined as

$$\mathbb{H}(p) = \mathbb{E}_{z \sim p}[-\log p(z)],$$

and measures the intrinsic uncertainty in Z .

- **Cross-entropy:** For two distributions $p(z)$ and $q(z)$, the cross-entropy is

$$\mathbb{H}(p, q) = \mathbb{E}_{z \sim p}[-\log q(z)],$$

which quantifies the discrepancy between p and q .

- **Kullback–Leibler (KL) divergence:** Defined as

$$D_{\text{KL}}(p||q) = \mathbb{E}_{z \sim p} \left[\log \frac{p(z)}{q(z)} \right],$$

represents the divergence from q to p , which is always non-negative and equal to zero if and only if $p = q$, but is not symmetric.

Recall that the log-likelihood of the data is given by

$$\ell(\boldsymbol{\theta}) = \sum_{i=1}^N \log p(\mathbf{K}_i^{eq} | \mathbf{X}_i, \boldsymbol{\theta}).$$

Normalizing by N allows this expression to be interpreted as an expectation with respect to the empirical distribution $\hat{p}_{data}$:

$$\hat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} \mathbb{E}_{(\mathbf{K}^{eq}, \mathbf{X}) \sim \hat{p}_{data}} [-\log p(\mathbf{K}^{eq} | \mathbf{X}, \boldsymbol{\theta})].$$

Thus, MLE corresponds to minimizing the cross-entropy

$$\mathbb{H}(\hat{p}_{data}(\mathbf{K}^{eq} | \mathbf{X}), p(\mathbf{K}^{eq} | \mathbf{X}, \boldsymbol{\theta})).$$

Cross-entropy can be decomposed as

$$\mathbb{H}(\hat{p}_{data}, p_{\boldsymbol{\theta}}) = D_{\text{KL}}(\hat{p}_{data} || p_{\boldsymbol{\theta}}) + \mathbb{H}(\hat{p}_{data}),$$

where the second term $\mathbb{H}(\hat{p}_{data})$ depends only on the data and is independent of $\boldsymbol{\theta}$. Consequently,

$$\hat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} D_{\text{KL}}(\hat{p}_{data} || p_{\boldsymbol{\theta}}).$$

In other words, maximum likelihood estimation identifies the surrogate distribution $p(\cdot | \hat{\boldsymbol{\theta}})$ that is, in terms of KL divergence, as close as possible to the empirical distribution of the observed data, see [78, p. 109].

5.1.2 Loss Function as MLE

Although in principle neural networks aim to minimize the Kullback–Leibler divergence between the empirical data distribution and the surrogate distribution, in practice, the exact data distribution is unknown and only samples are available. This leads to the common use of other loss functions derived from maximum likelihood estimation (MLE). A loss function is a criterion that the training algorithms of neural networks aim to minimize in order to improve surrogate accuracy. In general, the choice of loss function reflects assumptions about the underlying data distribution, with cross-entropy corresponding to categorical distributions (for classification tasks) and mean squared error to Gaussian-distributed errors (for regression tasks) [78].

In the regression task considered here, no probability distribution, as assumed by MLE, is predicted directly by the surrogate. Instead, six values of $\mathbf{K}_i^{eq}$ are produced by a surrogate for a single input. To connect this formulation with MLE, the outputs of the surrogate

$$\mathbf{K}^{eq} = f(\mathbf{X}, \boldsymbol{\theta})$$

are assumed to follow a normal distribution by introducing Gaussian noise with fixed, nonzero covariance matrix $\boldsymbol{\Sigma} \in \mathbb{R}^{6 \times 6}$ across samples:

$$\mathbf{K}_i^{eq} = f(\mathbf{X}_i, \boldsymbol{\theta}) + \eta_i, \quad \eta_i \sim \mathcal{N}(0, \boldsymbol{\Sigma}). \quad (5.7)$$

After substituting the probability density function of the multivariate normal distribution into the MLE formulation, the following expression is obtained:

$$\begin{aligned} \arg \max_{\boldsymbol{\theta}} p(\mathcal{L}|\boldsymbol{\theta}) &= \operatorname{argmin}_{\boldsymbol{\theta}} \sum_{i=1}^N -\log p(\mathbf{K}_i^{eq}|\mathbf{X}_i, \boldsymbol{\theta}) \\ &= \operatorname{argmin}_{\boldsymbol{\theta}} \sum_{i=1}^N -\log \left[\frac{1}{(2\pi)^3 |\boldsymbol{\Sigma}|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{K}_i^{eq} - f(\mathbf{X}_i, \boldsymbol{\theta}))^T \boldsymbol{\Sigma}^{-1} (\mathbf{K}_i^{eq} - f(\mathbf{X}_i, \boldsymbol{\theta})) \right) \right] \\ &= \operatorname{argmin}_{\boldsymbol{\theta}} N \log(2\pi)^3 + N \frac{1}{2} \log |\boldsymbol{\Sigma}| + \sum_{i=1}^N \frac{1}{2} (\mathbf{K}_i^{eq} - f(\mathbf{X}_i, \boldsymbol{\theta}))^T \boldsymbol{\Sigma}^{-1} (\mathbf{K}_i^{eq} - f(\mathbf{X}_i, \boldsymbol{\theta})) \end{aligned} \quad (5.8)$$

After omission of constants with respect to $\boldsymbol{\theta}$:

$$\arg \max_{\boldsymbol{\theta}} p(\mathcal{L}|\boldsymbol{\theta}) = \operatorname{argmin}_{\boldsymbol{\theta}} \sum_{i=1}^N (\mathbf{K}_i^{eq} - f(\mathbf{X}_i, \boldsymbol{\theta}))^T \boldsymbol{\Sigma}^{-1} (\mathbf{K}_i^{eq} - f(\mathbf{X}_i, \boldsymbol{\theta})) \quad (5.9)$$

Two scenarios can be distinguished:

1. If the covariance matrix $\boldsymbol{\Sigma}$ is diagonal, then

$$\arg \max_{\boldsymbol{\theta}} p(\mathcal{L}|\boldsymbol{\theta}) = \operatorname{argmin}_{\boldsymbol{\theta}} \sum_{i=1}^N \sum_{k=1}^6 \frac{(\mathbf{K}_{i,k}^{eq} - f(\mathbf{X}_i, \boldsymbol{\theta})_k)^2}{\sigma_k^2}. \quad (5.10)$$

If all variances σ_k^2 are identical, this reduces to the standard mean squared error (MSE):

$$\begin{aligned}\arg \max_{\boldsymbol{\theta}} p(\mathcal{L}|\boldsymbol{\theta}) &= \arg \min_{\boldsymbol{\theta}} \sum_{i=1}^N \sum_{k=1}^6 \frac{1}{\sigma^2} (\mathbf{K}_{i,k}^{eq} - f(\mathbf{X}_i, \boldsymbol{\theta})_k)^2 \\ &= \arg \min_{\boldsymbol{\theta}} \frac{1}{N} \sum_{i=1}^N \sum_{k=1}^6 (\mathbf{K}_{i,k}^{eq} - f(\mathbf{X}_i, \boldsymbol{\theta})_k)^2.\end{aligned}\tag{5.11}$$

Since only the estimate $\hat{\boldsymbol{\theta}}$ is of interest, the factor $\frac{1}{\sigma^2}$ can be omitted, and $\frac{1}{N}$ can be included to yield the familiar MSE formulation. Thus, the MSE loss function is minimized during training.

2. For a general covariance matrix $\boldsymbol{\Sigma}$,

$$\arg \max_{\boldsymbol{\theta}} p(\mathcal{L}|\boldsymbol{\theta}) = \arg \min_{\boldsymbol{\theta}} \frac{1}{N} \sum_{i=1}^N (\mathbf{K}_i^{eq} - f(\mathbf{X}_i, \boldsymbol{\theta}))^T \boldsymbol{\Sigma}^{-1} (\mathbf{K}_i^{eq} - f(\mathbf{X}_i, \boldsymbol{\theta})).\tag{5.12}$$

Here, the term

$$(\mathbf{K}_i^{eq} - f(\mathbf{X}_i, \boldsymbol{\theta}))^T \boldsymbol{\Sigma}^{-1} (\mathbf{K}_i^{eq} - f(\mathbf{X}_i, \boldsymbol{\theta}))$$

represents the squared Mahalanobis distance, which quantifies the deviation of the predicted output $f(\mathbf{X}_i, \boldsymbol{\theta})$ from the true output $\mathbf{K}_i^{eq}$, while accounting for the covariance structure $\boldsymbol{\Sigma}$.

MLE can also be derived from a generative perspective, see [78, p. 378]. In certain cases, exact computation of the MLE is not feasible, either due to computational complexity or due to incomplete data availability. In such situations, alternative estimation methods may be employed [78, p. 117], such as the method of moments or online estimation.

The whole training procedure consists of two phases. In the forward pass, the input data are propagated through the network to generate predictions. In the backward pass, also referred to as backpropagation, the gradient of the loss function with respect to the network's trainable parameters is computed using the chain rule of differentiation. Subsequently, the trainable parameters (weights and biases) are updated by means of an optimization algorithm. Stochastic gradient descent (SGD) and its adaptive variants, such as Adam [62], are commonly employed; for others, see [45]. It is important to note that minimizing the loss function on the training data leads to overfitting, where the surrogate performs very well on the training set but fails to generalize to unseen data. To mitigate this issue, the loss function is also evaluated on a separate validation set, and its value typically determines the best $\hat{\boldsymbol{\theta}}$. Additional forms of regularization can also be applied.

5.1.3 Regularization

Regularization refers to any technique that improves the generalization ability of a neural network, i.e., its performance on data not used during training. Among the most widely used techniques, and those tested in the presented work, are (see [45, ch. 7] for their details):

- **L1/L2 regularization:** penalization of the magnitude of learnable weights, implemented as a parameter of the optimizer.
- **Data augmentation:** artificial expansion of the dataset to improve generalization.
- **Early stopping:** termination of training once the validation error ceases to improve, thereby preventing the model from overfitting the training data.
- **Dropout:** reduction of overfitting by randomly deactivating neurons during training, thereby preventing co-adaptation, i.e., the reliance of groups of neurons on each other to produce meaningful outputs rather than learning robust, independent features.
- **Batch/Layer normalization:** stabilization of training by normalizing intermediate layer activations to have zero mean and unit variance. This improves numerical stability, reduces sensitivity to parameter scaling, and often leads to better generalization, and is therefore commonly regarded as a form of implicit regularization.

5.1.4 Activation Functions

Activation functions play a pivotal role in artificial neural networks. They introduce non-linearity into the surrogate, enabling neural networks to approximate complex functions and learn intricate patterns from data. Following the presentation in Goodfellow et al. [45], several activation functions are commonly employed.

The Rectified Linear Unit (ReLU) is defined as

$$f(x) = \max(0, x).$$

ReLU is considered computationally efficient and is known to alleviate the vanishing gradient problem. For this reason, it has become the default activation function in many CNN architectures. To avoid the “dying ReLU” issue, in which some neurons remain inactive, the Leaky ReLU was proposed. It introduces a small slope in the negative region:

$$f(x) = \begin{cases} x & \text{if } x > 0, \\ \alpha x & \text{otherwise.} \end{cases}$$

The Parametric ReLU (PReLU) extends this approach by allowing the slope α in the negative region to be learned during training. While the gradient with respect to

α vanishes for positive inputs, learning α provides additional flexibility for negative activations and helps reduce the risk of permanently inactive neurons.

Historically important activation functions include the sigmoid and the hyperbolic tangent (tanh). The sigmoid function is given by

$$f(x) = \frac{1}{1 + e^{-x}},$$

while the hyperbolic tangent is expressed as

$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}.$$

Both functions, however, are less frequently employed in modern CNNs due to saturation effects and vanishing gradients. More recently, the Swish activation function has been introduced. It is defined as

$$f(x) = x \cdot \sigma(\beta x),$$

where σ denotes the sigmoid function. Empirical studies have shown that Swish can outperform ReLU in certain deep architectures [88].

5.1.5 Surrogate Accuracy Assessment

During the training process, the accuracy of the surrogate's predictions is evaluated by a loss function, the mean squared error (MSE): $\gamma(\mathcal{M}, f) = \frac{1}{|\mathcal{M}|} \|\mathbf{K}_{\mathcal{M}}^{eq} - f(\mathbf{X}_{\mathcal{M}}, \boldsymbol{\theta})\|_2^2$, where $\mathcal{M} \subseteq \mathcal{D}$. Training loss $\gamma(\mathcal{L}, f)$ and validation loss $\gamma(\mathcal{V}, f)$ are observed.

For the sake of expressing the accuracy of surrogates, the coefficient of determination R^2 is often used. R^2 is calculated for each component $\mathbf{K}_i^{eq}$:

$$R_i^2 = 1 - \frac{\sum_{\mathcal{M}} (\mathbf{K}_i^{eq} - f(\mathbf{X}_{\mathcal{M}}, \boldsymbol{\theta}))^2}{\sum_{\mathcal{M}} (\mathbf{K}_i^{eq} - \overline{\mathbf{K}_{i\mathcal{M}}^{eq}})^2}, \quad (5.13)$$

where $\overline{\mathbf{K}_{i\mathcal{M}}^{eq}}$ is the average of $\mathbf{K}_i^{eq}$ over the dataset $\mathcal{M}$.

Generally, the closer R^2 is to 1, the better the surrogate. When $R^2 \leq 0$, the surrogate predictions are not better than the mean of the targets. It is necessary to be aware that R^2 is variance-sensitive and suffers from outliers. For this reason, I prefer to report the normalized root mean squared error (NRMSE), which expresses the prediction error relative to the variability in the target data. For component $\mathbf{K}_i^{eq}$:

$$\text{NRMSE}_i = \frac{\sqrt{\frac{1}{|\mathcal{M}|} \sum_{\mathcal{M}} (\mathbf{K}_i^{eq} - f(\mathbf{X}_{\mathcal{M}}, \boldsymbol{\theta}))^2}}{\sqrt{\frac{1}{|\mathcal{M}|} \sum_{\mathcal{M}} (\mathbf{K}_i^{eq} - \overline{\mathbf{K}_{i\mathcal{M}}^{eq}})^2}}, \quad (5.14)$$

where the numerator is the root mean squared error and the denominator is the empirical standard deviation of the target values over the set $\mathcal{M}$. This normalization makes NRMSE meaningful for comparisons between different components and datasets with varying output variances.

The NRMSE, normalized by the standard deviation of the target values, is closely related to the coefficient of determination:

$$R_i^2 = 1 - \text{NRMSE}_i^2. \quad (5.15)$$

Both metrics quantify predictive accuracy, but NRMSE directly expresses the average error relative to the spread of the data, while R^2 expresses the proportion of variance explained by the surrogate model. For further analysis, the average NRMSE across $\mathbf{K}^{eq}$ components is calculated:

$$\overline{\text{NRMSE}} = \frac{1}{6} \sum_i \text{NRMSE}_i. \quad (5.16)$$

5.2 Multilayer Perceptrons

Although various types of neural networks can be employed with the described learning procedure, multilayer perceptrons and convolutional neural networks are selected as suitable architectures for the surrogate task. These types of neural networks are examined in detail below.

A multilayer perceptron (MLP), representing a class of feedforward neural networks, with L layers, is a fundamental architecture in deep learning. It can be defined by the recursive relation:

$$h^{(l+1)} = \sigma_l(W^{(l)}h^{(l)} + b^{(l)}),$$

where $h^{(l)}$ denotes features at layer l (with $h^{(1)}$ being the network input), $W^{(l)}$ represents the weight matrix, $b^{(l)}$ is the bias vector, and $\sigma_l(\cdot)$ is an activation function.

5.3 Convolutional Neural Networks

Convolutional neural networks [45, ch. 9] (CNNs) process data on regular grids, particularly images. Numerous CNN architectures [6] have recently been developed for tasks such as image classification, segmentation, natural language processing, and also for 3D applications such as medical imaging and volumetric data analysis. CNNs utilize two fundamental operations: convolution and pooling. Popular software libraries, such as TensorFlow [2] and PyTorch [85], use the term convolution to refer to cross-correlation.

Consider a 3D voxelized volume V of shape $X \times Y \times Z \times D_{\text{in}}$, where D_{in} denotes the number of input channels (e.g., RGB channels for images). Let $\mathcal{W}$ be a 3D convolutional kernel of shape $F_h \times F_w \times F_d \times D_{\text{in}} \times D_{\text{out}}$, where D_{out} denotes the number of output channels. Let s represent the stride along each axis, and p the padding applied to the input volume. The output shape of the convolution is given by:

$$O_x = \left\lfloor \frac{X + 2p - F_h}{s} \right\rfloor + 1, \quad O_y = \left\lfloor \frac{Y + 2p - F_w}{s} \right\rfloor + 1, \quad O_z = \left\lfloor \frac{Z + 2p - F_d}{s} \right\rfloor + 1.$$

The cross-correlation operation can then be expressed as:

$$O(x, y, z, d_{\text{out}}) = (V * \mathcal{W})(x, y, z, d_{\text{out}}) = \sum_m \sum_n \sum_l \sum_{d_{\text{in}}} V(xs - m + p, ys - n + p, zs - l + p, d_{\text{in}}) \mathcal{W}(m, n, l, d_{\text{in}}, d_{\text{out}}). \quad (5.17)$$

After convolution, features extracted from the data are passed through a non-linear activation function (e.g., ReLU). Pooling is often applied to enhance computational efficiency and reduce the spatial dimensions of the data. A pooling operation typically applies a summary statistic (e.g., max, average, or sum) over small regions of each channel of O (e.g., $2 \times 2 \times 2$, $4 \times 4 \times 4$, etc.). Consider a pooling layer with a kernel of size $Q_h \times Q_w \times Q_d$ (height, width, depth) and stride s_q . The output shape of the pooling operation is:

$$O'_x = \left\lfloor \frac{O_x - Q_h}{s_q} \right\rfloor + 1, \quad O'_y = \left\lfloor \frac{O_y - Q_w}{s_q} \right\rfloor + 1, \quad O'_z = \left\lfloor \frac{O_z - Q_d}{s_q} \right\rfloor + 1,$$

where O'_x, O'_y, O'_z are the output dimensions after pooling.

The number of learnable parameters (i.e., kernel size) typically varies across convolutional layers, with the number of channels often increasing. It is possible to think of each channel as responding to some different set of features, so different channels could become specialized to recognize different objects, as described by Zhang et al. [115]. A key advantage of convolutional neural networks (CNNs) lies in parameter sharing. At each layer, the convolutional kernel $\mathcal{W}$ employs a single set of parameters that are shared across all spatial locations. This property renders CNNs more efficient than traditional fully connected neural networks (see [45, p. 328]).

CNNs are further characterized by shift invariance, i.e., the ability to recognize patterns independently of their position in the input space. This property arises from the convolution operation, in which the same filter (kernel) is applied across the entire input grid. Consequently, if an object is shifted within an image, the same features can still be detected by the network, albeit at different spatial locations. However, CNNs are not permutation invariant: if the order of the input grid or image is altered (e.g., by swapping rows or columns), their functionality is compromised, since the convolution operation presupposes a fixed grid structure.

5.4 Graph Convolutional Neural Networks

During the first two years of my Ph.D. studies, I primarily focused on surrogate models based on graph convolutional neural networks (GCNNs) - a class of deep neural networks that apply convolution operations in the graph domain (e.g., representations of unstructured meshes). GCNNs are typically categorized according to whether the convolution is defined in the spatial or spectral domain. To enable the use of GCNNs, the dual graph $G = (V, E)$ of a computational mesh is constructed. The vertex set V corresponds to mesh elements and might be associated with the

values of an input random field. The edge set E encodes mesh connectivity, with edges connecting pairs of adjacent elements.

In this regard, two papers were published. The first, a conference paper [120], compared support vector regression, ChebNet GCNN, and GraphSage GCNN as surrogates for a modified aquifer problem. The second, a journal article [121], proposed and tested a surrogate model combining a ChebNet GCNN with a multilayer perceptron for a simplified contaminant transport problem. While these surrogates achieved a reduction in MLMC computational cost, several challenges were also encountered. The graph convolutional neural network (GCNN) framework and the considerations that ultimately led to the adoption of convolutional neural networks (CNNs) for the problems addressed in this work are described below.

Spectral Graph Convolutional Neural Networks

Spectral GCNNs conduct convolution in the spectral domain. A graph signal (a vector of all vertex values [83]) is transformed to the spectral domain and then back to the spatial domain by means of the graph Fourier transform. W. Hamilton [49] provides details of the whole procedure, including defining graph convolution, etc. The following lines describe the basics.

Let A be an adjacency matrix of graph G , $g \in \mathbb{R}^{|V|}$ be a graph signal. Let L be the normalized Laplacian matrix

$$L = I - D^{-1/2}AD^{-1/2}, \quad (5.18)$$

where D is the diagonal matrix of degrees of the vertices. L is a symmetric real-valued positive semi-definite matrix that can be factorized into a matrix of eigenvectors U and a diagonal matrix of eigenvalues Λ such that $L = U\Lambda U^T$. L can be viewed as a discrete analog of the Laplace operator. The eigenfunctions of the general Laplace operator correspond to the modes of the frequency domain in the Fourier transform. In a graph-based notation, eigenvectors U are considered as graph Fourier modes, and g can be transformed from the spatial domain to the spectral domain by the graph Fourier transform

$$F(g) = U^T g, \quad (5.19)$$

and from the spectral domain to the spatial domain by the inverse graph Fourier transform

$$F^{-1}(g) = Ug. \quad (5.20)$$

Thus, the graph convolution of g and a filter h has the following form:

$$g *_G h = F^{-1}(F(g) \odot F(h)) = U(U^T g \odot U^T h), \quad (5.21)$$

where $*_G$ is the convolution operator on the graph G , $U^T h$ is a spectral filter and $\odot$ stands for the element-wise Hadamard product. Then the graph convolution for the spectral filter based on the eigenvalues of L can be written as follows:

$$g *_G h = (Up_N(\Lambda)U^T)g = p_N(L)g, \quad (5.22)$$

where the spectral filter $p_N(L)$ is a degree N polynomial of the Laplacian L . Coefficients of p_N are learnable parameters. Different $p_N(L)$ have been published in recent years [49, p. 85]. For instance, Defferrard et al. [37] introduced the popular ChebNet GCNN, where Chebyshev polynomials $T_k(x)$ form the filter

$$p_N(L) = \sum_{k=0}^{K-1} \theta_k T_k(L), \quad (5.23)$$

where $\theta_k \in \mathbb{R}$ is the learnable Chebyshev coefficient. Using the degree k polynomial, the filter modifies the information at a particular vertex based on the information from the vertices in its k -hop neighborhood.

Spatial Graph Convolutional Neural Networks

Spatial GCNNs [35] conduct convolution directly in the graph domain. The representation of the vertices is propagated and aggregated between adjacent vertices. The basic idea is based on the message-passing neural networks [49, section 5.1]. The spatial graph convolution is defined as follows [110, p. 12]:

$$h_v^{(k)} = U_k(h_v^{(k-1)}, \sum_{u \in N(v)} M_k(h_v^{(k-1)}, h_u^{(k-1)}, x_{vu}^e)), \quad (5.24)$$

where $h_v^{(k)}$ represents features of the vertex v at k -th message-passing iteration, x_{vu}^e is the optional feature vector of the edge (v, u) . U_k and M_k are update and message functions with learnable parameters, $N(v)$ is the set of vertices adjacent to v . During each iteration of message passing k , $\forall v \in V$: features $h_v^{(k)}$ are updated according to the information aggregated from adjacent vertices of the vertex v and features from the previous iteration $h_v^{(k-1)}$.

In recent years, several spatial GCNNs have been proposed [110, Table 3]. They mainly differ in operations for aggregation and updating of vertex features. For instance, GraphSage, proposed by W. Hamilton et al. [50], can be applied to huge graphs. Instead of using all neighboring vertices $N(v)$, a subset is randomly selected $S_{N(v)} \subseteq N(v)$. The following message-passing procedure is conducted:

$$h_v^{(k)} = \sigma(W^{(k)} \cdot f_k(h_v^{(k-1)}, \{h_u^{(k-1)} \mid u \in S_{N(v)}\})), \quad (5.25)$$

where σ represents a nonlinear activation function, $W^{(k)}$ is the weight matrix at iteration k , f_k is the aggregation function (e.g., avg, sum, max).

According to the comparison made by Wu et al. [110, p. 13], spatial GCNNs are usually preferred over spectral GCNNs. Spatial GCNNs can perform computations on a subset of vertices instead of the entire graph, which yields better scalability for large graphs than spectral GCNNs that assume a fixed graph. Many GCNNs have public code repositories (see [117, Table B.7]) which facilitate testing.

Multilayer perceptrons and convolutional neural networks can be regarded as special cases of graph neural networks, as seen in [81, 110]. However, graph neural networks are generally permutation invariant, as they aggregate information from

neighboring nodes regardless of the order in which the nodes are presented. This property enables GNNs to process graph data that is unordered or permuted, provided that the underlying graph structure remains intact.

Permutation invariance, however, was found to pose a challenge for the presented application. When mesh elements or voxels are permuted, the corresponding hydraulic conductivity tensor may change, meaning that the network must be sensitive to the ordering of the nodes. Experiments conducted with GCNNs yielded only mediocre results. Although the inclusion of spatial coordinates in vertex representations can partially alleviate this limitation, convolutional neural networks, which do not exhibit permutation invariance by default, emerge as a natural first choice for the presented application.

6 Datasets

This chapter presents the datasets of homogenization block pairs $(\mathbf{X}, \mathbf{K}^{eq})$ employed for surrogate training and prediction performance assessment. First, the procedure used to construct the datasets is described. Subsequently, the datasets are analyzed, and appropriate preprocessing strategies are proposed.

6.1 Dataset Configuration

In principle, the training data could be generated by following the full multilevel procedure described in Section 4.4, including the construction of extended domains and overlapping homogenization blocks. However, this is not required for surrogate training, since the numerical homogenization operator P_h acts locally on individual blocks. Instead, each dataset sample is generated by applying P_h independently on a fixed cubic domain $\Omega_0 = (0, 15)^3$, which directly represents a single homogenization block. This simplified construction is technically more straightforward, while remaining fully consistent with the definition of P_h and sufficient for learning the local block-to-tensor mapping performed by numerical homogenization.

The discrete fracture network (DFN) realizations used for dataset generation are parameterized using statistical characteristics inferred from field observations reported in [59, p. 23]. For the training dataset, the DFN parameters summarized in Table 6.1 are adopted. Fracture sizes follow a power-law distribution as introduced in Section 3.1,

$$f_s \sim Cr^{-\alpha},$$

where α denotes the power-law exponent. In Table 6.1, the exponent α corresponds to the parameter denoted k_r in the original reports. Fracture density is quantified by the parameter P_{30} , defined as the number of fractures per unit volume. Following the procedure described in Section 3.1, P_{30} is derived from the fracture surface density P_{32} and the fracture size distribution. To obtain the prescribed target value of P_{30} for the full fracture population, the minimum admissible fracture size r_0 is adjusted. Here, r_0 represents a lower cutoff on fracture size used during DFN generation to control the total fracture intensity while preserving the shape of the power-law distribution. For all datasets, the fracture aperture factor is set to $a = 1 \times 10^{-4}$, and the fracture size bounds are $\underline{r} = 5$ and $\bar{r} = 100$. Each realization is discretized with a maximum mesh step size of $h = 5$.

Table 6.1: Discrete fracture network parameters.

Fracture set name	Orientation (trend, plunge), concentration	Size model power-law α (k_r)	P_{32} (mean fracture surface area) ($r_0 = 1, 564$ m)	P_{30} (fracture intensity) ($r_0 = 1, 564$ m)
NS	(292, 1) 17.8	2.50	0.094	0.0196
NE	(326, 2) 14.3	2.70	0.163	0.0427
NW	(60, 6) 12.9	3.10	0.098	0.0348
EW	(15, 2) 14.0	3.10	0.039	0.0138
HZ	(5, 86) 15.2	2.38	0.141	0.0247

The problem is analyzed for various fracture-to-matrix hydraulic conductivity ratios K_f/K_m . In general, K_f is the average hydraulic conductivity assigned to fractures, while K_m denotes the average value of the K_{xx} component of the matrix hydraulic conductivity tensors. The ratio K_f/K_m thus provides an effective measure of the conductivity contrast between fractures and the surrounding matrix for a given sample. Three representative conductivity contrasts are selected for this work, namely $K_f/K_m \in \{1 \times 10^3, 1 \times 10^5, 1 \times 10^7\}$. Accordingly, three datasets characterized by different K_f/K_m ratios are constructed:

- **Dataset $\mathcal{A}$:** $K_f/K_m = 1 \times 10^3$,
- **Dataset $\mathcal{B}$:** $K_f/K_m = 1 \times 10^5$,
- **Dataset $\mathcal{C}$:** $K_f/K_m = 1 \times 10^7$.

The matrix SRF is generated using the covariance matrix: $\Sigma = \begin{bmatrix} 0.25 & 0.2 & 0.2 \\ 0.2 & 0.25 & 0.2 \\ 0.2 & 0.2 & 0.25 \end{bmatrix}$.

The log-scale mean vector differs depending on the dataset: $\mu = [-4.0, -3.8, -3.9]$ for Dataset $\mathcal{A}$, $\mu = [-6.0, -5.8, -5.9]$ for Dataset $\mathcal{B}$, $\mu = [-8.0, -7.8, -7.9]$ for Dataset $\mathcal{C}$. Because the fracture hydraulic conductivity values within Ω_0 occur on the order of magnitude 1×10^{-1} , the fracture conductivity is fixed at $K_f = 0.1$ in all datasets for simplicity.

Each dataset contains 75,000 samples, evenly distributed between $P_{30} = 0.001$ and $P_{30} = 0.0025$. To enable generalization across different matrix correlation lengths, each dataset includes equal representation of $\lambda \in \{0, 10, 25\}$. Samples are split into 80% training set $\mathcal{L}$ and 20% test set $\mathcal{T}$. The training set is further divided into 80% training and 20% validation subsets ($\mathcal{V}$).

6.2 Voxelization

In order to form these datasets of homogenization blocks, the original input hydraulic conductivities $\mathbf{K}_G$ prescribed on a regular grid and the discrete fractures $\mathcal{F}_{h,H}$ contained within each homogenization block are voxelized onto a regular grid with a prescribed number of voxels. The fracture network is projected onto this

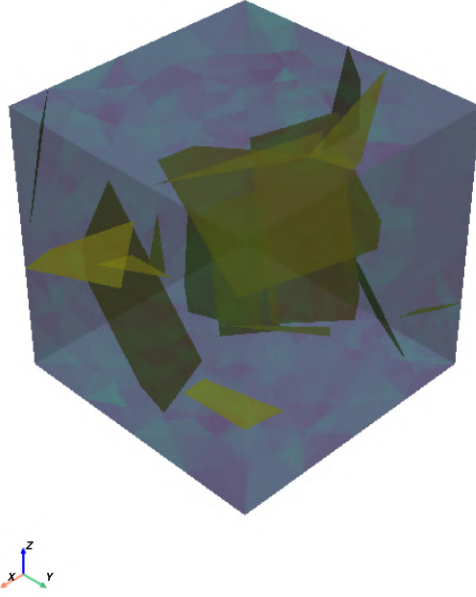
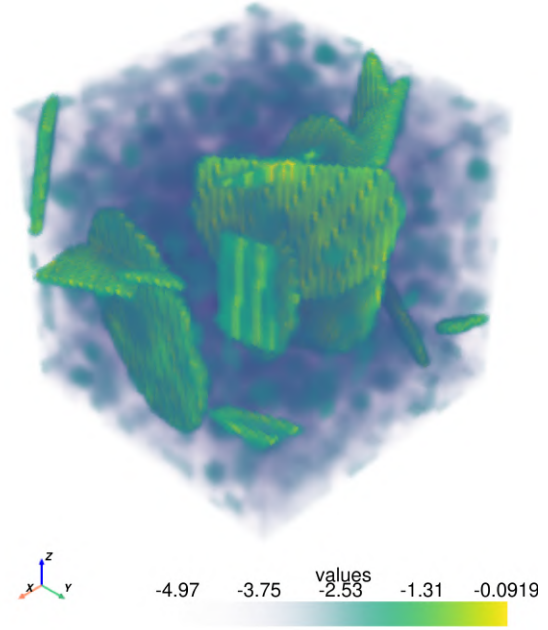
K_{xx} (bulk surface + fractures) K_{xx} voxelized

Figure 6.1: Illustration of the input random field of K_{xx} defined on an unstructured mesh and its voxelized representation.

grid by computing geometric intersections between fractures and voxels. For voxels intersected by fractures, the local conductivity tensors are computed as weighted combinations of the fracture and matrix ($\mathbf{K}_G$) conductivities, with weights proportional to the intersection volume fractions. To simplify and speed up the procedure, the voxelization algorithm considers disc-shaped fractures of equivalent area. The entire voxelization and mapping procedure is performed using the Bgem library [29]. Figure 6.1 shows an input mesh with fractures and its voxelized counterpart, illustrating the K_{xx} component of the hydraulic conductivity tensors. The plots are adjusted to enhance fracture visibility. As the voxelization quality required for accurate neural network predictions has not been established, no systematic assessment is performed. Based on experimental results, this approximation captures the key structural patterns and provides sufficient information for surrogate training.

6.3 Datasets Analysis

The distributions of components of equivalent hydraulic conductivity tensors for datasets $\mathcal{A}$, $\mathcal{B}$, and $\mathcal{C}$ are depicted in Figure 6.3. Focusing on the diagonal components k_{xx} , k_{yy} , k_{zz} , it is observed that, for Dataset $\mathcal{A}$, the equivalent tensor is primarily influenced by matrix elements characterized by $\boldsymbol{\mu} = [-4.0, -3.8, -3.9]$.

The analysis of Dataset $\mathcal{A}$ revealed that the peak observed in the distribution of the diagonal components cannot be attributed to the presence of fractures, but rather reflects the influence of varying correlation lengths within the dataset. In particular, the peak arises from samples with $\lambda = 0$ (see Figure 6.2). For Dataset $\mathcal{B}$, a stronger influence of fractures is observed. The distributions exhibit a heavier left tail, which corresponds to samples containing a small number of fractures that do not form connected pathways to increase overall hydraulic conductivity. As the ratio K_f/K_m increases, the left tail becomes heavier, since K_m is reduced in this setting. In Dataset $\mathcal{C}$, the effects described for Dataset $\mathcal{B}$ are further amplified. With regard to off-diagonal components k_{yz} , k_{xz} , and k_{xy} , the distributions of Datasets $\mathcal{B}$ and $\mathcal{C}$ display very similar shapes.

Overall, the equivalent tensor distributions exhibit substantial differences in both scale and shape across datasets. As a result, training a single surrogate on the combined dataset covering all K_f/K_m ratios has proven extremely challenging and has resulted in unsatisfactory accuracy thus far. To overcome this limitation, separate surrogates are trained: Surrogate A for Dataset $\mathcal{A}$, Surrogate B for Dataset $\mathcal{B}$, and Surrogate C for Dataset $\mathcal{C}$. For a given input $\mathbf{X}_j$, the appropriate surrogate is selected based on the corresponding K_f/K_m ratio.

6.4 Datasets Preprocessing

In order to facilitate surrogate training and improve generalization across different conductivity scales, the dataset inputs $\mathbf{X}_j$ and outputs $\mathbf{K}_j^{eq}$ are preprocessed in two distinct steps. The first step introduces a sample-dependent physical scaling, while the second step applies a fixed statistical normalization used during training.

6.4.1 Sample-wise Physical Scaling

For each sample j , the average matrix hydraulic conductivity $\overline{\mathbf{X}}_j^m$ is computed as a scalar by averaging the matrix conductivity tensor over all spatial locations and tensor components. This quantity represents a characteristic conductivity scale of the matrix for the given realization. Using this scale, the raw input tensors $\mathbf{X}_j$ and the corresponding homogenized output tensor $\mathbf{K}_j^{eq}$ are rescaled as

$$\hat{\mathbf{X}}_j = \frac{\mathbf{X}_j}{\overline{\mathbf{X}}_j^m}, \quad \hat{\mathbf{K}}_j = \frac{\mathbf{K}_j^{eq}}{\overline{\mathbf{X}}_j^m}.$$

This scaling is sample-dependent and constitutes an intrinsic part of the surrogate formulation. It removes the absolute conductivity magnitude while preserving the relative contrast between fracture and matrix conductivities, which is governed by the ratio K_f/K_m .

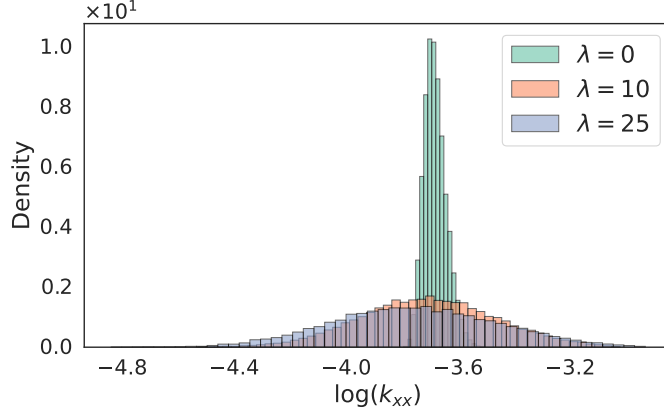


Figure 6.2: Distributions of $\mathbf{K}^{eq}$ component k_{xx} for Dataset $\mathcal{A}$, shown for different matrix correlation lengths $\lambda \in \{0, 10, 25\}$.

6.4.2 Statistical Normalization for Training

In a second step, the physically scaled tensors $\hat{\mathbf{X}}_j$ and $\hat{\mathbf{K}}_j$ are standardized using statistics estimated from the training dataset. This normalization is fixed after training and serves solely to improve numerical conditioning and convergence of the learning process; it can be interpreted as part of the surrogate parameters. The standardization is applied independently to each tensor component. Diagonal components of the hydraulic conductivity tensor are strictly positive and typically span several orders of magnitude. Applying a logarithmic transform, therefore, reduces their range and improves numerical stability before standardization. Off-diagonal components may take both positive and negative values and are standardized directly. Each component $\hat{k}_{\alpha\beta}^j$ of the physically scaled hydraulic conductivity tensor ($\alpha, \beta \in \{x, y, z\}$) is transformed as

$$\tilde{k}_{\alpha\beta}^j = \begin{cases} \frac{\log(\hat{k}_{\alpha\alpha}^j) - \hat{\mu}_{\mathcal{L}}(\log \hat{k}_{\alpha\alpha})}{\hat{\sigma}_{\mathcal{L}}(\log \hat{k}_{\alpha\alpha})}, & \alpha = \beta, \\ \frac{\hat{k}_{\alpha\beta}^j - \hat{\mu}_{\mathcal{L}}(\hat{k}_{\alpha\beta})}{\hat{\sigma}_{\mathcal{L}}(\hat{k}_{\alpha\beta})}, & \alpha \neq \beta, \end{cases}$$

where the empirical mean

$$\hat{\mu}_{\mathcal{L}}(k_{\alpha\beta}) = \frac{1}{|\mathcal{L}|} \sum_{j \in \mathcal{L}} \hat{k}_{\alpha\beta}^j$$

and empirical standard deviation

$$\hat{\sigma}_{\mathcal{L}}(k_{\alpha\beta}) = \sqrt{\frac{1}{|\mathcal{L}| - 1} \sum_{j \in \mathcal{L}} (\hat{k}_{\alpha\beta}^j - \hat{\mu}_{\mathcal{L}}(k_{\alpha\beta}))^2}$$

are computed from the physically scaled training set $\mathcal{L}$.

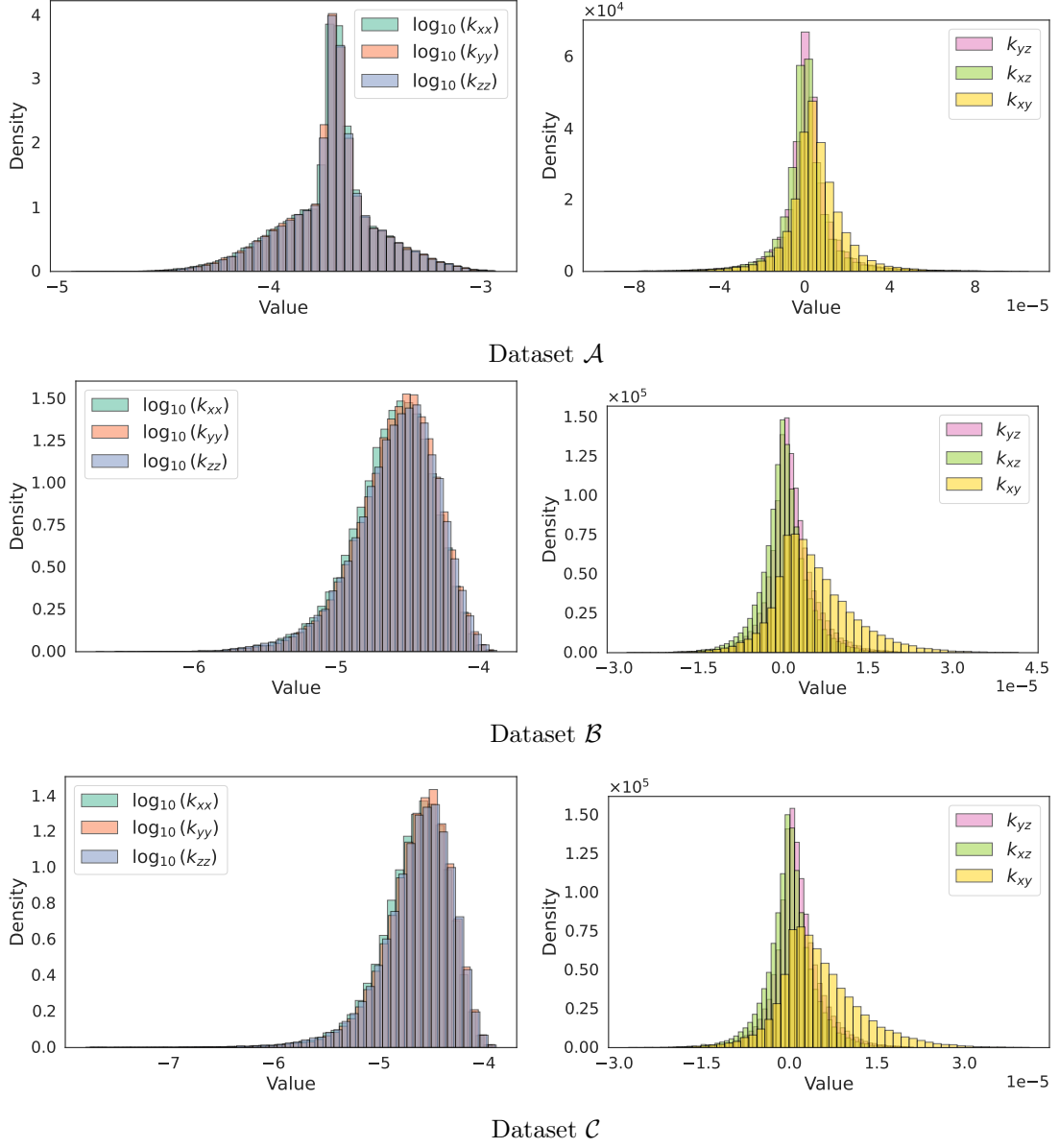


Figure 6.3: Distributions of $\mathbf{K}^{eq}$ components for datasets of different K_f/K_m , Dataset $\mathcal{A}$ for $K_f/K_m = 1 \times 10^3$, Dataset $\mathcal{B}$ for $K_f/K_m = 1 \times 10^5$, and Dataset $\mathcal{C}$ for $K_f/K_m = 1 \times 10^7$. Diagonal components are shown on a $\log_{10}$ scale.

7 Surrogate Prediction Accuracy Assessment

In this chapter, the prediction accuracy of trained surrogates is examined. First, the architecture of the designed surrogate is delineated. Afterwards, the surrogates are evaluated based on their performance on the test datasets $\mathcal{T}_A$, $\mathcal{T}_B$, and $\mathcal{T}_C$. Two loss functions, namely the mean squared error (MSE) and the Mahalanobis distance, are compared in Section 7.2. For surrogates trained with the selected loss, prediction capability on the test datasets is investigated in detail in Section 7.3. To further assess the generalizability of the trained surrogates, additional test datasets are constructed for each $K_f/K_m \in \{1 \times 10^3, 1 \times 10^5, 1 \times 10^7\}$. Using these datasets, the impact of the SRF correlation length (Section 7.4), different mean vectors and covariance matrices of SRFs (Section 7.5), the number of fractures (Section 7.6), varying DFN configurations (Section 7.7), and the homogenization block size (Section 7.8) is examined. Finally, in Section 7.10, the equivalent hydraulic conductivity tensors predicted by the surrogates are compared with those obtained by numerical homogenization for the macroscale problems described in Sections 3.3.1 and 3.3.2.

7.1 Surrogate Architecture

When developing the surrogate architecture, the structure of the dataset samples plays a crucial role. Given the nature of the input data, the surrogates are designed as a combination of a CNN (see Section 5.3), which serves as a feature extractor, and an MPL (see Section 5.2), which performs the regression task. To determine the final architecture, various combinations of convolutional layers, kernel sizes, and padding settings were explored, along with different MLP depths and widths, several activation functions, and both max-pooling and average-pooling strategies. While some of these alternatives produced predictions with a comparable accuracy, the configuration presented below consistently offered the best balance between performance and computational efficiency.

The CNN component of the surrogate consists of a Conv3D layer that integrates convolution and max pooling. A convolutional kernel of size $F_h = 3$, $F_w = 3$, $F_d = 3$ with stride $s = 1$ and padding $p = 1$ is employed. Each convolution is followed by max pooling with kernel sizes $Q_h = 2$, $Q_w = 2$, $Q_d = 2$, and stride $s_p = 2$. Batch normalization is applied to facilitate the training process. After four convolutional layers, the original input data of shape $64 \times 64 \times 64 \times 6$ is reduced to

Table 7.1: The architecture of the surrogate.

Layer	Type	Output size	Learnable parameters
	Input	$64 \times 64 \times 64 \times 6$	
1	Conv3D	$32 \times 32 \times 32 \times 48$	7,824
2	Conv3D	$16 \times 16 \times 16 \times 144$	186,768
3	Conv3D	$8 \times 8 \times 8 \times 432$	1,680,048
4	Conv3D	$4 \times 4 \times 4 \times 1,296$	15,117,840
5	AvgPool	$1 \times 1 \times 1 \times 1,296$	
6	Linear	2,048	2,656,256
7	Linear	2,048	4,196,352
8	Linear	1,024	2,098,176
9	LinearOutput	6	6,150

$4 \times 4 \times 4 \times 1,296$. This representation is subsequently flattened by a global average pooling layer before entering three fully connected linear layers of 2,048, 2,048, and 1,024 neurons, respectively. ReLU activation functions are applied in all convolutional and fully connected linear layers. The output layer consists of 6 neurons with an identity activation function. An overview of the surrogate architecture is provided in Table 7.1. Surrogate training was performed for 125 epochs using the Adam optimizer. The model corresponding to the lowest validation loss was saved for subsequent use. An initial learning rate of 0.0025 was employed and was adaptively reduced by 50% whenever the validation loss failed to decrease for 10 consecutive epochs. A training batch size of 64 was used.

7.2 Loss Comparison

When the MLE concept is used for surrogate training, the squared Mahalanobis distance (Mah. d.) is identified as an appropriate loss function (see Section 5.1.2), since it incorporates a covariance matrix estimated from the training data. If the correlations between the components of $\mathbf{K}^{eq}$ are not considered, the mean squared error (MSE) is regarded as a natural alternative. In this section, prediction accuracy on the test data is discussed across surrogates trained using the squared Mahalanobis distance and the mean squared error losses. The structure of the surrogate, as well as all hyperparameters, is kept unchanged (see Section 7.1 for surrogate architecture). Table 7.2 reports R^2 values between targets and corresponding predictions across components of $\mathbf{K}^{eq}$ for both MSE loss and Mahalanobis distance loss. It can be observed that both losses provided remarkably similar results.

Although the MSE loss did not incorporate prior knowledge of the covariance between data features, prediction accuracy on the test dataset is comparable to or, in most cases, slightly better than that achieved with the Mahalanobis distance loss. This observation suggests that the surrogates learned dependencies between features directly from the data without the requirement for an explicitly defined covariance matrix. Since the covariance used in the Mahalanobis distance is estimated from a

Table 7.2: Prediction accuracy (R^2) of surrogates trained with mean squared error (MSE) and squared Mahalanobis distance losses. Separate R^2 values are reported for each component of the equivalent hydraulic conductivity tensor $\mathbf{K}^{eq}$.

	Loss	R^2					
		k_{xx}	k_{yy}	k_{zz}	k_{yz}	k_{xz}	k_{xy}
Surrogate A	MSE	0.99851	0.99863	0.99866	0.98152	0.98203	0.98049
	Mah. d.	0.99855	0.99852	0.99863	0.98133	0.98211	0.97943
Surrogate B	MSE	0.98556	0.98683	0.98689	0.98443	0.98659	0.98629
	Mah. d.	0.98664	0.98600	0.98638	0.98721	0.98764	0.98643
Surrogate C	MSE	0.98394	0.98427	0.98326	0.98604	0.98749	0.98897
	Mah. d.	0.98332	0.98297	0.97946	0.98344	0.98510	0.97950

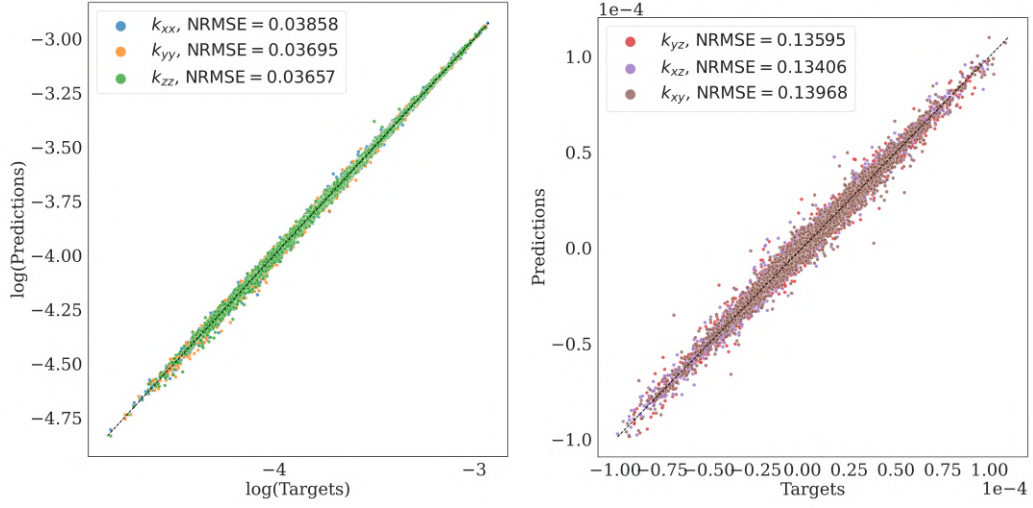
finite training set, estimation errors could be introduced that negatively influenced the training process and the generalization gap. For the remainder of this work, all results are presented using surrogates trained with MSE loss.

7.3 Prediction Accuracy on Test Datasets

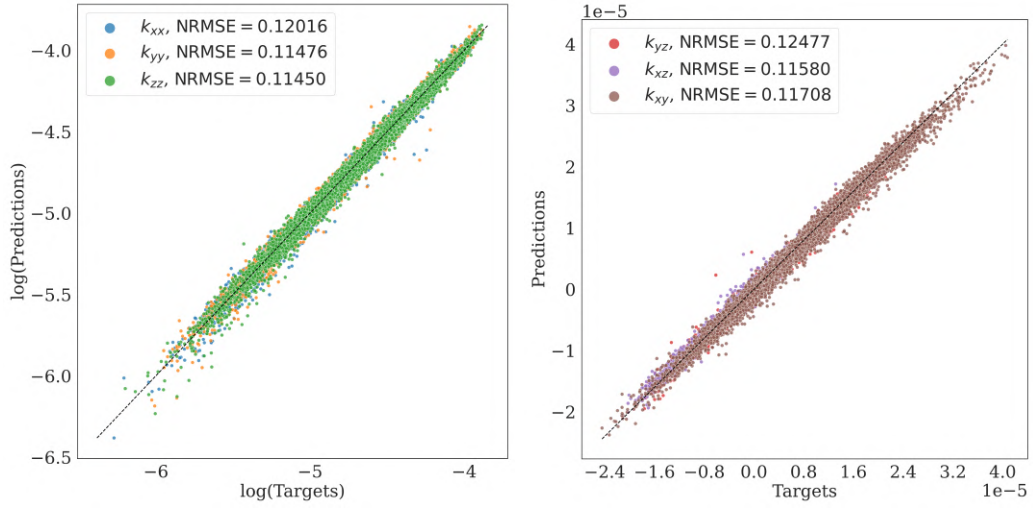
In this section, prediction accuracy of test datasets $\mathcal{T}_A$, $\mathcal{T}_B$, and $\mathcal{T}_C$ is discussed. These datasets originate from the same distributions as the corresponding training datasets and share identical SRF parameters and DFN configurations. Figure 7.1 depicts prediction accuracy expressed by a targets-predictions plot and NRMSE for all trained surrogates and each component of $\mathbf{K}^{eq}$.

For the diagonal components (left column), Surrogate A demonstrates the highest accuracy (NRMSE < 0.04), which can be attributed to the narrow distribution of these components. With training datasets of equal size, narrower distributions tend to be better represented, resulting in improved model performance. As the data distributions become broader and more complex (Figure 6.3), the predictive accuracy of the surrogates declines (Figure 7.1). This trend is especially pronounced for Surrogate C, where predictions for samples in the left tail of the distribution are considerably less accurate, likely due to their underrepresentation in the training set. This issue becomes more severe with increasing K_f/K_m . In contrast, for the off-diagonal components, Surrogate A performs the worst (NRMSE ≈ 0.14), reflecting the broader and more complex nature of these components' distributions. As a result, the overall prediction accuracy of Surrogate A is largely constrained by its ability to estimate off-diagonal components. Surrogate B achieves a more uniform accuracy across all components of $\mathbf{K}^{eq}$. For Surrogate C, the off-diagonal components are predicted with even slightly higher accuracy than the diagonal ones.

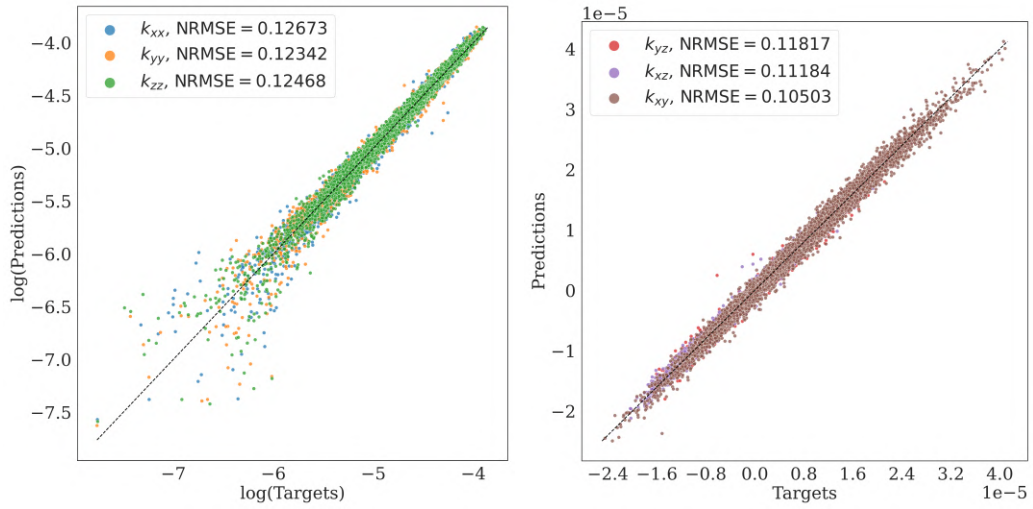
The trained surrogates show very good predictive performance on the test datasets. Although surrogate accuracy degrades in underrepresented regions of the training distribution, targeted dataset augmentation could mitigate this. However, there is no straightforward way to generate DFM homogenization blocks for a prescribed $\mathbf{K}^{eq}$.



Surrogate A



Surrogate B



Surrogate C

Figure 7.1: The prediction accuracy of the trained surrogates for the components of $\mathbf{K}^{eq}$ evaluated on test datasets.

7.4 Impact of SRF Correlation Length on Prediction Accuracy

This section investigates the impact of the matrix's SRF correlation length λ on prediction accuracy. Test datasets of 3,500 samples with a fixed maximum number of fractures $P_{30} = 0.0015$ are generated for $\lambda \in \{0, 5, 10, 15, 25, 50, 75, 100, 250, 500, 1,000\}$, while the surrogates were trained on data with $\lambda \in \{0, 10, 25\}$.

Figure 7.2 shows that Surrogate A exhibits increasing prediction accuracy with larger values of λ up to $\lambda = 75$. This trend is consistent with expectations, as the influence of fractures is limited in this case and longer correlation lengths lead to reduced variability in the SRF field, which makes the learning task easier for the neural network. The lack of training data for $\lambda > 75$ is reflected in a slightly reduced prediction performance. In detail, Figure 7.3 shows that the prediction accuracy of Surrogate A differs between the diagonal and off-diagonal components of $\mathbf{K}^{eq}$. For $\lambda = 0$, the maximum NRMSE reaches approximately 0.22 for the diagonal components of $\mathbf{K}^{eq}$ and 0.32 for the off-diagonal components. When $\lambda = 10$, the prediction of k_{xx} , k_{yy} and k_{zz} improves substantially, achieving near-perfect accuracy (NRMSE ≈ 0.03). The prediction accuracy of the off-diagonal components k_{yz} , k_{xz} and k_{xy} also increases (NRMSE ≈ 0.18). As λ increases, a consistent improvement in the prediction accuracy of the off-diagonal components is observed. Even in the best case ($\lambda = 75$), however, their accuracy (NRMSE ≈ 0.065) remains roughly twice lower than that of the diagonal components (NRMSE ≈ 0.03). The effect of increasing λ is first manifested in the improved prediction accuracy of the diagonal components.

It is important to note that slightly lower prediction accuracy is observed when the same architecture of Surrogate A is trained exclusively on data with $\lambda = 0$ (10,000 training samples), using identical hyperparameters. This indicates that the reduced prediction accuracy cannot be attributed to the presence of training samples with varying λ . It is likely that further improvements in NRMSE for the $\lambda = 0$ case could be achieved by increasing the amount of training data or by fine-tuning the hyperparameters. Interestingly, training on the entire Dataset $\mathcal{A}$ led to improved prediction accuracy even for samples with $\lambda = 0$, compared to training solely on $\lambda = 0$ data. Since a complete absence of spatial correlation is not realistic in natural settings, the obtained results are considered satisfactory.

The behavior described above is not observed for Surrogates B and C, for which the prediction accuracy is more uniformly distributed across the components of $\mathbf{K}^{eq}$. Figure 7.2 shows that with increasing λ , the prediction accuracy gets worse in a very similar way for both Surrogate B and Surrogate C. As mentioned above, Surrogate B and Surrogate C were trained on datasets with $K_f/K_m = 1 \times 10^5$ and $K_f/K_m = 1 \times 10^7$, where the influence of hydraulic conductivity of fractures on $\mathbf{K}^{eq}$ increases. Consequently, samples with very similar DFNs but dissimilar SRFs can result in similar $\mathbf{K}^{eq}$. As the correlation length increases, it becomes increasingly challenging for Surrogate B and Surrogate C to differentiate between these two cases, potentially leading to lower prediction accuracy. This is more pronounced for large

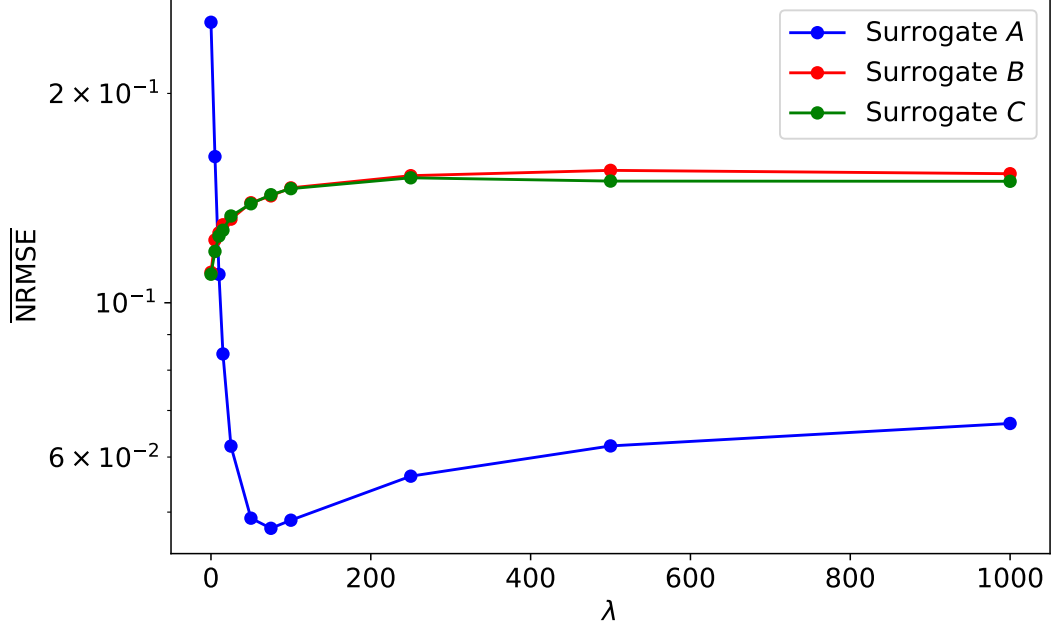


Figure 7.2: Effect of the SRF correlation length λ on the prediction accuracy of the surrogates.

correlation lengths that are underrepresented in the training dataset. Despite the equal representation of all three correlation lengths $\lambda = \{0, 10, 25\}$ in the training dataset, Surrogate B and Surrogate C exhibit the best predictive performance for uncorrelated SRFs ($\lambda = 0$). Generally, the NRMSE is for these surrogates higher than for Surrogate A, which is in accordance with what was observed on a test dataset $\mathcal{T}_A$, $\mathcal{T}_B$, $\mathcal{T}_C$ in the previous section.

In summary, the correlation length of the matrix SRF has a clear influence on surrogate prediction accuracy. Surrogate A benefits from increasing λ , with substantially improved accuracy for both diagonal and off-diagonal components, whereas Surrogates B and C experience a gradual decline in performance as higher correlation lengths reduce the distinguishability of SRF effects on $\mathbf{K}^{eq}$. For all three surrogates, prediction accuracy for samples with correlation lengths $\lambda = \{5, 15\}$ mirrors trends observed for the correlation lengths included in training datasets. This suggests that a wide range of correlation lengths on the homogenization block (of size $15 \times 15 \times 15$) can be effectively covered using datasets with $\lambda = \{0, 10, 25\}$, where the SRFs are generated following the procedure described in Section 3.2. The same trends were also observed for 2D cases [119]. For further analysis, test datasets with $\lambda = 10$ are considered. For this correlation length, the prediction accuracy is the most similar among all three surrogates.

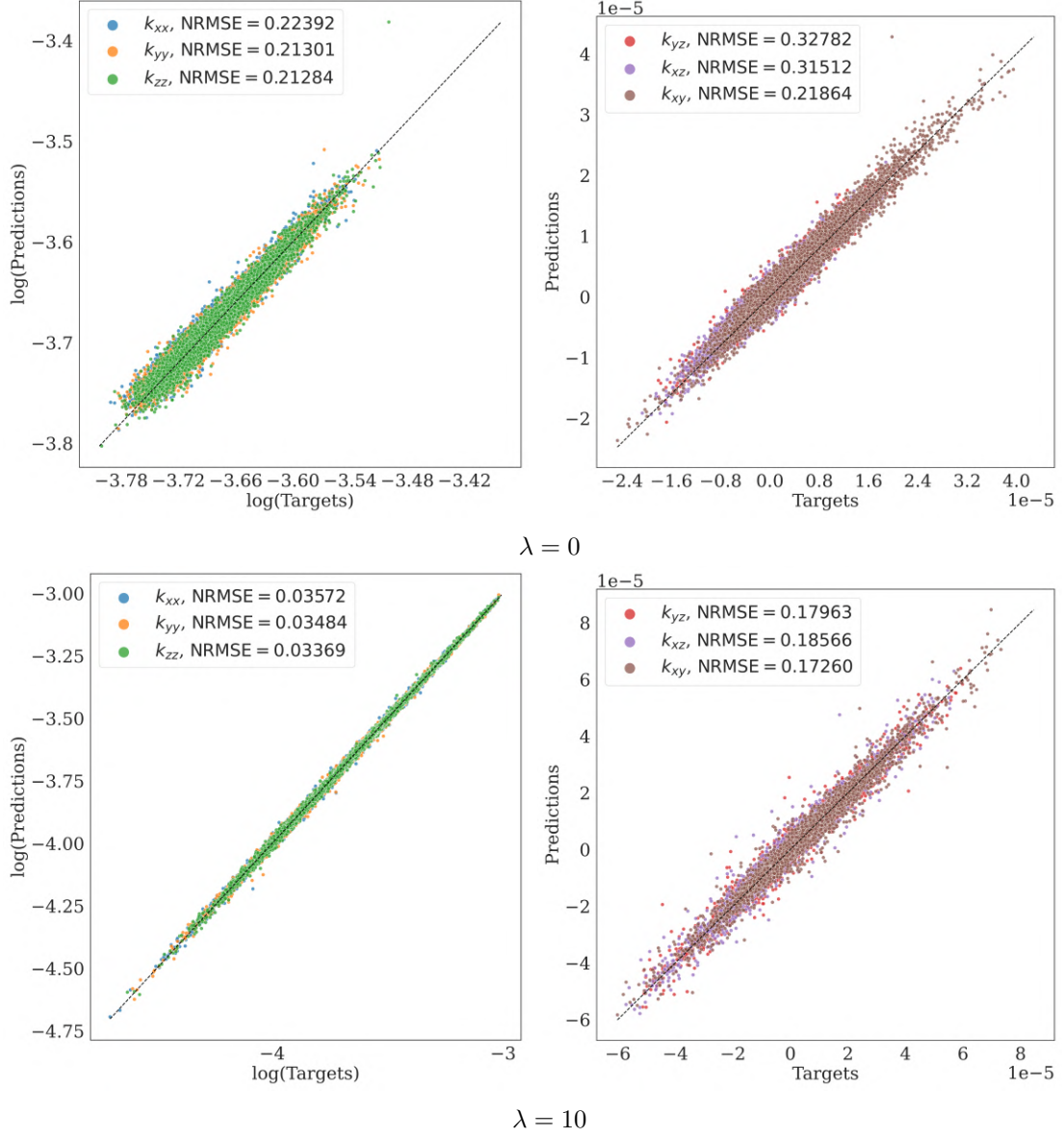


Figure 7.3: Prediction accuracy of Surrogate A for the diagonal and off-diagonal components of the equivalent hydraulic conductivity tensor $\mathbf{K}^{eq}$, evaluated on test datasets with $\lambda \in \{0, 10\}$.

7.5 Prediction Accuracy for Different SRF Means and Covariances

The prediction accuracy of the surrogates is further examined using test datasets generated with different covariance matrices Σ_i and mean vectors μ_i for the spatial random field (SRF). Each test dataset consists of 3,500 samples with parameters $\lambda = 10$ and $P_{30} = 0.0015$.

For reference, the hydraulic conductivity tensor in the bulk medium (see Section 3.2) is computed as

$$\mathbf{K} = \mathbf{Q}^T \mathbf{\Lambda} \mathbf{Q},$$

where $\mathbf{Q}$ is the rotation matrix and $\mathbf{\Lambda}$ is the diagonal matrix of principal hydraulic conductivities:

$$\mathbf{\Lambda} = \text{diag}(k_x, k_y, k_z),$$

with $(\log k_x, \log k_y, \log k_z) \sim \mathcal{N}(\mu, \Sigma)$. Here, μ denotes the mean vector and Σ the covariance matrix.

For training $\Sigma = \begin{bmatrix} 0.25 & 0.2 & 0.2 \\ 0.2 & 0.25 & 0.2 \\ 0.2 & 0.2 & 0.25 \end{bmatrix}$ was used. For testing, the following covariance matrices are employed:

$$\begin{aligned} \Sigma_1 &= \begin{bmatrix} 0.5 & 0.2 & 0.2 \\ 0.2 & 0.5 & 0.2 \\ 0.2 & 0.2 & 0.5 \end{bmatrix}, & \Sigma_2 &= \begin{bmatrix} 0.25 & 0.1 & 0.1 \\ 0.1 & 0.25 & 0.1 \\ 0.1 & 0.1 & 0.25 \end{bmatrix}, & \Sigma_3 &= \begin{bmatrix} 0.5 & 0.1 & 0.1 \\ 0.1 & 0.25 & 0.1 \\ 0.1 & 0.1 & 0.25 \end{bmatrix}, \\ \Sigma_4 &= \begin{bmatrix} 0.5 & 0.4 & 0.4 \\ 0.4 & 0.5 & 0.4 \\ 0.4 & 0.4 & 0.5 \end{bmatrix}, & \Sigma_5 &= \begin{bmatrix} 0.125 & 0.1 & 0.1 \\ 0.1 & 0.125 & 0.1 \\ 0.1 & 0.1 & 0.125 \end{bmatrix}, & \Sigma_6 &= \begin{bmatrix} 0.05 & 0.03 & 0.04 \\ 0.03 & 0.06 & 0.02 \\ 0.04 & 0.02 & 0.09 \end{bmatrix}, \\ \Sigma_7 &= \begin{bmatrix} 0.8 & 0.4 & 0.7 \\ 0.4 & 0.3 & 0.5 \\ 0.7 & 0.5 & 1 \end{bmatrix}. \end{aligned}$$

Table 7.3 presents the prediction accuracy of the individual components of $\mathbf{K}^{eq}$ for all three investigated surrogates when test datasets are generated using different covariance matrices Σ_i .

Matrix Σ_1 represents a case with higher variance than the reference matrix Σ . Consequently, the surrogates are required to predict data that lie outside their training distribution. This leads to reduced prediction accuracy for both the diagonal and off-diagonal components of $\mathbf{K}^{eq}$. A further increase in variance would likely result in an even greater deterioration of NRMSE.

Matrix Σ_2 corresponds to a case with lower covariance than Σ . The prediction accuracy in this case remains comparable to that of the reference scenario, except for the diagonal components predicted by Surrogate A, which are slightly less accurate. Unlike Surrogates B and C, Surrogate A exhibits diagonal values that are more

narrowly distributed. Combined with its already low NRMSE ≈ 0.035 , even small prediction errors can cause a noticeable increase to NRMSE ≈ 0.055 .

When the variance of a single component is increased, as in matrix Σ_3 , a rise in NRMSE is observed across all components of K^{eq} . As expected, this increase in NRMSE is the smallest for Surrogate C, where the bulk has the weakest influence on K^{eq} , and the largest for Surrogate A, where the matrix contribution dominates.

Doubling both the variance and the covariance, as in matrix Σ_4 , leads to worse prediction accuracy than in the Σ_1 case. The NRMSE approximately doubles for the diagonal components of Surrogate A but remains lower than in the Σ_3 case. This suggests that maintaining the overall pattern of the covariance matrix, even with higher magnitudes, helps preserve prediction accuracy.

Matrix Σ_5 represents a case with both variance and covariance reduced by half relative to Σ . Similar to Σ_2 , comparable or slightly improved prediction accuracy is achieved, although slightly worse results are again obtained for the diagonal components predicted by Surrogate A.

The covariance matrix Σ_6 represents a pseudorandom covariance structure with both lower covariances and smaller variances compared to Σ . Consequently, the surrogates predict within a narrower range of the data-generating distribution, which results in very high prediction accuracy. Conversely, matrix Σ_7 corresponds to a pseudorandom case with higher variances and covariances than Σ , thereby introducing stronger correlations and larger variability in the input data. As a result, the surrogates again make predictions on data outside their training distribution, and the prediction accuracy deteriorates significantly.

Overall, when all entries of Σ_i are less than or equal to those in Σ - that is, for Σ_2 , Σ_5 , and Σ_6 - the NRMSE values remain very similar to the reference case, regardless of the specific structure of Σ_i . In contrast, for cases where Σ_i exhibits larger values - namely Σ_1 , Σ_3 , Σ_4 , and Σ_7 - prediction accuracy deteriorates. In these scenarios, the surrogates show limited generalization capability beyond the range of their training data.

Different mean vectors μ_i are further analyzed:

$$\begin{aligned}
\mu &= [\mu_0, \mu_1, \mu_2] \\
\mu_1 &= [\mu_2, \mu_1, \mu_0] \\
\mu_2 &= [\mu_1, \mu_2, \mu_0] \\
\mu_3 &= [\mu_2, \mu_0, \mu_1] \\
\mu_4 &= \mu - 0.5 \\
\mu_5 &= \mu + 0.5 \\
\mu_6 &= [\mu_0 - 0.28, \mu_1 - 0.68, \mu_2 + 0.15] \\
\mu_7 &= [\mu_0 - 0.41, \mu_1 + 0.23, \mu_2 - 0.31] \\
\mu_8 &= [\mu_0 + 0.40, \mu_1 - 0.27, \mu_2 - 0.55] \\
\mu_9 &= \mu - 1 \\
\mu_{10} &= \mu + 1
\end{aligned}$$

For reference, the following mean vectors were used during training:

Table 7.3: Prediction accuracy (NRMSE) of surrogates evaluated on test datasets generated with different covariance matrices Σ_i of the spatial random field (SRF). Parameters: $\lambda = 10$ and $P_{30} = 0.0015$.

Surrogate		NRMSE					
		$\mathbf{k}_{xx}$	$\mathbf{k}_{yy}$	$\mathbf{k}_{zz}$	$\mathbf{k}_{yz}$	$\mathbf{k}_{xz}$	$\mathbf{k}_{xy}$
Σ	A	0.03626	0.03737	0.03377	0.17416	0.18252	0.19522
	B	0.12963	0.12441	0.11875	0.13409	0.12156	0.12780
	C	0.13626	0.13041	0.12960	0.12019	0.11432	0.11775
Σ_1	A	0.16481	0.16759	0.15561	0.30733	0.30402	0.35183
	B	0.29317	0.28748	0.24442	0.22987	0.27792	0.21342
	C	0.23484	0.26295	0.23037	0.21725	0.21770	0.21766
Σ_2	A	0.05223	0.05680	0.05335	0.16997	0.17360	0.18495
	B	0.12977	0.12344	0.11814	0.13233	0.11943	0.12768
	C	0.13095	0.12770	0.12560	0.11670	0.11186	0.11488
Σ_3	A	0.12162	0.11415	0.10882	0.25138	0.22462	0.25320
	B	0.18599	0.17223	0.16858	0.15036	0.17126	0.15813
	C	0.16368	0.17906	0.15291	0.14349	0.13370	0.12979
Σ_4	A	0.07070	0.07044	0.06684	0.28526	0.29872	0.31628
	B	0.18785	0.19655	0.18003	0.21428	0.27068	0.19896
	C	0.15862	0.15756	0.15676	0.20974	0.22204	0.20786
Σ_5	A	0.04143	0.04047	0.03761	0.14252	0.14705	0.15358
	B	0.12861	0.11975	0.11538	0.11803	0.10983	0.11327
	C	0.13514	0.12998	0.12843	0.10924	0.10821	0.10754
Σ_6	A	0.05860	0.05866	0.05983	0.11763	0.12271	0.13188
	B	0.12684	0.11945	0.11828	0.10991	0.10586	0.10883
	C	0.13282	0.12962	0.12835	0.10651	0.10656	0.10545
Σ_7	A	0.32044	0.31033	0.28715	0.54963	0.61910	0.58357
	B	3.3384	3.3095	2.8332	2.3958	3.7037	1.4492
	C	8.7913	10.881	8.1036	1.5946	4.0276	2.1271

- $\boldsymbol{\mu} = [\mu_0 = -4.0, \mu_1 = -3.8, \mu_2 = -3.9]$ for Dataset $\mathcal{A}$ and Surrogate A,
- $\boldsymbol{\mu} = [\mu_0 = -6.0, \mu_1 = -5.8, \mu_2 = -5.9]$ for Dataset $\mathcal{B}$ and Surrogate B,
- $\boldsymbol{\mu} = [\mu_0 = -8.0, \mu_1 = -7.8, \mu_2 = -7.9]$ for Dataset $\mathcal{C}$ and Surrogate C.

The corresponding surrogates are used for prediction, regardless of how modifications in $\boldsymbol{\mu}$ affect the K_f/K_m ratio.

Table 7.4 summarizes the prediction accuracy for test datasets generated using specific $\boldsymbol{\mu}_i$ values when constructing spatial random fields. When permutations of the original, yet similar, $\boldsymbol{\mu}$ values are used (i.e., $\boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \boldsymbol{\mu}_3$), prediction accuracy remains nearly unchanged across all surrogates.

In contrast, for $\boldsymbol{\mu}_4 = \boldsymbol{\mu} - 0.5$, which corresponds to an increased K_f/K_m ratio, prediction accuracy deteriorates for all surrogates. As shown in Figure 7.4, Surrogate A tends to predict samples lying in the left tail of the distribution of the diagonal components of $\mathbf{K}^{eq}$ for $\boldsymbol{\mu}$. The increased prediction error likely results from insufficient training data in that region of the distribution. For Surrogates B and C, the predicted distributions are not significantly shifted. In the case of Surrogate B, the higher K_f/K_m ratio leads to an overestimation of the diagonal components of $\mathbf{K}^{eq}$, especially in the right tail. A similar trend is observed for Surrogate C, though the influence of the bulk is weaker, resulting in a smaller increase in NRMSE compared to Surrogate B.

For $\boldsymbol{\mu}_5 = \boldsymbol{\mu} + 0.5$, the prediction performance is similar to the case with $\boldsymbol{\mu}$. To interpret this behavior, the dataset preprocessing procedure (see Section 6.4) is recalled. For each sample j , both inputs and outputs are scaled by the average matrix hydraulic conductivity $\overline{\mathbf{X}}_j^m$, computed over the entire tensor. As a result, bulk values remain identical for $\boldsymbol{\mu}, \boldsymbol{\mu}_4$, and $\boldsymbol{\mu}_5$, whereas the fracture conductivity - and consequently the K_f/K_m ratio - varies. Figure 7.5 illustrates the resulting distributions of $\mathbf{K}^{eq}/\overline{\mathbf{X}}_j^m$ for $\boldsymbol{\mu}, \boldsymbol{\mu}_4$, and $\boldsymbol{\mu}_5$. The distributions for $\boldsymbol{\mu}_5$ largely overlap with those for $\boldsymbol{\mu}$, while the distributions for $\boldsymbol{\mu}_4$ contain out-of-distribution samples, which are more challenging for the surrogates to predict accurately.

For $\boldsymbol{\mu}_6, \boldsymbol{\mu}_7$, and $\boldsymbol{\mu}_8$, pseudorandom perturbations are introduced into the original $\boldsymbol{\mu}$. In all three cases, prediction accuracy is affected but not severely degraded, and the results remain acceptable for practical applications.

For $\boldsymbol{\mu}_9 = \boldsymbol{\mu} - 1$, prediction errors become more pronounced than those observed for $\boldsymbol{\mu}_4$. In particular, Surrogates B and C exhibit insufficient accuracy in this case. Similarly, for $\boldsymbol{\mu}_{10} = \boldsymbol{\mu} + 1$, the trends observed for $\boldsymbol{\mu}_5$ are amplified. The off-diagonal components of $\mathbf{K}^{eq}$ are predicted less accurately by Surrogates B and C. Samples generated with $\boldsymbol{\mu}_9$ and $\boldsymbol{\mu}_{10}$ would likely be more accurately predicted using surrogates trained on datasets corresponding to K_f/K_m ratios approximately ten times lower and ten times higher, respectively.

Overall, the surrogates generalize well for samples with only minor deviations in $\boldsymbol{\mu}$. Furthermore, samples with slightly higher $\boldsymbol{\mu}$ values, such as $\boldsymbol{\mu}_5$ (corresponding to a lower K_f/K_m ratio), are predicted more accurately than those with slightly lower values, such as $\boldsymbol{\mu}_4$ (corresponding to a higher K_f/K_m ratio). This can be attributed to greater overlap between the data distributions of $\boldsymbol{\mu}$ and $\boldsymbol{\mu}_5$ compared to

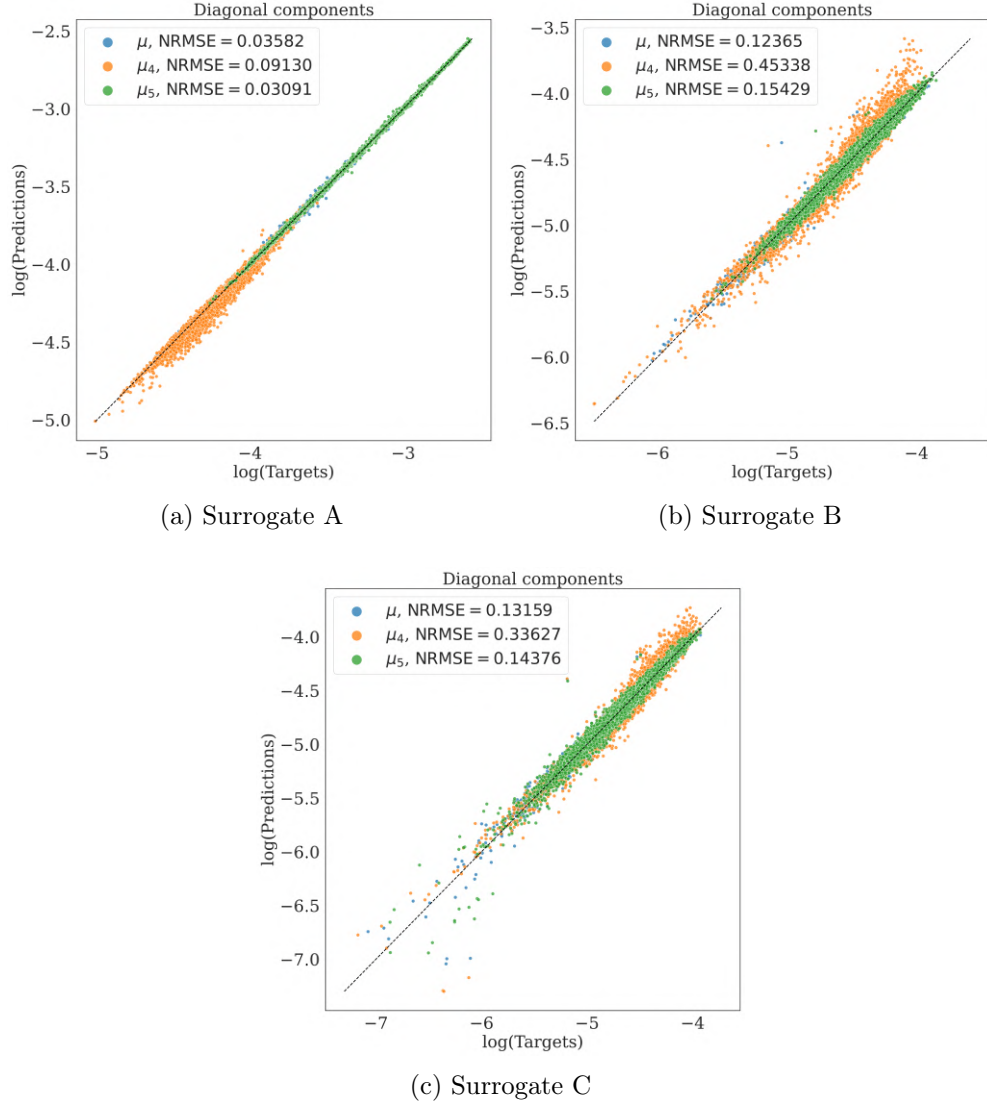


Figure 7.4: Prediction accuracy of K^{eq} diagonal components when using Surrogate A, Surrogate B, and Surrogate C to predict on data with μ , μ_4 , and μ_5 .

that between μ and μ_4 , given the way the datasets are constructed. For larger deviations, such as those represented by μ_9 and μ_{10} , surrogate models trained specifically on data with the corresponding K_f/K_m ratios would be more appropriate.

Table 7.4: Prediction accuracy (NRMSE) of surrogates evaluated on test datasets generated with different mean vectors μ_i of the spatial random field (SRF). Parameters: $\lambda = 10$ and $P_{30} = 0.0015$.

Surrogate		NRMSE					
		k_{xx}	k_{yy}	k_{zz}	k_{yz}	k_{xz}	k_{xy}
μ	A	0.03626	0.03737	0.03377	0.17416	0.18252	0.19522
	B	0.12963	0.12441	0.11875	0.13409	0.12156	0.12780
	C	0.13626	0.13041	0.12960	0.12019	0.11432	0.11775
μ_1	A	0.03662	0.03682	0.03326	0.16293	0.17481	0.18053
	B	0.12873	0.12508	0.11938	0.13540	0.12107	0.12771
	C	0.13682	0.13062	0.12924	0.11967	0.11316	0.11753
μ_2	A	0.03782	0.03732	0.03439	0.15982	0.16476	0.17083
	B	0.12862	0.12519	0.11961	0.13707	0.12168	0.12886
	C	0.13617	0.13091	0.13015	0.12011	0.11355	0.11699
μ_3	A	0.03808	0.03830	0.03630	0.15944	0.17530	0.18363
	B	0.13078	0.12476	0.11984	0.13615	0.12407	0.12966
	C	0.13730	0.13136	0.13059	0.12057	0.11410	0.11793
μ_4	A	0.08599	0.09518	0.09254	0.19192	0.19611	0.21854
	B	0.41593	0.56431	0.36538	0.22618	0.20616	0.21115
	C	0.34101	0.35409	0.31774	0.21134	0.19842	0.20094
μ_5	A	0.03197	0.03050	0.03023	0.16647	0.17852	0.18881
	B	0.15243	0.15621	0.15486	0.25893	0.29188	0.29582
	C	0.14106	0.14425	0.14629	0.19998	0.18249	0.19547
μ_6	A	0.10056	0.10456	0.09716	0.15406	0.16652	0.17371
	B	0.15311	0.15107	0.13510	0.13849	0.12988	0.13372
	C	0.15815	0.15215	0.15629	0.12797	0.12058	0.12125
μ_7	A	0.05305	0.05447	0.05334	0.19772	0.21010	0.19802
	B	0.13885	0.13744	0.12546	0.13228	0.12575	0.13022
	C	0.14874	0.13906	0.13886	0.12031	0.11525	0.11598
μ_8	A	0.08853	0.09962	0.08989	0.15783	0.17654	0.16317
	B	0.13958	0.14163	0.13012	0.14949	0.14722	0.14229
	C	0.14034	0.13541	0.13827	0.13071	0.11900	0.12316
μ_9	A	0.31958	0.37756	0.37193	0.39823	0.39130	0.47453
	B	4.9905	7.9230	3.3156	0.53280	0.49370	0.54898
	C	2.3237	4.0238	2.3949	0.43063	0.39786	0.42945
μ_{10}	A	0.03608	0.03148	0.03552	0.17113	0.18024	0.19636
	B	0.21143	0.29369	0.23087	0.53154	0.91124	0.87068
	C	0.20973	0.21186	0.22409	0.42084	0.65398	0.45627

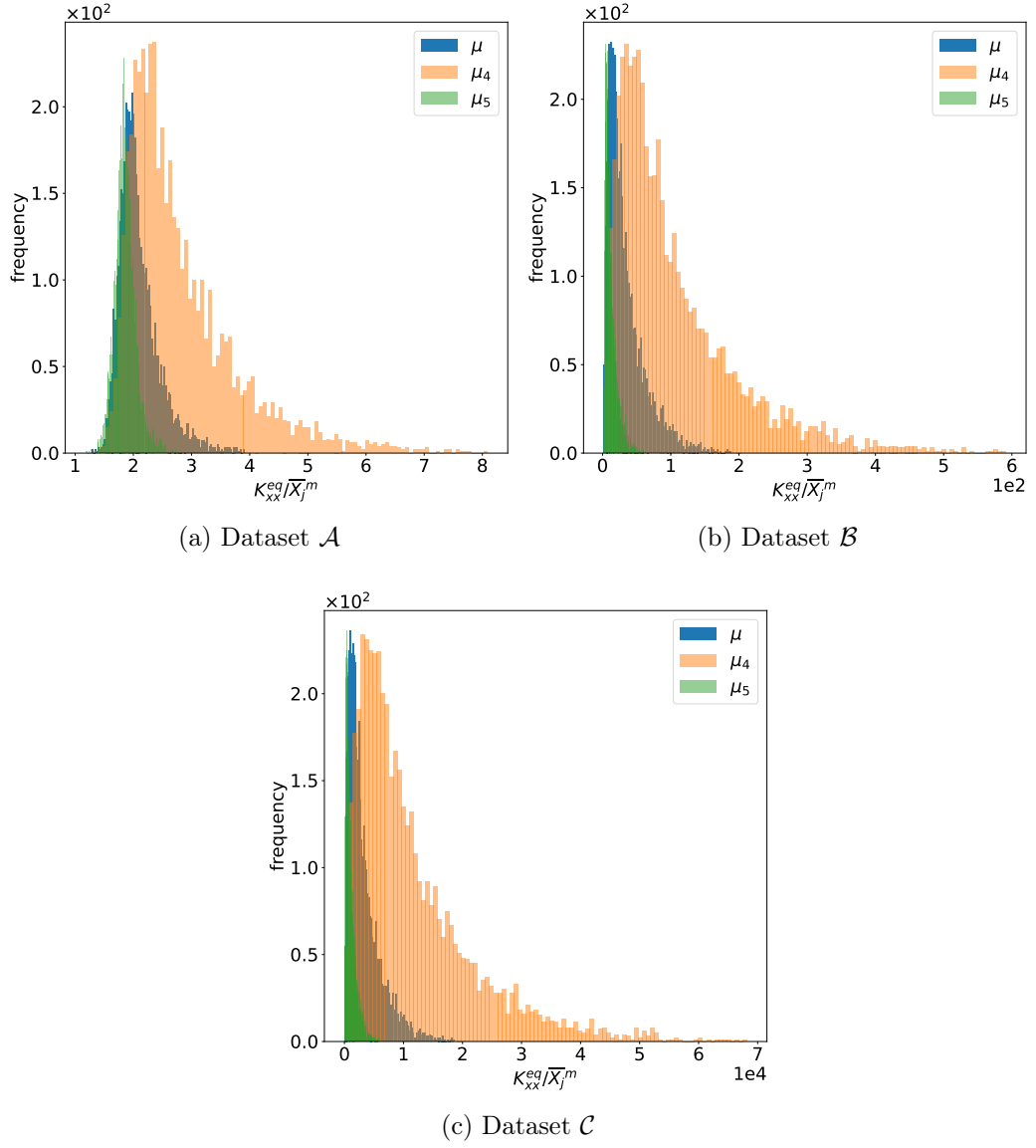


Figure 7.5: Distributions of K_{xx}^{eq} divided by the average matrix hydraulic conductivity $\overline{X}_j^m$. Datasets with μ , μ_4 , and μ_5 are compared.

7.6 Impact of Number of Fractures on Prediction Accuracy

The prediction accuracy of the surrogates is further analyzed for test datasets with varying numbers of fractures. The surrogates were originally trained on datasets containing samples with equally represented fracture densities of $P_{30} = 0.0010$ and $P_{30} = 0.0025$. For evaluation, test datasets consisting of 3,500 samples are generated with $\lambda = 10$ and $P_{30} \in \{0.0004, 0.0006, 0.0008, 0.0012, 0.0015, 0.0020, 0.0025, 0.0030, 0.0050, 0.0075, 0.0100, 0.0150, 0.0200\}$.

Figure 7.6 shows the variation of $\overline{\text{NRMSE}}$ with respect to P_{30} . Across all surrogates, the best prediction accuracy is achieved for $P_{30} = 0.0008$. For Surrogate A, $\overline{\text{NRMSE}}$ remains nearly constant for $P_{30} \in \{0.0004, 0.0006, 0.0008, 0.0012\}$, followed by a steep increase, except for $P_{30} = 0.0025$, which is explicitly included in the training data. Even at the upper extreme of $P_{30} = 0.0200$, the achieved $\overline{\text{NRMSE}} = 0.2151$ remains acceptable. For Surrogates B and C, a decrease in $\overline{\text{NRMSE}}$ is observed from $P_{30} = 0.0004$ to $P_{30} = 0.0008$, followed by an increase up to $P_{30} = 0.0020$. As with Surrogate A, $\overline{\text{NRMSE}}$ subsequently increases for larger P_{30} . Surrogate A is the most sensitive to P_{30} values outside the training range, whereas Surrogate B shows lower sensitivity to such variations. Overall, the obtained prediction accuracies, represented by $\overline{\text{NRMSE}}$, can be considered satisfactory for practical applications.

To gain further insight into these trends, Figure 7.7 presents the prediction accuracy (NRMSE) for diagonal and off-diagonal components of $\mathbf{K}^{eq}$ separately. Consistent with the findings in Section 7.3, the performance of Surrogate A is mainly governed by its ability to predict the tensor's off-diagonal components. For this surrogate, both diagonal and off-diagonal NRMSE increase similarly with increasing P_{30} . Surrogates B and C exhibit comparable NRMSE values for the diagonal components up to $P_{30} = 0.0012$. Beyond this value, the accuracy deteriorates more rapidly for Surrogate C, reflecting its greater sensitivity to fracture effects. For the off-diagonal components, both Surrogate B and C follow similar trends, with Surrogate C providing slightly better accuracy. The more rapid increase in $\overline{\text{NRMSE}}$ for $P_{30} > 0.0020$ in Surrogate C is primarily driven by the degradation in the prediction of the diagonal components.

In summary, the prediction accuracy of all surrogates varies systematically with the fracture density P_{30} , with the best performance occurring near $P_{30} = 0.0008$ and a noticeable degradation occurring outside the range represented in the training data. Surrogate A is the most sensitive to deviations in P_{30} , primarily due to difficulties in predicting the off-diagonal components of $\mathbf{K}^{eq}$. Surrogates B and C exhibit slightly more stable performance, although a reduction in accuracy is still observed at higher fracture densities. Overall, the NRMSE remains below 0.22 across all tested values of P_{30} . If higher accuracy is required, the training dataset can be extended to include a wider range of fracture densities.

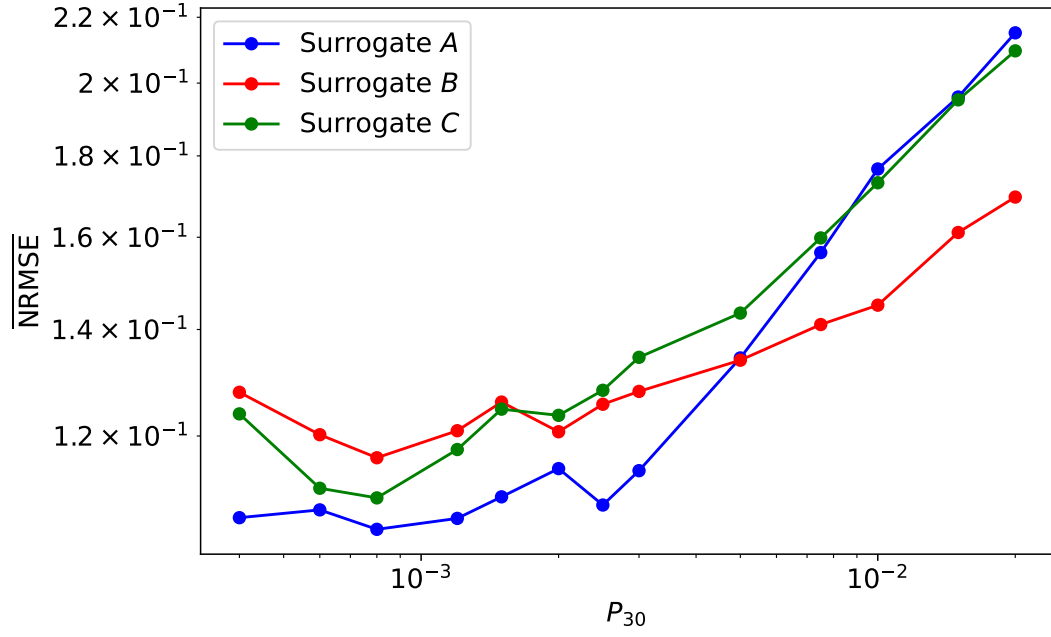
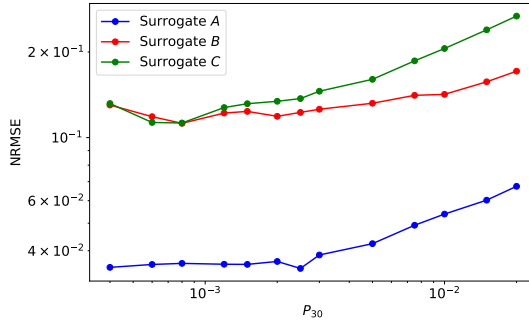
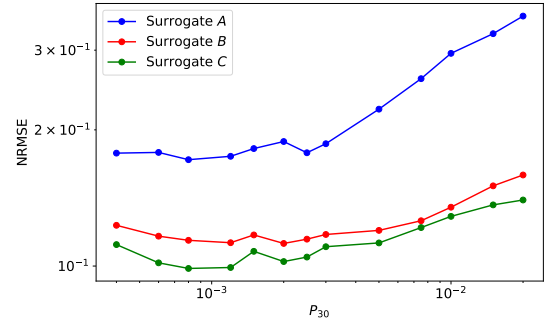


Figure 7.6: Impact of the number of fractures (P_{30}) on the surrogate's prediction accuracy.



(a) Diagonal components of $\mathbf{K}^{eq}$



(b) Off-diagonal components of $\mathbf{K}^{eq}$

Figure 7.7: Impact of the number of fractures (P_{30}) on the surrogate's prediction accuracy for the diagonal and off-diagonal components of $\mathbf{K}^{eq}$.

7.7 Prediction Accuracy Across Datasets with Varying DFN Configurations

To evaluate the prediction accuracy of the surrogates under different DFN parameter settings, four additional test datasets are generated. Each dataset comprised 3,500 samples with $\lambda = 10$ and $P_{30} = 0.0015$. Table 7.5 summarizes the adopted DFN configurations. DFN 1 and DFN 2 follow the hydrological DFN model of [59][Table 2–3], developed for groundwater flow simulations in granite bedrock at the Forsmark nuclear power plant site in Sweden. DFN 1 corresponds to depths > -200 m, whereas DFN 2 applies to depths between -200 m and -400 m. Configurations DFN 3 and DFN 4 are adopted from [93] for site-descriptive modeling of the Laxemar area in Sweden, which also features granite bedrock. Specifically, DFN 3 [93, Table 10–31] was used for upscaling studies, while DFN 4 [93, Table 10–16] was employed in models estimating open fracture intensity.

Table 7.5: Test DFN configurations.

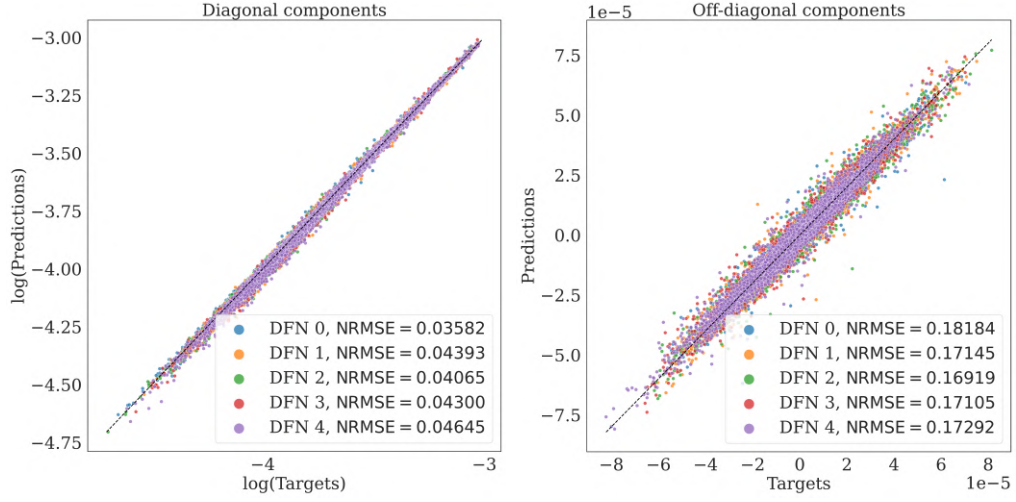
DFN	Fracture set name	Orientation (trend, plunge), concentration	Size model power-law α (k_r)	P_{32} (mean fracture surface area) ($r_0 = 1, 564$ m)	P_{30} (fracture intensity) ($r_0 = 1, 564$ m)
DFN 1	NS	(292, 1) 17.8	2.50	0.073	0.0152
	NE	(326, 2) 14.3	2.70	0.319	0.0837
	NW	(60, 6) 12.9	3.10	0.107	0.0380
	EW	(15, 2) 14.0	3.10	0.088	0.0313
	HZ	(5, 86) 15.2	2.38	0.543	0.0953
DFN 2	NS	(292, 1) 17.8	2.50	0.142	0.0296
	NE	(326, 2) 14.3	2.70	0.345	0.0905
	NW	(60, 6) 12.9	3.10	0.133	0.0472
	EW	(15, 2) 14.0	3.10	0.081	0.0288
	HZ	(5, 86) 15.2	2.38	0.316	0.0554
DFN 3	NNE	(283, 6.5) 13	2.50	0.230	0.0480
	ENE	(332, 0.5) 21	2.45	0.180	0.0351
	WNW	(201, 5) 12	2.30	0.190	0.0291
	NNW	(246, 10) 12	2.30	0.050	0.0077
	Sub-H	(334, 87) 10	2.20	0.350	0.0443
DFN 4	ENE	(340.3, 1.2) 15	2.60	0.390	0.0921
	WNW	(208.9, 2.2) 10.9	2.30	0.550	0.0843
	NS	(272.8, 12) 11.5	2.70	0.320	0.0840
	SubH	(277.1, 84.3) 11.1	2.65	0.290	0.0723

Figure 7.8 presents a comparison of prediction accuracy, expressed in terms of NRMSE, for test datasets generated with the DFN configurations introduced above (DFN 1–4). For reference, the training configuration, denoted as DFN 0, is also

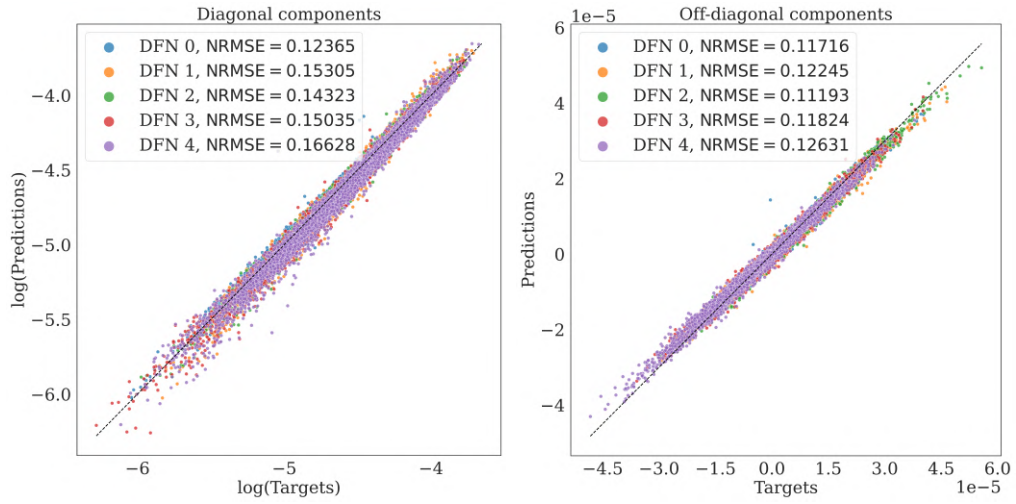
included. The figure shows prediction accuracies separately for the diagonal components (k_{xx} , k_{yy} , k_{zz} combined) and off-diagonal components (k_{yz} , k_{xz} , k_{xy} combined) of $\mathbf{K}^{eq}$, as predicted by Surrogates A, B, and C.

The prediction accuracies of the off-diagonal components remain consistent across DFN configurations for each surrogate. The minor variations observed are attributed to the limited dataset size and the potential influence of outliers. The prediction accuracy for the diagonal components varies slightly across configurations. For Surrogate A, no notable differences are observed among the DFN configurations. When the influence of fractures on $\mathbf{K}^{eq}$ increases, a small subset of samples exhibits a pronounced fracture effect, forming the tail of the diagonal component distributions. This effect, combined with the limited dataset size, results in a modest increase in NRMSE for DFN 1–4 in the case of Surrogates B and C. However, this increase is not considered significant with respect to the practical applicability of the surrogates.

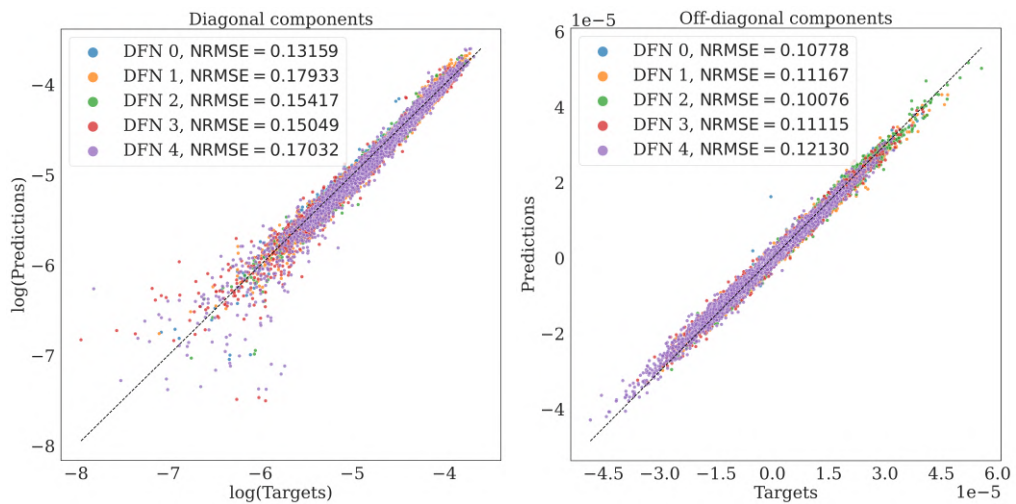
Although the examined DFN configurations differ, the predictive capabilities of the surrogates remain largely unaffected, with only minor variations in accuracy for the diagonal components of $\mathbf{K}^{eq}$. This suggests that the fractures within the homogenization block do not exhibit spatial patterns that would influence the generalization capability of the neural networks. These results indicate that the surrogates generalize well across different DFN parameterizations, suggesting that their predictive capability is largely independent of specific DFN configurations.



Surrogate A



Surrogate B



Surrogate C

Figure 7.8: The prediction accuracy of the trained surrogates for the diagonal and off-diagonal components of $\mathbf{K}^{eq}$ is evaluated on test datasets with different DFNs. Diagonal components are shown on a $\log_{10}$ scale.

7.8 Prediction Accuracy with Varying Homogenization Block Sizes

As described in Section 4.4, the homogenization block size is assumed as $b = 1.5H$, where H denotes the maximum element size of a coarse mesh. For each homogenization block, the intersection of the fracture network $\mathcal{F}_{h,H}$ with the block is identified, and a compatible fine mesh is generated with maximum element size h , corresponding to the resolution of a fine mesh. The original dataset $\mathcal{D}$ is generated using $b = 15$, $H = 10$, and $h = 5$. This section investigates how variations in the parameters b , H , and h influence the prediction accuracy of the trained surrogates. For clarity, all comparisons are conducted using Surrogate B with $\lambda = 10$ for the SRF and $P_{30} = 0.0015$.

Figure 7.9 presents the prediction accuracy for data generated with different block sizes $b \in \{10, 15, 20, 30\}$. The fine mesh step $h = 5$ and coarse mesh step $H = 10$ are kept constant in this case. An increase in the block size b leads to a narrower range of values in the components of $\mathbf{K}^{eq}$. Larger blocks produce a more homogeneous fracture influence within the domain. However, prediction accuracy decreases with increasing b , as the number of fractures and SRF bulk elements grows relative to the training data. Consequently, the voxelized inputs ($64 \times 64 \times 64$ voxels) exhibit greater internal variability than those used for training, in which larger contiguous regions with a single SRF value were present. A different trend is observed for $b = 10$. Although the total number of fractures is smaller than for $b = 15$, the orientations of fractures within these smaller blocks strongly affect prediction accuracy, particularly for the off-diagonal components of $\mathbf{K}^{eq}$. Because fracture sizes remain unchanged, smaller blocks can contain fractures spanning the entire block, leading to higher equivalent hydraulic conductivities. This effect likely causes the surrogate to underestimate conductivities, most notably in the diagonal components of $\mathbf{K}^{eq}$. These findings suggest that, without corresponding adjustments to the fine and coarse mesh resolutions, using homogenization block sizes that deviate significantly from the training configuration reduces prediction accuracy. Slightly larger blocks, up to approximately 33% larger than the training size, still yield acceptable predictions, whereas smaller blocks lead to notable degradation in surrogate performance.

Figure 7.10 shows results for the comparison of block sizes $b \in \{10, 15, 20, 30\}$, where the ratio between b , H , and h is kept constant across all cases. Under this configuration, each homogenization block contains a proportionally similar number of bulk elements of a fine mesh. The results show how the equivalent hydraulic conductivity increases with block size. Larger fractures are progressively included within the homogenization blocks. When combined with the unchanged bulk SRF, this leads to higher overall equivalent hydraulic conductivities. Since Surrogate B was trained for a fracture-to-matrix hydraulic conductivity ratio of $K_f/K_m = 1 \times 10^5$, its predictive performance is affected, particularly for the configuration $b = 30$, $H = 20$, and $h = 10$. While the NRMSE of the diagonal components deteriorates in a generally expected manner with increasing b , an exception is observed for the

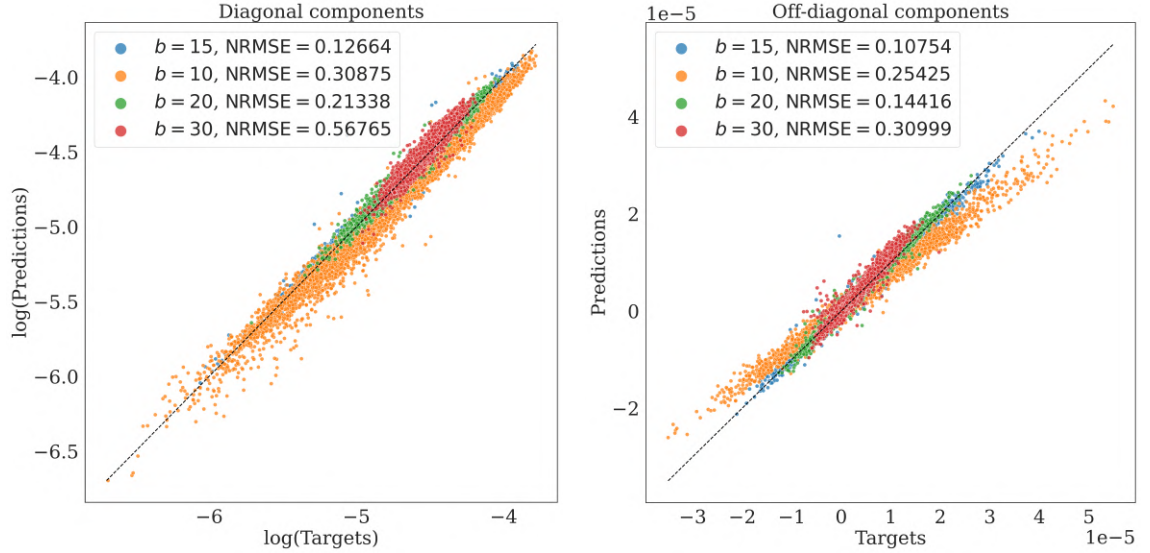


Figure 7.9: Prediction accuracy (NRMSE) on data with different block sizes $b \in \{10, 20, 30\}$, reference $b = 15$. Constant fine mesh step $h = 5$ and coarse mesh step $H = 10$. Results correspond to Surrogate B with $\lambda = 10$ and $P_{30} = 0.0015$.

off-diagonal components in the case $b = 10$, $H = 6.67$, and $h = 3.33$. The fracture length range is identical in all cases ($\underline{r} = 5$, $\bar{r} = 100$); however, the relative absence of small fractures in this configuration may result in a more heterogeneous influence on $\mathbf{K}^{eq}$, a behavior not fully captured by the training data.

In summary, the prediction accuracy of the surrogates is sensitive to changes in the homogenization block size when mesh resolutions are kept fixed, as deviations from the training configuration lead to distributional shifts in both fracture and SRF representations. When the ratios between b , H , and h are preserved, the surrogates maintain satisfactory predictive performance, with only minor accuracy degradation, except when the resulting data fall outside the training distribution. These results indicate that consistent geometric and discretization scaling relative to the training configuration is essential for reliable surrogate predictions.

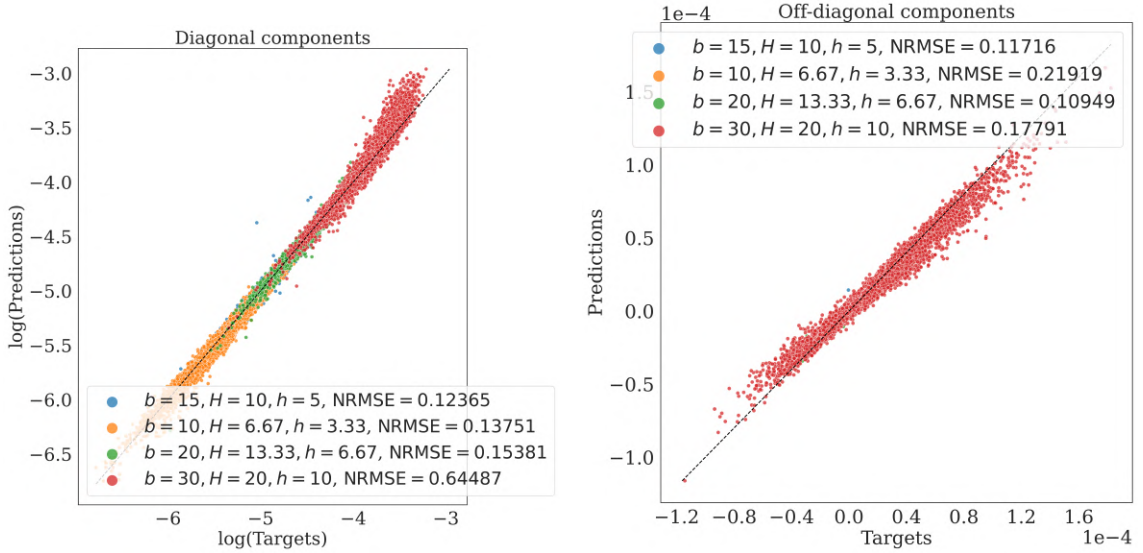


Figure 7.10: Prediction accuracy (NRMSE) on data with different block sizes $b \in \{10, 20, 30\}$, reference $b = 15$. Constant ratio between b , H , and h . Results correspond to Surrogate B with $\lambda = 10$ and $P_{30} = 0.0015$.

7.9 Computational Cost Reduction by Surrogates

The surrogates are primarily employed to accelerate numerical homogenization. To assess their efficiency, the computational cost of numerical homogenization, measured as CPU time C_H , is compared with the corresponding surrogate costs on the CPU and GPU, denoted by C_S and C_S^{GPU} , respectively. The surrogate training time is excluded from this comparison. Several scenarios with different numbers of homogenization blocks are evaluated. The domain size Ω is varied, while the placement of homogenization blocks follows the procedure described in Section 4.4.

Figure 7.11 reports the computational time required for numerical homogenization C_H and for surrogate inference on the CPU (C_S) and GPU (C_S^{GPU}). CPU-based experiments were performed on the Charon cluster of MetaCentrum using a single Intel Xeon Silver 4114 CPU (2.2 GHz) with 45 GB RAM. GPU-based experiments were conducted on the Adan cluster using an Intel Xeon Gold 5218 CPU (2.30 GHz) together with an NVIDIA Tesla T4 (16 GB) GPU. As expected, the computational cost increases with domain size and the corresponding number of homogenization blocks. The SRF and fracture network are generated prior to upscaling and are identical across all scenarios; these steps are therefore excluded from the comparison. Numerical homogenization is the most computationally expensive approach. The cost C_H includes mesh generation and SRF interpolation (approximately 60% of the total runtime) and the simulation time of the numerical solver (approximately 30%), which is executed separately for each homogenization block.

For CPU-based surrogate inference (C_S), the dominant contribution (approximately 85%) arises from the forward pass of the neural network used to predict the equivalent hydraulic conductivity tensor. Voxelization of the input SRF and fractures accounts for roughly 10% of the cost. The entire domain Ω is voxelized once

and then partitioned into homogenization blocks. This step requires substantial memory (45 GB RAM in the reported configuration). If memory becomes limiting, voxelization can be performed in smaller subdomains. The remaining cost corresponds to dataset handling and preprocessing. For GPU-based inference (C_S^{GPU}), voxelization constitutes the largest share of runtime (approximately 58%), while neural network inference is negligible. The remaining time is attributed to dataset manipulation.

The computational gains achieved by using surrogates instead of numerical homogenization are shown in Figure 7.12. The speedup increases with the number of homogenization blocks, reaching $C_H/C_S \approx 17$ and $C_H/C_S^{GPU} \approx 120$ for 1,331 blocks. For larger numbers of blocks, the growth of C_H is slower than that of C_S and C_S^{GPU} , leading to a slight reduction in speedup. Even in the case with 3,375 blocks, speedups of approximately $C_H/C_S \approx 12$ and $C_H/C_S^{GPU} \approx 102$ are obtained.

In summary, surrogate-based upscaling provides substantial computational savings compared to numerical homogenization, achieving speedups of approximately one order of magnitude on the CPU and up to two orders of magnitude on the GPU. While numerical homogenization is dominated by mesh generation, SRF interpolation, and repeated solver runs, surrogate inference is significantly faster, with voxelization becoming the main computational bottleneck when GPUs are used. Overall, the results demonstrate that surrogates offer an efficient means of accelerating upscaling workflows, particularly when large numbers of homogenization blocks are required.

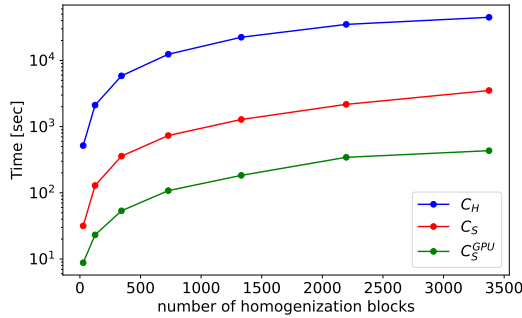


Figure 7.11: Comparison of computational time between numerical homogenization (C_H) and its surrogate counterpart. Surrogate inference is performed on both CPU (C_S) and GPU (C_S^{GPU}).

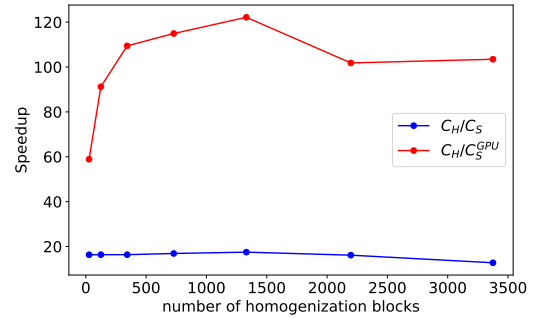


Figure 7.12: Speedup obtained by replacing numerical homogenization with the surrogate model. Results are reported for surrogate inference executed on CPU (C_H/C_S) and GPU (C_H/C_S^{GPU}).

7.10 Utilization of Upscaled Equivalent Hydraulic Conductivity Tensors

Although the surrogates exhibit high predictive accuracy (with NRMSE < 0.22 , corresponding to a coefficient of determination $R^2 > 0.95$), it remains necessary to

assess how this level of accuracy translates to practical applications. To this end, the influence of surrogate-predicted equivalent tensors $\mathbf{K}_H$ is examined for the Constraint and Anisotropy problems described in Section 3.3. Following the procedure outlined in Section 4.4, the homogenized or surrogate-predicted $\mathbf{K}_H$ fields are interpolated to the barycenters of bulk elements in the unstructured DFM mesh. For each test case, the outputs of the macroscale DFM models obtained using numerical homogenization are taken as the reference (target) values.

DFM samples characterized by $P_{30} = 0.0015$, DFN 0 configuration, $\lambda = 10$, fine mesh step size of $h = 5$, and coarse mesh step size of $H = 10$ are considered. Surrogates A, B, and C are used to predict $\mathbf{K}^{eq}$ values, which are subsequently assembled into the coarse-scale tensor field $\mathbf{K}_H$. Two domain sizes are analyzed: $\Omega_{UP} = (60, 60, 60)$ and $\Omega_{UP} = (15, 15, 15)$. The smaller domain represents a case in which the coarse sample contains only a small number of fractures, resulting in a limited influence on the overall hydraulic conductivity. For a homogenization block size of $b = 15$ ($1.5H$) with an overlap of $b/2$, the corresponding numbers of homogenization blocks per sample are 729 for $\Omega_{UP} = (60, 60, 60)$ and 27 for $\Omega_{UP} = (15, 15, 15)$, respectively.

7.10.1 Constraint Problem

The upscaled DFM models are first compared for the Constraint problem described in Section 3.3.1. Table 7.6 reports the NRMSE between the target outflow Y (obtained using numerical homogenization for upscaling) and the predicted outflow (obtained using surrogate-based upscaling) for the domain $\Omega_{UP} = (60, 60, 60)$. In all three cases, the use of surrogates to determine $\mathbf{K}_H$ results in coarse-scale outflow values that closely match the reference, with $\text{NRMSE} < 0.01$. Furthermore, the absolute values of Y are similar across the three K_f/K_m ratios, indicating that the outflow is primarily governed by the explicitly represented fractures. As the influence of fractures increases with K_f/K_m , the NRMSE decreases further, largely independent of differences in the predictive performance of the individual surrogates.

Table 7.6: Comparison of the NRMSE for the Constraint problem outflow Y obtained from coarse DFM models for different fracture-to-matrix conductivity ratios K_f/K_m . For each ratio, numerical homogenization and surrogate-based upscaling are compared. Parameters: $H = 10$, $h = 5$, $\Omega_{UP} = (60, 60, 60)$, and 250 samples.

		NRMSE
K_f/K_m		Y
Surrogate A	1×10^3	0.00968
Surrogate B	1×10^5	0.00418
Surrogate C	1×10^7	0.00381

Table 7.7 presents results for the smaller domain, $\Omega_{UP} = (15, 15, 15)$. The NRMSE values for the coarse model outflow Y - comparing numerical homoge-

nization and surrogate-based upscaling - reflect trends in the prediction accuracy of the surrogates on the test data (see Section 7.3). The closest agreement occurs for $K_f/K_m = 1 \times 10^3$, where Surrogate A also achieved the highest accuracy on the test dataset. For this smaller domain, the upscaled conductivities have a more pronounced impact on the outflow Y , resulting in generally worse NRMSE values compared to the larger domain case $\Omega_{UP} = (60, 60, 60)$.

Table 7.7: Comparison of the NRMSE for the Constraint problem outflow Y obtained from coarse DFM models for different fracture-to-matrix conductivity ratios K_f/K_m . For each ratio, numerical homogenization and surrogate-based upscaling are compared. Parameters: $H = 10$, $h = 5$, $\Omega_{UP} = (15, 15, 15)$, and 250 samples.

	K_f/K_m	$\frac{\text{NRMSE}}{Y}$
Surrogate A	1×10^3	0.01323
Surrogate B	1×10^5	0.06235
Surrogate C	1×10^7	0.06419

7.10.2 Anisotropy Problem

Second, the equivalent hydraulic conductivity tensors of the coarse DFM models upscaled using numerical homogenization and surrogate-based upscaling are compared. The equivalent hydraulic conductivity tensor $\mathbf{K}_{UP}^{eq}$ is computed for the entire domain Ω_{UP} following the procedure described in Section 3.3.2. As shown in Table 7.8, the NRMSE values for the case $\Omega_{UP} = (60, 60, 60)$ demonstrate that all surrogates provide highly accurate predictions of $\mathbf{K}^{eq}$, regardless of the value of $K_f/K_m \in \{1 \times 10^3, 1 \times 10^5, 1 \times 10^7\}$. The overall NRMSE is approximately an order of magnitude lower than that observed for the Constraint problem. This finding is consistent with results reported for the 2D case in [119], where the Constraint problem is shown to be more sensitive to matrix conductivity variations than the Anisotropy problem.

As shown in Table 7.9, the Anisotropy problem also exhibits a decline in the accuracy of the $\mathbf{K}_{UP}^{eq}$ components when the influence of fractures that are explicitly resolved by the coarse model is limited, as is the case for the smaller domain $\Omega_{UP} = (15, 15, 15)$. The accuracy of the off-diagonal components improves with increasing K_f/K_m . This behavior can be attributed to the decreasing relative contribution of the surrogate-predicted matrix conductivity as K_f/K_m increases, thereby reducing the sensitivity of the equivalent tensor to prediction errors in the matrix values.

Across both investigated problems, the results demonstrate that, even in scenarios where the influence of the upscaled conductivity is substantial, the trained surrogates produce outputs that remain in very close agreement with those obtained using standard numerical homogenization.

Table 7.8: Comparison of the NRMSE of the components of the upscaled equivalent hydraulic conductivity tensor $\mathbf{K}_{\text{UP}}^{eq}$ obtained from coarse DFM models for different fracture-to-matrix conductivity ratios K_f/K_m . For each ratio, surrogate-based upscaling is compared against numerical homogenization. Parameters: $H = 10$, $h = 5$, domain size $\Omega_{\text{UP}} = (60, 60, 60)$, and 250 samples.

		NRMSE					
	K_f/K_m	$\mathbf{k}_{xx}$	$\mathbf{k}_{yy}$	$\mathbf{k}_{zz}$	$\mathbf{k}_{yz}$	$\mathbf{k}_{xz}$	$\mathbf{k}_{xy}$
Surrogate A	1×10^3	0.00133	0.00054	0.00046	0.00057	0.00084	0.00053
Surrogate B	1×10^5	0.00038	0.00022	0.00030	0.00064	0.00037	0.00065
Surrogate C	1×10^7	0.00022	0.00022	0.00033	0.00041	0.00022	0.00025

Table 7.9: Comparison of the NRMSE of the components of the upscaled equivalent hydraulic conductivity tensor $\mathbf{K}_{\text{UP}}^{eq}$ obtained from coarse DFM models for different fracture-to-matrix conductivity ratios K_f/K_m . For each ratio, surrogate-based upscaling is compared against numerical homogenization. Parameters: $H = 10$, $h = 5$, domain size $\Omega_{\text{UP}} = (15, 15, 15)$, and 250 samples.

		NRMSE					
	K_f/K_m	$\mathbf{k}_{xx}$	$\mathbf{k}_{yy}$	$\mathbf{k}_{zz}$	$\mathbf{k}_{yz}$	$\mathbf{k}_{xz}$	$\mathbf{k}_{xy}$
Surrogate A	1×10^3	0.01222	0.01372	0.01793	0.03631	0.04027	0.03262
Surrogate B	1×10^5	0.01483	0.01297	0.01289	0.02076	0.01229	0.01897
Surrogate C	1×10^7	0.01543	0.01228	0.01247	0.01562	0.01272	0.01179

7.10.3 Comparison to Upscaling without Homogenization

Although the necessity of numerical homogenization to account for the influence of fractures has been assumed from the very beginning of this thesis, this subsection provides a comparison between upscaling based on homogenization and a simpler approach that directly reuses the fine SRF for a coarse problem. More specifically, as described in Section 4.4, the fine-mesh SRF $\mathbf{K}_h$ is interpolated from $\mathbf{K}_G$, a hydraulic conductivity SRF generated on a regular grid. In this section, the approach of reusing $\mathbf{K}_G$ to directly interpolate $\mathbf{K}_H$ is compared with the case where surrogates are employed for upscaling.

Figure 7.13 illustrates the results for the Constraint problem on the domain $\Omega_{\text{UP}} = (60, 60, 60)$. It is evident that omitting homogenization leads to an underestimation of the outflow Y in all three investigated cases of K_f/K_m . The outflow is highly sensitive to the presence of fractures, and even though only fractures smaller than H are excluded, the results are noticeably affected. Moreover, a slight but consistent trend can be observed: as the influence of fractures increases (i.e., with higher K_f/K_m), the NRMSE becomes worse for cases where no homogenization is applied during upscaling.

For the Anisotropy problem, Figure 7.14 presents the results for the k_{xx} com-

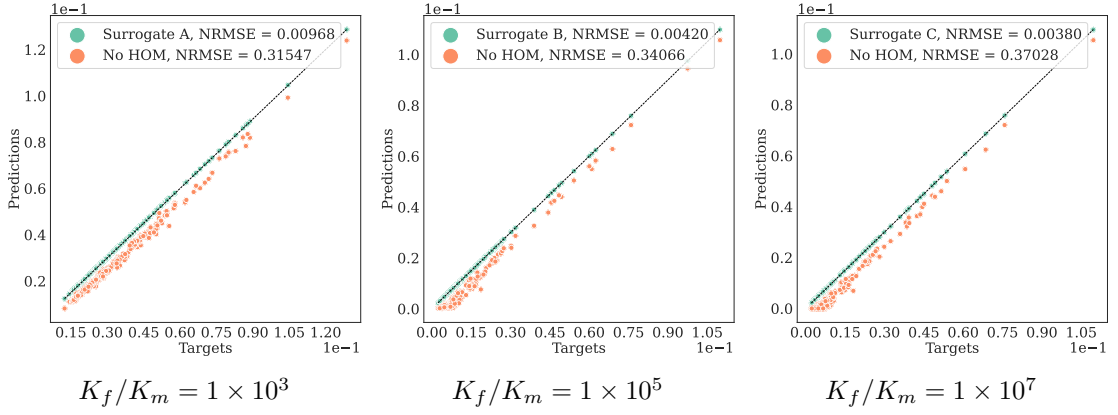


Figure 7.13: Accuracy comparison of the outflow Y from macroscale DFM models of the Constraint problem. Reference (target) values are obtained using numerical homogenization. Predicted values are obtained from models upscaled either via surrogate models or by reusing fine SRF data for coarse problems (No HOM). Parameters: $H = 10$, $h = 5$, $\Omega_{UP} = (60, 60, 60)$, and 250 samples.

ponent of K_{UP}^{eq} in the domain $\Omega_{UP} = (60, 60, 60)$. Although the NRMSE values obtained without homogenization are higher than those achieved using surrogates, their absolute magnitudes remain relatively small. Moreover, also with no homogenization, the NRMSE decreases with increasing fracture influence, reaching its lowest value for $K_f/K_m = 1 \times 10^7$. A slight underestimation is still apparent in the predictions. Overall, the Anisotropy problem exhibits lower sensitivity to the omission of small fractures compared to the Constraint problem on the same domain.

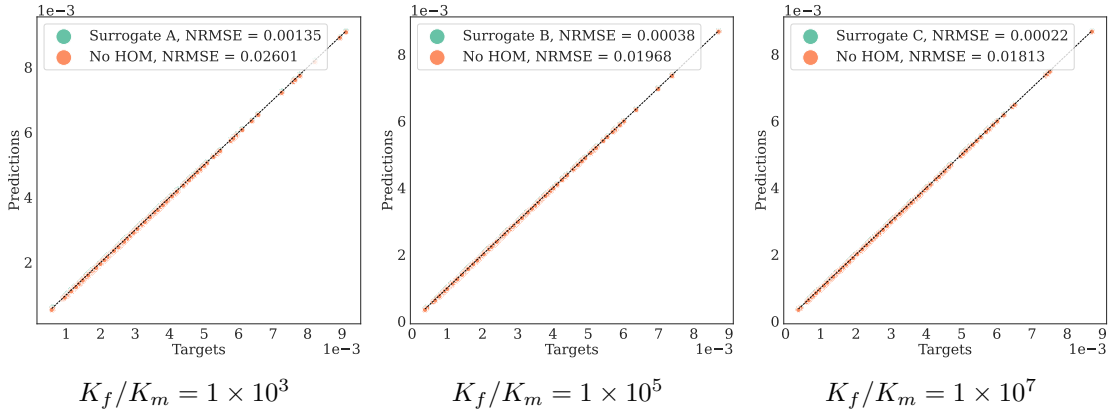


Figure 7.14: Accuracy comparison of the k_{xx} component of K_{UP}^{eq} from macroscale DFM models of the Anisotropy problem. Reference (target) values are obtained using numerical homogenization. Predicted values are obtained from models upscaled either via surrogate models or by reusing fine SRF data for coarse problems (No HOM). Parameters: $H = 10$, $h = 5$, $\Omega_{UP} = (60, 60, 60)$, and 250 samples.

To further illustrate the impact of missing fractures on the Anisotropy problem, the same setup is applied to a smaller domain, $\Omega_{UP} = (15, 15, 15)$. In this case, a clear deterioration is evident both in the NRMSE values and in the visual comparison plots, with a noticeable underestimation appearing from $K_f/K_m \geq 1 \times 10^5$. The

NRMSE values are now comparable to those observed for the Constraint problem on the larger domain, $\Omega_{UP} = (60, 60, 60)$, which is consistent with the Anisotropy problem being less sensitive to the effects of missing fractures than the Constraint problem.

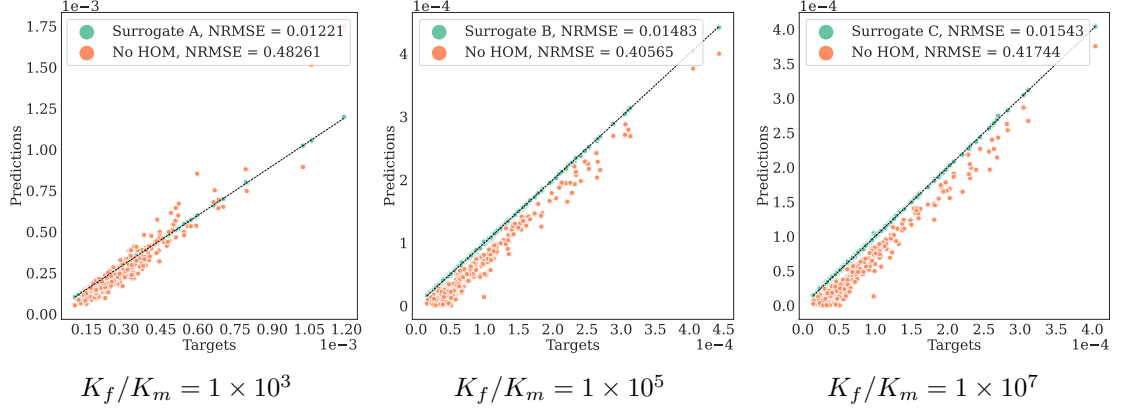


Figure 7.15: Accuracy comparison of the k_{xx} component of K_{UP}^{eq} from macroscale DFM models of the Anisotropy problem. Reference (target) values are obtained using numerical homogenization. Predicted values are obtained from models upscaled either via surrogate models or by reusing fine SRF data for coarse problems (no HOM). Parameters: $H = 10$, $h = 5$, $\Omega_{UP} = (15, 15, 15)$, and 250 samples.

8 Results on MLMC with Surrogate

In this chapter, the results obtained using Monte Carlo methods for discrete fracture-matrix (DFM) models are presented. Upscaling is required for the transition between fine- and coarse-scale samples on levels $l > 1$, and this is performed by default using trained surrogates A, B, and C with inference running on an NVIDIA Tesla T4 GPU (16 GB) to achieve the full acceleration potential. The appropriate surrogate is selected based on the fracture-to-matrix hydraulic conductivity ratio K_f/K_m , computed from the voxelized representation of the entire sample prior to subdivision into homogenization blocks. The exact same procedure as described in Section 4.4 is employed to construct MLMC for DFM models. Both the Constraint problem and the Anisotropy problem are investigated under identical settings. The computational domain $\Omega = (60, 60, 60)$ is considered, with the finest scale of mesh step $h = 5$ available. The spatial random field (SRF) on the finest level is prescribed with correlation length $\lambda = 10$, mean vector $\boldsymbol{\mu} = [-6.0, -5.8, -5.9]$, and covariance

matrix $\boldsymbol{\Sigma} = \begin{bmatrix} 0.25 & 0.2 & 0.2 \\ 0.2 & 0.25 & 0.2 \\ 0.2 & 0.2 & 0.25 \end{bmatrix}$. The fracture density is set to $P_{30} = 0.0015$.

For each problem, results of the standard MC method are first presented, including the reference MC estimates used as benchmark values. Subsequently, MLMC results obtained with different numbers of levels are compared to identify the configuration that yields the lowest total computational cost, C , measured as wall-clock time, for a prescribed target variance V_t . The cost C includes the runtime of both fine- and coarse-sample simulations, the generation of spatial random fields (SRFs), the construction of discrete fracture networks (DFNs), and the numerical homogenization step. In surrogate-accelerated configurations, the cost of numerical homogenization is replaced by the cost of surrogate inference (on CPU or GPU), while all remaining components of the workflow remain unchanged. Offline costs associated with prior surrogate training are excluded from C , as the focus is on the online performance of the MLMC estimator. Then the optimal number of samples per level, $\mathbf{N} = (N_1, \dots, N_L)$, is determined using the expressions provided in Section 4.2.2. Finally, the runtime decomposition is reported to quantify the contribution of homogenization costs, including comparisons between GPU- and CPU-based surrogate inference and runs performed entirely without surrogates.

Although the sampling algorithm described in Section 4.3.1 can be employed for determining $\mathbf{N}$, it is not used directly here due to the extensive number of MLMC configurations that must be executed on the computing cluster to obtain the results presented in this chapter. Instead, three modified iterations of the algorithm are

performed:

1. An initial sample set $\mathbf{N}_0^s$ is generated, defined by default as a geometric sequence decreasing from 500 for $l = 1$ to 10 for $l = L$. The specific values of $\mathbf{N}_0^s$ are not necessarily constrained to follow this geometric pattern. They may be increased to ensure that at least 10,000 homogenization samples are available per level, as motivated in Section 4.4. Particularly when a population $\mathcal{B}_l$ of equivalent hydraulic conductivity tensors (approach (A), see Section 4.4.2) is used to generate $\mathbf{K}_G$ for fine samples on a coarser levels ($l < L$). All samples are generated sequentially from the finest to the coarsest level; that is, samples for level $l = L$ are produced first, followed by those for $l = L - 1$, and so on down to level $l = 1$. The exact values of $\mathbf{N}_0^s$ used for each MLMC configuration are specified a priori.
2. Using the variance and cost estimated from $\mathbf{N}_0^s$, the target sample allocation $\mathbf{N}_0^t$ is determined and set to run on a cluster. In this step, samples at all levels can be run simultaneously.
3. To ensure the final allocation decision of $\mathbf{N}$ is robust and to reduce the risk of undersampling/oversampling, $\mathbf{N}$ is determined based on the variance and cost estimated from already generated $\mathbf{N}_0^t$ samples.

When it is not stated otherwise, only the approach (A) is used within this chapter to generate $\mathbf{K}_G$ for SRF for fine samples on levels $l < L$.

Within this chapter, MC/MLMC methods are employed to estimate the expectation and variance of a quantity of interest Z , which is modeled as a random variable. The expectation is denoted by $\phi_1(Z) = \mathbb{E}[Z]$. The variance, $\phi_2(Z) = \mathbb{V}[Z]$, is estimated by applying MC or MLMC to the derived random variable $(Z - \phi_1(Z))^2$, where the exact expectation $\phi_1(Z)$ is replaced in practice by its corresponding estimate. All results presented for the MC and MLMC methods are based on a single realization of the underlying random parameters ω . This choice is motivated by the computational expense of executing all investigated methods on a computing cluster with limited resources.

Table 8.1 presents an overview of the parameters of the investigated MLMC estimators. The table lists the mesh steps used at each level (h_1, h_2, h_3, h_4), with the finest level consistently fixed at $h = 5$. The number of homogenization blocks per sample is also reported, providing insight into how many equivalent hydraulic conductivity tensors are used to construct $\mathbf{K}_G$ at the coarser levels. To assemble $\mathcal{B}_l$ of 10,000 samples for the initial variance and cost estimates, the presented number of samples $\mathbf{N}_0^s$ is carried out. The configurations differ in the mesh step sizes prescribed at the coarser levels ($l < L$), and consequently in the number of homogenization blocks contained within each fine sample. For the two-level Monte Carlo methods, the coarse-level mesh step h_1 is defined as a multiple of the fine-level mesh step h_2 , using factors of $1.5\times$, $2\times$, $2.5\times$, $3\times$, and $4\times$. For the three-level MLMC estimators, multiplication factors of $1.5\times$ and $2\times$ are employed between successive levels. The four-level MLMC configuration applies a uniform multiplication factor of $1.5\times$ across all level transitions.

Table 8.1: Overview of all multilevel Monte Carlo (MLMC) methods investigated.

Name	mesh step				Hom. blocks per sample	N_0^s
	h_1	h_2	h_3	h_4		
2LMC-1	7.5	5			1,728	[500; 10]
2LMC-2	10	5			729	[500; 14]
2LMC-3	12.5	5			343	[500; 30]
2LMC-4	15	5			216	[500; 47]
2LMC-5	20	5			125	[500; 80]
3LMC-1	11.25	7.5	5		[512; 1,728]	[500; 20; 10]
3LMC-2	20	10	5		[125; 729]	[500; 80; 14]
4LMC-1	16.875	11.25	7.5	5	[216; 512; 1,728]	[500; 47; 20; 10]

8.1 Multilevel Monte Carlo Estimator Validity

In order for the MLMC estimator described in Section 4.2 to be valid, the samples P_l are required to be independent across levels, and the condition $\mathbb{E}[P_l^f] = \mathbb{E}[P_l^c]$ is assumed to hold to ensure unbiasedness of the estimator. Independence across levels is ensured by construction: independent realizations of DFNs are employed together with sampling with replacement from the large populations $\mathcal{B}_l$ when approach (A) is used, whereas independent realizations of the SRF $\mathbf{K}_h$ are generated when approach (B) is adopted. In contrast, approach (A) employs different procedures for generating samples P_l^f and P_l^c , and the unbiasedness of the resulting estimator cannot be guaranteed a priori. For this reason, this issue is further experimentally examined.

Both approaches are employed to generate fine-scale SRFs on a coarser level. For simplicity, the two-level Monte Carlo method is considered, in which these approaches are applied to the SRF on the coarsest level ($l = 1$). Approach (B) does not rely on sampling SRF values from a precomputed population of tensors. Instead, the full homogenization procedure is executed, ensuring that the SRFs for P_1^c and P_1^f are generated in an identical manner. As a result, the condition $\mathbb{E}[P_1^f] = \mathbb{E}[P_1^c]$ is expected to hold. Consequently, samples generated using approach (B) are adopted as a benchmark in this section. The Constraint problem, which exhibits higher sensitivity to mesh resolution, is used for this comparison.

Figure 8.1 presents log-scaled distributions of the first and second moments, ϕ_1 and ϕ_2 , computed from 15,000 corresponding realizations of P_1^f generated using approach (A), with $|\mathcal{B}| = 15,000$, and approach (B). Results are shown for four representative values of the coarse mesh step $h_1 \in \{7.5, 15, 20, 40\}$, with the homogenization block size set to $b = 1.5h_1$, the fine mesh step fixed at $h_2 = 5$, and the domain $\Omega = (60, 60, 60)$. To assess whether the two approaches yield comparable mean values of ϕ_1 and ϕ_2 , a Welch two-sample t -test is applied with a significance level $\alpha = 0.05$. Welch's test does not require the assumption of equal variances and is well suited to large-sample settings such as the presented case, with 15,000

realizations per approach.

For cases in which h_1 remains relatively close to h_2 , specifically $h_1 \in \{7.5, 15\}$, close agreement is observed between the mean values of both ϕ_1 and ϕ_2 obtained using approaches (A) and (B). In these cases, the Welch t -test yields p -values of at least 0.75, indicating that no statistically significant difference between the corresponding mean values is detected at the chosen significance level. For larger values of $h_1 \in \{20, 40\}$, the agreement between the mean values deteriorates. In these configurations, the homogenization procedure produces fewer, larger homogenization blocks, with only a few fractures remaining after the homogenization process. For example, when $h_1 = 40$, only 27 homogenization blocks are present per sample, capturing fractures with sizes up to 40. The underlying causes of this behavior remain subject to future investigation. A plausible explanation is that spatial patterns emerging in a small number of large homogenization blocks are not preserved when SRF values are sampled from the population $\mathcal{B}_l$ without accounting for positional information, resulting in discrepancies in the mean values of the simulated quantities. When the mean values differ, the MLMC estimator becomes biased. Depending on the magnitude of this bias, the accuracy and applicability of the resulting estimates may be significantly affected. Consequently, comparisons with reference solutions are required, and limitations associated with large discrepancies between mesh resolutions across levels must be taken into account. For these reasons, the maximum coarse mesh step size is restricted to $h_1 = 20$ in the investigated MLMC estimators. Even though, in the considered three-level and four-level MLMC configurations, the differences between successive mesh resolutions h_l are smaller, which is expected to improve agreement between mean values across levels.

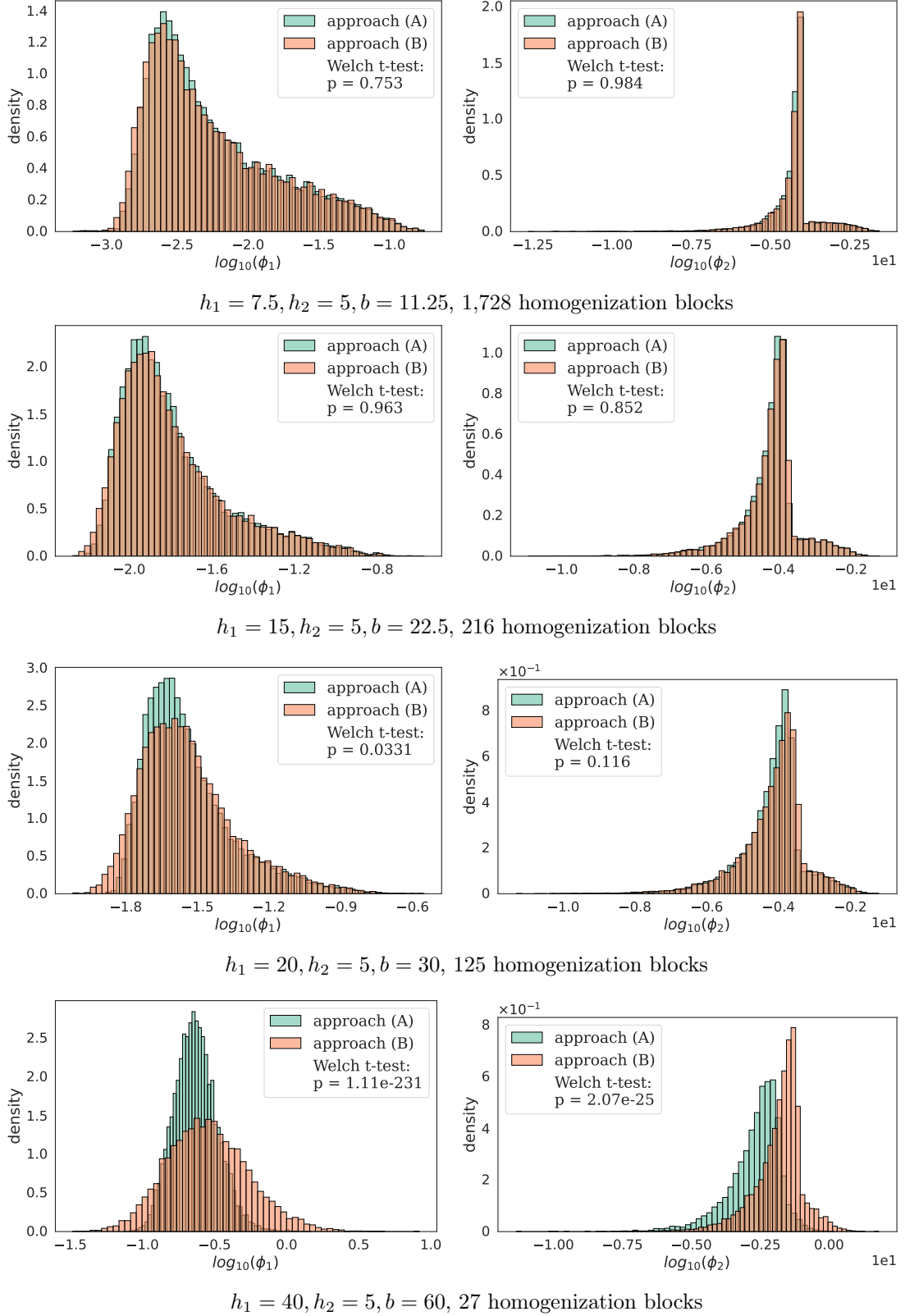


Figure 8.1: Comparison of distributions of estimated moments obtained by approach (A) and approach (B), plotted on a logarithmic scale for improved visualization. Statistics are calculated from the original estimated data, with 15,000 samples per dataset and $\Omega = (60, 60, 60)$.

8.2 Constraint Problem

The MC/MLMC estimates are first evaluated for the Constraint problem introduced in Section 3.3.1, which yields a single scalar quantity. The sample allocations are chosen to ensure that the corresponding estimator achieves the prescribed target variance $V_t = 5 \times 10^{-8}$ ($\sqrt{V_t} \approx 2.24 \times 10^{-4}$).

8.2.1 Reference and Standard Monte Carlo

Since the quantity of interest P is not directly accessible computationally, a high-accuracy reference standard Monte Carlo simulation with a target variance of $V_t^{\text{ref}} = 1 \times 10^{-9}$ ($\sqrt{V_t^{\text{ref}}} \approx 3.16 \times 10^{-5}$) is conducted to provide benchmark values for assessing the bias of all other MC and MLMC estimators. All remaining settings, including the geometry, fracture configurations, and SRF parameters, remain identical. The reference MC yields the moment estimates

$$\begin{aligned}\tilde{\phi}_1^{\text{ref}} &= 0.0095611 \quad \text{with} \quad \sqrt{\tilde{V}_{\tilde{\phi}_1^{\text{ref}}}} = 3.16 \times 10^{-5}, \\ \tilde{\phi}_2^{\text{ref}} &= 0.00034871 \quad \text{with} \quad \sqrt{\tilde{V}_{\tilde{\phi}_2^{\text{ref}}}} = 2.63 \times 10^{-6}.\end{aligned}$$

based on $N = 346,032$ samples, with a total computational cost $C = 594,134,550$ sec.

For comparison, a standard single-level Monte Carlo simulation is performed under the same settings to meet V_t . Using $N = 7,686$ samples, the estimated moments are:

$$\begin{aligned}\tilde{\phi}_1 &= 0.0098516 \quad \text{with} \quad \sqrt{\tilde{V}_{\tilde{\phi}_1}} = 2.17 \times 10^{-4}, \\ \tilde{\phi}_2 &= 0.00036081 \quad \text{with} \quad \sqrt{\tilde{V}_{\tilde{\phi}_2}} = 1.73 \times 10^{-5}.\end{aligned}$$

with a total computational cost $C = 13,196,808$ sec. The absolute differences $|\tilde{\phi}_1 - \tilde{\phi}_1^{\text{ref}}| = 2.91 \times 10^{-4}$ and $|\tilde{\phi}_2 - \tilde{\phi}_2^{\text{ref}}| = 1.21 \times 10^{-5}$ are taken as reference values for subsequent comparison with MLMC results.

8.2.2 MLMC Estimators

The computational cost of the MLMC estimators is analyzed in the first place. Table 8.2 shows the total computational costs C , costs per level sample C_l , and final number of samples collected $\mathbf{N}$ of the investigated MLMC estimators. For 2LMC, there is an increased number of samples performed on $l = 2$ with increasing h_1 , which is attributed to the increased variance of differences on these levels as well. Although C_1 generally decreases as the coarse simulation on this level uses a larger mesh step, this cost is also influenced by the necessity of generating initial SRF $\mathbf{K}_{h_2}$ on an extended domain in order to ensure that larger homogenization blocks capture boundary regions, as described in Section 4.4. Overall, 3LMC-2 provides the smallest $C = 767,955$, which accounts for ca. 6% of the computational cost of the single-level MC method.

Table 8.2: Computational cost of MLMC estimators for the Constraint problem.

	N	C_l	C
2LMC-1	[7,557; 10]	[360; 3,191]	2,749,033
2LMC-2	[9,275; 61]	[150; 2,009]	1,517,726
2LMC-3	[11,243; 139]	[70; 2,136]	1,086,882
2LMC-4	[14,057; 379]	[45; 1,646]	1,255,154
2LMC-5	[16,731; 1,475]	[22; 1,666]	2,828,701
3LMC-1	[9,241; 49; 13]	[105; 619; 3,357]	1,047,671
3LMC-2	[16,523; 748; 86]	[24; 220; 2,386]	767,955
4LMC-1	[11,564; 302; 69; 30]	[49; 240; 806; 3,101]	792,204

Computational cost is a key consideration. However, the estimated mean values and standard errors of the estimators must also be examined. Table 8.3 reports the estimated mean values of the moments $\tilde{\phi}_1$ and $\tilde{\phi}_2$, their absolute deviations from the reference values, $|\tilde{\phi}_1 - \tilde{\phi}_1^{\text{ref}}|$ and $|\tilde{\phi}_2 - \tilde{\phi}_2^{\text{ref}}|$, as well as the corresponding estimator standard errors $\sqrt{\tilde{V}_{\phi_1}}$ and $\sqrt{\tilde{V}_{\phi_2}}$. When determining the number of samples required to satisfy the target variance V_t (resp. $\sqrt{V_t}$), the larger of the two values is taken as the governing quantity; in this case, $\sqrt{\tilde{V}_{\phi_1}}$ dominates. Although the achieved estimator standard errors do not match the prescribed target value $\sqrt{V_t}$ exactly, their magnitudes are sufficiently close to allow a meaningful comparison, and no further refinement of the sample allocation is required. For most estimators, the absolute deviations of the estimated moments from their reference values are smaller than, or of the same order as, the corresponding standard errors $\sqrt{\tilde{V}_{\phi_1}}$ and $\sqrt{\tilde{V}_{\phi_2}}$. This indicates that the observed discrepancies are consistent with the expected statistical uncertainty of the estimators and are comparable to those obtained with the standard Monte Carlo method. The remaining differences are attributed to statistical fluctuations arising from the use of a single realization of ω .

Figure 8.2 presents the level variances V_l as functions of the mesh size h_l on a logarithmic scale. Under geometric refinement, the MLMC variance bound $V_l \leq c_2 2^{-\beta l}$ from Theorem 4.2.1 is equivalently expressed in terms of the mesh size as $V_l \sim h_l^\beta$. The slopes shown in the figure therefore correspond to the MLMC variance decay exponent β , estimated by linear regression in log-log scale. For each hierarchy, the larger value obtained from the two moments (ϕ_1, ϕ_2) is reported. Figure 8.3 analogously shows the level-dependent computational costs C_l together with the corresponding MLMC cost exponent γ . Taken together, these plots provide insight into the efficiency of the investigated MLMC estimators. As stated in the theorem, the most favorable regime is obtained when the dominant computational cost is incurred on the low-accuracy levels ($\beta > \gamma$) or when cost and variance are distributed evenly across the levels ($\beta = \gamma$). This condition is satisfied for all examined methods except 2LMC-4 ($\beta = 1.87, \gamma = 3.28$) and 2LMC-5 ($\beta = 1.49, \gamma = 3.12$), for which

Table 8.3: Moment estimates and corresponding standard errors of MLMC estimators for the Constraint problem.

	$\sqrt{\tilde{V}_{\phi_1}}$	$\tilde{\phi}_1$	$ \tilde{\phi}_1 - \tilde{\phi}_1^{\text{ref}} $	$\sqrt{\tilde{V}_{\phi_2}}$	$\tilde{\phi}_2$	$ \tilde{\phi}_2 - \tilde{\phi}_2^{\text{ref}} $
2LMC-1	2.51×10^{-4}	0.0099039	3.43×10^{-4}	1.67×10^{-5}	0.00035498	6.27×10^{-6}
2LMC-2	2.53×10^{-4}	0.0094558	1.05×10^{-4}	1.70×10^{-5}	0.00034804	6.70×10^{-7}
2LMC-3	2.39×10^{-4}	0.0098878	3.27×10^{-4}	1.48×10^{-5}	0.00034649	2.22×10^{-6}
2LMC-4	2.32×10^{-4}	0.0097778	2.17×10^{-4}	1.45×10^{-5}	0.00035192	3.21×10^{-6}
2LMC-5	2.32×10^{-4}	0.0095698	8.70×10^{-6}	1.38×10^{-5}	0.00031334	3.54×10^{-5}
3LMC-1	2.47×10^{-4}	0.0094692	9.19×10^{-5}	1.67×10^{-5}	0.00034608	2.63×10^{-6}
3LMC-2	2.35×10^{-4}	0.0095649	3.80×10^{-6}	1.41×10^{-5}	0.00034673	1.98×10^{-6}
4LMC-1	2.51×10^{-4}	0.0097320	1.71×10^{-4}	1.52×10^{-5}	0.00032192	2.68×10^{-5}

the dominant computational cost is incurred on the finest level.

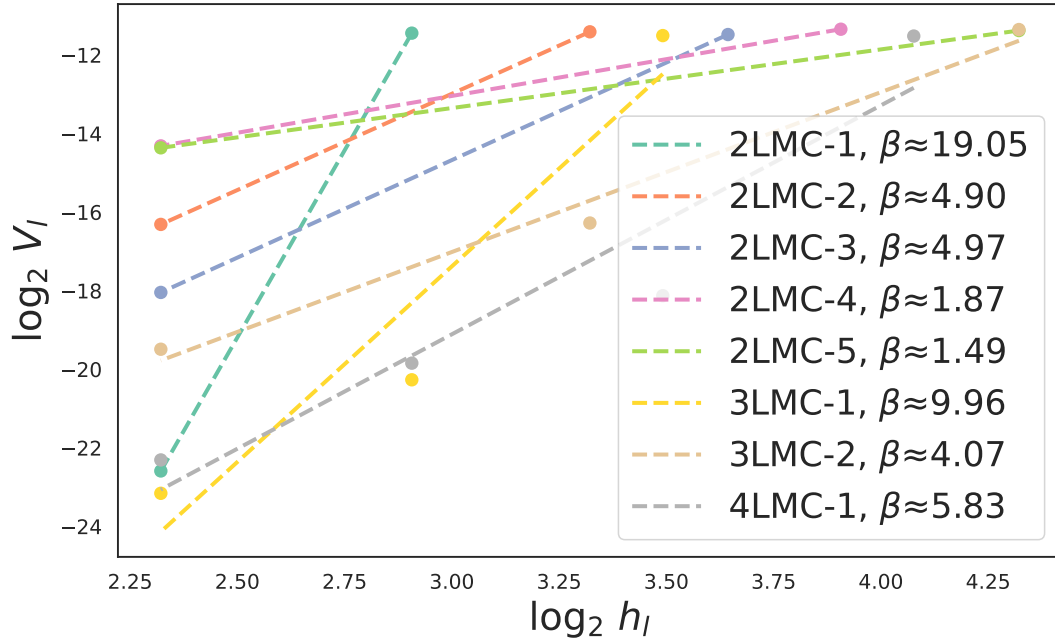


Figure 8.2: Level variances V_l for the Constraint problem, plotted against the mesh step h_l on a logarithmic scale. The fitted slope parameter β from MLMC Theorem 4.2.1 quantifies the decay rate of V_l with the level.

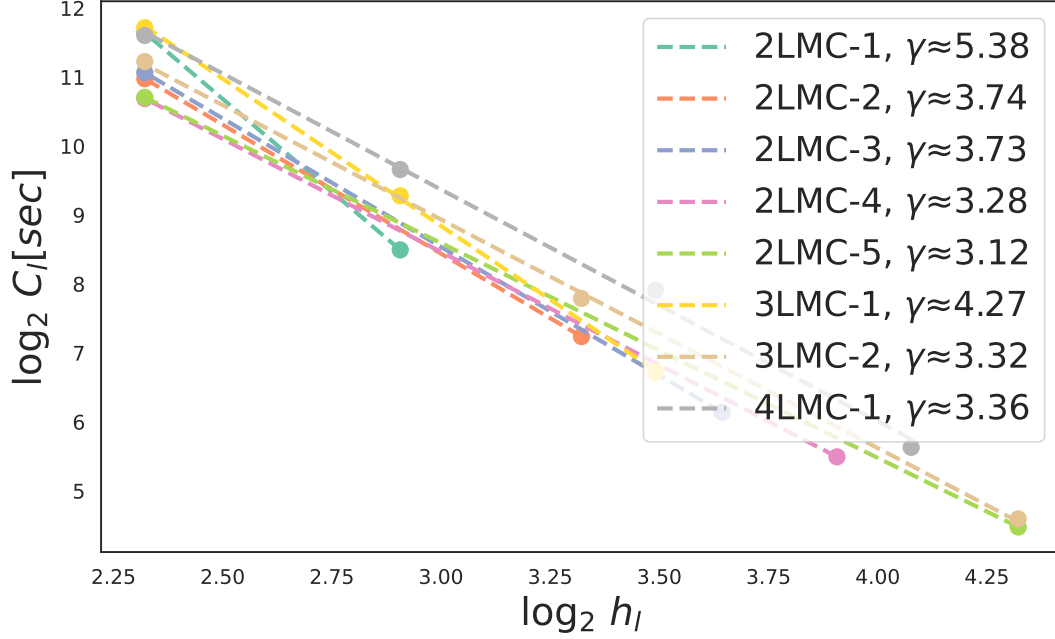


Figure 8.3: Level computational cost C_l for the Constraint problem, plotted against the mesh step h_l on a logarithmic scale. The fitted slope parameter γ from MLMC Theorem 4.2.1 describes the growth of C_l with level.

8.2.3 Performance Assessment of 3LMC-2

The computationally most efficient method, 3LMC-2, is further analyzed in this section. Its performance is evaluated in comparison with a variant employing surrogate inference on the CPU (3LMC-2-CPU) and with a configuration using numerical homogenization without a surrogate (3LMC-2-NH), in which the simulator is executed separately for each homogenization block. To enable clear comparison, the 3LMC-2 computational costs are reused for all three cases, with the exception of the homogenization cost part, which is the only difference among them.

Table 8.4 reports the total computational costs for the configurations 3LMC-2, 3LMC-2-CPU, and 3LMC-2-NH, all determined to meet the target variance V_t . Using the CPU for surrogate inference increases the total cost C by approximately 13% compared to GPU-based inference. Replacing surrogate-based upscaling with numerical homogenization results in a substantially higher cost, with C increasing by approximately 279%. Nevertheless, relative to the standard Monte Carlo method ($C = 13,196,808$), even the most expensive configuration, 3LMC-2-NH, requires only approximately 23% of the computational cost.

To gain a better insight into the cost parts, Figure 8.4 shows the runtime breakdown of the main computational components, averaged over 20 samples. Computational times are compared for levels $l \in \{2, 3\}$ of 3LMC-2, 3LMC-2-CPU, and 3LMC-2-NH. Although the GPU inference for the surrogate is generally faster (see Section 7.9) than CPU inference, the difference is not so significant compared to carrying out numerical homogenization without a surrogate. In that case, homoge-

Table 8.4: Computational cost of 3LMC-2 variants for the Constraint problem.

	N	C_l	C
3LMC-2	[16,523; 748; 86]	[24; 220; 2,386]	767,955
3LMC-2-CPU	[17,405; 748; 86]	[24; 302; 2,593]	867,851
3LMC-2-NH	[22,103; 578; 85]	[24; 2,356; 11,776]	2,894,770

nization dominates the runtime and is $64\times$ slower for $l = 2$ and $91\times$ slower for $l = 3$, compared to the GPU run. When surrogates are used, the fine simulation run is the most costly one on both levels, with the fine SRF creation time being negligible for $l = 2$. For $l = 3$ and GPU inference, the homogenization takes only 3.5% of the total runtime of a sample.

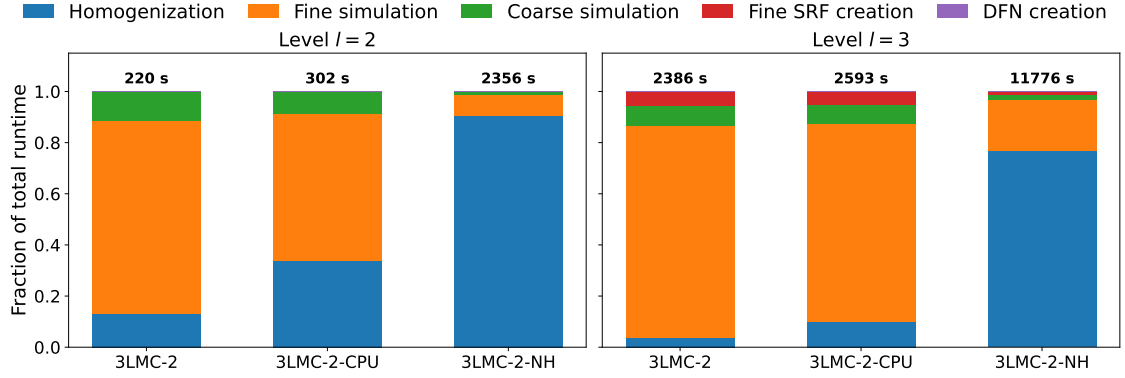


Figure 8.4: Normalized runtime breakdown for the Constraint problem using three-level MLMC estimators. For levels $l \in \{2, 3\}$, stacked bars show the fractional contribution of the main computational components. 3LMC-2 (GPU-based surrogate inference), 3LMC-2-CPU (CPU-based surrogate inference), and 3LMC-2-NH (run without surrogates) are compared. Absolute total runtimes for each configuration are annotated above the bars.

The important cost-variance relation, alongside the MLMC Theorem parameters β and γ , is depicted in Figure 8.5. A key observation is that 3LMC-2-NH yields no better variance reduction across levels compared to the 3LMC-2. Additionally, for 3LMC-2-NH, $\beta < \gamma$, indicating that the dominant computational cost is on the finest level. The efficiency is not optimal, and potentially adding a new coarsest level could provide only a limited benefit.

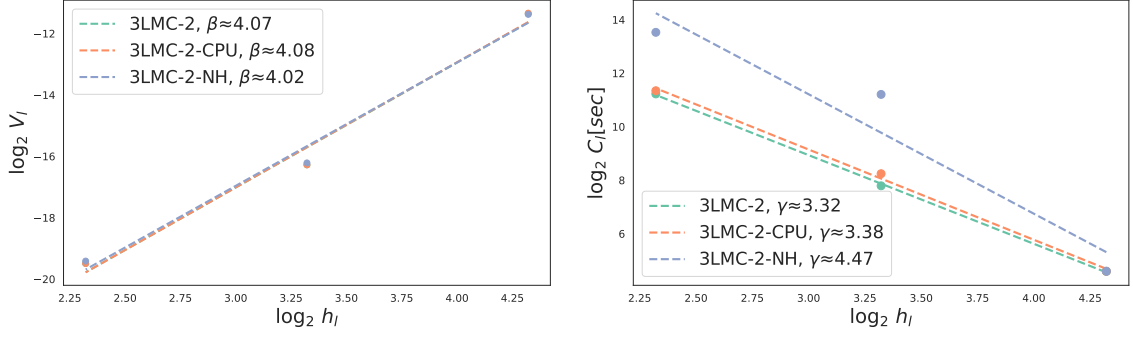


Figure 8.5: Variance (left) and computational cost (right) per level of 3LMC-2 variants for the Constraint problem. The level is determined by the mesh step h_l . Parameters β and γ correspond to the MLMC Theorem.

8.3 Anisotropy Problem

The MC/MLMC estimates are subsequently evaluated for the Anisotropy problem, as defined in Section 3.3.2. It yields equivalent hydraulic conductivity tensors, treated as six quantities of interest. Given their relatively small values, the target variance for the estimators is set to $V_t = 2.5 \times 10^{-10}$ ($\sqrt{V_t} \approx 1.58 \times 10^{-5}$).

8.3.1 Reference and Standard Monte Carlo

Reference standard Monte Carlo estimates are conducted with a target variance $V_t^{\text{ref}} = 1 \times 10^{-11}$ ($\sqrt{V_t^{\text{ref}}} \approx 3.16 \times 10^{-6}$) and yield the moment estimates:

$$\begin{aligned} \tilde{\phi}_1^{\text{ref}} &= [2.85 \times 10^{-3}, 3.12 \times 10^{-3}, 2.49 \times 10^{-3}, 1.47 \times 10^{-4}, -4.63 \times 10^{-6}, 5.96 \times 10^{-4}], \\ \text{with } \sqrt{\tilde{V}_{\tilde{\phi}_1^{\text{ref}}}} &= [4.24 \times 10^{-6}, 4.12 \times 10^{-6}, 2.53 \times 10^{-6}, 6.78 \times 10^{-7}, 7.00 \times 10^{-7}, 3.32 \times 10^{-6}], \\ \tilde{\phi}_2^{\text{ref}} &= [6.14 \times 10^{-6}, 5.93 \times 10^{-6}, 2.19 \times 10^{-6}, 1.58 \times 10^{-7}, 1.66 \times 10^{-7}, 3.67 \times 10^{-6}], \\ \text{with } \sqrt{\tilde{V}_{\tilde{\phi}_2^{\text{ref}}}} &= [3.61 \times 10^{-6}, 3.10 \times 10^{-6}, 1.34 \times 10^{-8}, 6.00 \times 10^{-9}, 6.40 \times 10^{-9}, 3.32 \times 10^{-6}] \end{aligned}$$

based on $N = 341,273$ samples, with a total computational cost $C = 1,206,895,469$ sec.

The standard single-level Monte Carlo method is performed to meet V_t and uses $N = 11,070$ samples to obtain estimates:

$$\begin{aligned} \tilde{\phi}_1 &= [2.83 \times 10^{-3}; 3.10 \times 10^{-3}; 2.46 \times 10^{-3}; 1.42 \times 10^{-4}; -6.85 \times 10^{-6}; 5.80 \times 10^{-4}], \\ \text{with } \sqrt{\tilde{V}_{\tilde{\phi}_1}} &= [1.48 \times 10^{-5}, 1.58 \times 10^{-5}, 1.38 \times 10^{-5}, 3.61 \times 10^{-6}, 3.74 \times 10^{-6}, 5.20 \times 10^{-6}], \\ \tilde{\phi}_2 &= [2.45 \times 10^{-6}; 2.78 \times 10^{-6}; 2.14 \times 10^{-6}; 1.47 \times 10^{-7}; 1.56 \times 10^{-7}; 2.98 \times 10^{-7}], \\ \text{with } \sqrt{\tilde{V}_{\tilde{\phi}_2}} &= [4.58 \times 10^{-8}, 5.20 \times 10^{-8}, 4.69 \times 10^{-8}, 3.61 \times 10^{-9}, 3.61 \times 10^{-9}, 7.14 \times 10^{-9}] \end{aligned}$$

with a total computational cost $C = 24,317,334$ sec. The absolute differences

$$|\tilde{\phi}_1 - \tilde{\phi}_1^{\text{ref}}| = [1.61 \times 10^{-5}; 2.19 \times 10^{-5}; 2.77 \times 10^{-5}; 5.72 \times 10^{-6}; 2.22 \times 10^{-6}; 1.64 \times 10^{-5}]$$

$$|\tilde{\phi}_2 - \tilde{\phi}_2^{\text{ref}}| = [3.69 \times 10^{-6}; 3.15 \times 10^{-6}; 4.85 \times 10^{-8}; 1.08 \times 10^{-8}; 9.34 \times 10^{-9}; 3.38 \times 10^{-9}]$$

are again taken as reference for subsequent comparison with the MLMC results.

8.3.2 MLMC Estimators

Table 8.5 reports the total computational cost C , level-wise sample costs C_l , and the final number of samples $\mathbf{N}$ for the considered MLMC estimators. Across all the presented estimators, the number of samples on the finest levels ($l > 1$) is generally low and is mainly determined by the minimum set in $\mathbf{N}_0^s$. It means that variances are mainly only about an order of magnitude higher than the desired V_t . The Anisotropy problem is not so sensitive to mesh resolutions, and it might be potentially useful to increase h at the finest level. Consistent with the Constraint problem, the 3LMC-2 estimator achieves the lowest total cost, here $C = 428,651$, corresponding to approximately 1.8% of the single-level MC cost ($C = 24,317,334$).

Table 8.5: Computational cost of MLMC estimators for the Anisotropy problem.

Name	$\mathbf{N}$	C_l	C
2LMC-1	[10,349; 10]	[510; 3,754]	5,826,764
2LMC-2	[12,906; 26]	[196; 2,679]	2,602,397
2LMC-3	[11,599; 30]	[93; 2,492]	1,154,129
2LMC-4	[17,318; 257]	[83; 2,033]	1,955,894
2LMC-5	[13,649; 80]	[57; 2,243]	958,223
3LMC-1	[11,329; 32; 10]	[188; 1,006; 3,965]	2,203,644
3LMC-2	[11,731; 80; 14]	[31; 345; 2,989]	428,651
4LMC-1	[11,350; 47; 22; 10]	[62; 330; 1,158; 4,522]	787,623

Table 8.6 summarizes, for each MLMC estimator, the estimates of the two moment quantities $\tilde{\phi}_1$ and $\tilde{\phi}_2$ across the six tensor components, together with their absolute deviations from the corresponding reference values and the associated estimator standard errors. For all MLMC estimators and tensor components, the standard error $\sqrt{\tilde{V}_{\phi_1}}$ is consistently several orders of magnitude larger than $\sqrt{\tilde{V}_{\phi_2}}$ and therefore determines the sample allocation required to meet the prescribed target variance V_t (resp. $\sqrt{V_t}$). At the considered scale, the uncertainty associated with $\tilde{\phi}_2$ is negligible. The estimated moments are in good agreement with the reference solutions. For both $\tilde{\phi}_1$ and $\tilde{\phi}_2$, the absolute deviations from the reference values are generally smaller than, or comparable to, the corresponding standard errors. No systematic dependence on the MLMC hierarchy or tensor component

is observed. Overall, all MLMC estimators achieve comparable accuracy for the Anisotropy problem, and the differences between methods are therefore primarily attributable to computational cost rather than estimator quality.

Table 8.6: Moment estimates and corresponding standard errors of MLMC estimators for the Anisotropy problem.

Estimator	Item	$\sqrt{\tilde{V}_{\phi_1}}$	$\tilde{\phi}_1$	$ \tilde{\phi}_1 - \tilde{\phi}_1^{\text{ref}} $	$\sqrt{\tilde{V}_{\phi_2}}$	$\tilde{\phi}_2$	$ \tilde{\phi}_2 - \tilde{\phi}_2^{\text{ref}} $
2LMC-1	1	1.48×10^{-5}	2.85×10^{-3}	3.63×10^{-6}	4.90×10^{-8}	2.49×10^{-6}	3.65×10^{-6}
	2	1.58×10^{-5}	3.13×10^{-3}	4.40×10^{-6}	5.20×10^{-8}	2.81×10^{-6}	3.12×10^{-6}
	3	1.38×10^{-5}	2.50×10^{-3}	1.54×10^{-6}	4.36×10^{-8}	2.19×10^{-6}	8.08×10^{-9}
	4	3.74×10^{-6}	1.45×10^{-4}	2.59×10^{-6}	3.87×10^{-9}	1.54×10^{-7}	4.53×10^{-9}
	5	3.74×10^{-6}	-3.36×10^{-6}	1.27×10^{-6}	3.61×10^{-9}	1.59×10^{-7}	6.67×10^{-9}
	6	5.29×10^{-6}	5.94×10^{-4}	2.05×10^{-6}	6.78×10^{-9}	3.17×10^{-7}	3.35×10^{-6}
2LMC-2	1	1.38×10^{-5}	2.88×10^{-3}	3.03×10^{-5}	4.58×10^{-8}	2.50×10^{-6}	3.63×10^{-6}
	2	1.48×10^{-5}	3.16×10^{-3}	3.57×10^{-5}	5.20×10^{-8}	2.85×10^{-6}	3.08×10^{-6}
	3	1.30×10^{-5}	2.53×10^{-3}	3.78×10^{-5}	4.12×10^{-8}	2.18×10^{-6}	8.78×10^{-9}
	4	3.46×10^{-6}	1.50×10^{-4}	2.56×10^{-6}	3.46×10^{-9}	1.55×10^{-7}	3.11×10^{-9}
	5	3.61×10^{-6}	-3.80×10^{-6}	8.30×10^{-7}	3.61×10^{-9}	1.63×10^{-7}	2.92×10^{-9}
	6	5.00×10^{-6}	6.06×10^{-4}	9.50×10^{-6}	6.63×10^{-9}	3.27×10^{-7}	3.34×10^{-6}
2LMC-3	1	1.48×10^{-5}	2.89×10^{-3}	3.93×10^{-5}	4.58×10^{-8}	2.52×10^{-6}	3.62×10^{-6}
	2	1.55×10^{-5}	3.17×10^{-3}	4.18×10^{-5}	5.10×10^{-8}	2.81×10^{-6}	3.12×10^{-6}
	3	1.38×10^{-5}	2.51×10^{-3}	1.72×10^{-5}	4.36×10^{-8}	2.17×10^{-6}	1.95×10^{-8}
	4	3.61×10^{-6}	1.48×10^{-4}	5.96×10^{-7}	3.61×10^{-9}	1.56×10^{-7}	2.06×10^{-9}
	5	3.74×10^{-6}	-8.70×10^{-6}	4.08×10^{-6}	3.61×10^{-9}	1.63×10^{-7}	2.38×10^{-9}
	6	5.20×10^{-6}	5.92×10^{-4}	3.90×10^{-7}	6.63×10^{-9}	3.11×10^{-7}	3.36×10^{-6}
2LMC-4	1	1.34×10^{-5}	2.85×10^{-3}	8.23×10^{-6}	4.47×10^{-8}	2.52×10^{-6}	3.61×10^{-6}
	2	1.48×10^{-5}	3.14×10^{-3}	2.13×10^{-5}	5.10×10^{-8}	2.88×10^{-6}	3.05×10^{-6}
	3	1.41×10^{-5}	2.50×10^{-3}	1.10×10^{-5}	4.12×10^{-8}	2.22×10^{-6}	3.09×10^{-8}
	4	3.08×10^{-6}	1.48×10^{-4}	4.43×10^{-7}	2.98×10^{-9}	1.53×10^{-7}	5.22×10^{-9}
	5	3.46×10^{-6}	-8.17×10^{-6}	3.54×10^{-6}	3.11×10^{-9}	1.62×10^{-7}	3.44×10^{-9}
	6	4.36×10^{-6}	5.96×10^{-4}	2.73×10^{-7}	5.66×10^{-9}	3.21×10^{-7}	3.35×10^{-6}
2LMC-5	1	1.45×10^{-5}	2.86×10^{-3}	1.83×10^{-5}	5.00×10^{-8}	2.49×10^{-6}	3.64×10^{-6}
	2	1.58×10^{-5}	3.15×10^{-3}	2.17×10^{-5}	6.16×10^{-8}	2.85×10^{-6}	3.07×10^{-6}
	3	1.30×10^{-5}	2.51×10^{-3}	1.54×10^{-5}	4.58×10^{-8}	2.17×10^{-6}	1.92×10^{-8}
	4	3.32×10^{-6}	1.49×10^{-4}	1.87×10^{-6}	3.46×10^{-9}	1.52×10^{-7}	5.64×10^{-9}
	5	3.46×10^{-6}	-6.29×10^{-6}	1.66×10^{-6}	3.61×10^{-9}	1.63×10^{-7}	2.31×10^{-9}
	6	4.90×10^{-6}	5.98×10^{-4}	2.12×10^{-6}	6.32×10^{-9}	3.19×10^{-7}	3.35×10^{-6}
3LMC-1	1	1.48×10^{-5}	2.87×10^{-3}	2.04×10^{-5}	4.80×10^{-8}	2.51×10^{-6}	3.63×10^{-6}
	2	1.58×10^{-5}	3.14×10^{-3}	1.47×10^{-5}	5.10×10^{-8}	2.78×10^{-6}	3.15×10^{-6}
	3	1.41×10^{-5}	2.51×10^{-3}	2.30×10^{-5}	4.36×10^{-8}	2.21×10^{-6}	2.60×10^{-8}

Continued on next page

Estimator	Item	$\sqrt{\tilde{V}_{\phi_1}}$	$\tilde{\phi}_1$	$ \tilde{\phi}_1 - \tilde{\phi}_1^{\text{ref}} $	$\sqrt{\tilde{V}_{\phi_2}}$	$\tilde{\phi}_2$	$ \tilde{\phi}_2 - \tilde{\phi}_2^{\text{ref}} $
3LMC-2	4	3.74×10^{-6}	1.47×10^{-4}	6.23×10^{-7}	3.61×10^{-9}	1.56×10^{-7}	3.10×10^{-9}
	5	3.74×10^{-6}	-2.44×10^{-6}	2.18×10^{-6}	4.00×10^{-9}	1.64×10^{-7}	2.00×10^{-9}
	6	5.29×10^{-6}	5.93×10^{-4}	2.96×10^{-6}	6.56×10^{-9}	3.17×10^{-7}	3.35×10^{-6}
	1	1.48×10^{-5}	2.87×10^{-3}	2.72×10^{-5}	5.20×10^{-8}	2.57×10^{-6}	3.56×10^{-6}
	2	1.58×10^{-5}	3.15×10^{-3}	2.80×10^{-5}	5.48×10^{-8}	2.88×10^{-6}	3.05×10^{-6}
	3	1.38×10^{-5}	2.51×10^{-3}	2.18×10^{-5}	4.47×10^{-8}	2.23×10^{-6}	4.12×10^{-8}
	4	3.74×10^{-6}	1.50×10^{-4}	2.72×10^{-6}	3.87×10^{-9}	1.58×10^{-7}	4.49×10^{-9}
	5	3.74×10^{-6}	-6.15×10^{-6}	1.53×10^{-6}	3.74×10^{-9}	1.62×10^{-7}	3.85×10^{-9}
	6	5.39×10^{-6}	5.93×10^{-4}	2.64×10^{-6}	7.21×10^{-9}	3.31×10^{-7}	3.34×10^{-6}
	1	1.52×10^{-5}	2.88×10^{-3}	2.96×10^{-5}	5.00×10^{-8}	2.56×10^{-6}	3.58×10^{-6}
	2	1.58×10^{-5}	3.16×10^{-3}	3.24×10^{-5}	5.39×10^{-8}	2.88×10^{-6}	3.05×10^{-6}
	3	1.41×10^{-5}	2.52×10^{-3}	3.18×10^{-6}	4.58×10^{-8}	2.26×10^{-6}	7.40×10^{-8}
	4	3.74×10^{-6}	1.47×10^{-4}	4.34×10^{-7}	3.74×10^{-9}	1.55×10^{-7}	3.22×10^{-9}
4LMC-1	5	3.74×10^{-6}	-2.42×10^{-6}	2.20×10^{-6}	3.61×10^{-9}	1.59×10^{-7}	6.00×10^{-9}
	6	5.29×10^{-6}	5.99×10^{-4}	3.84×10^{-6}	7.00×10^{-9}	3.22×10^{-7}	3.35×10^{-6}

Figures 8.6 and 8.7 present the log-scale variance and cost per level, together with the MLMC rate parameters β and γ . All investigated methods fall into the regime $\beta > \gamma$, indicating that the estimator variance is dominated by the coarse levels, while the fine levels primarily serve to control the bias. This behavior is particularly pronounced for 2LMC-1, 2LMC-2, and 3LMC-1. Consequently, introducing additional coarse levels could further improve computational efficiency.

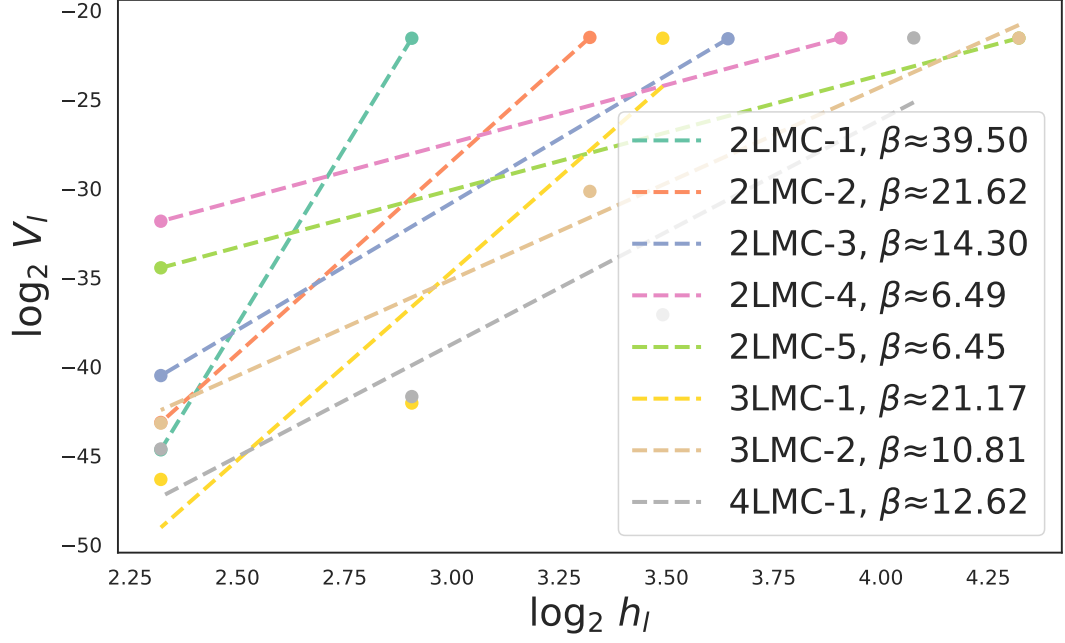


Figure 8.6: Level variances V_l for the Anisotropy problem, plotted against the mesh step h_l on a logarithmic scale. The fitted slope parameter β from MLMC Theorem 4.2.1 characterizes the decay rate of V_l with level.

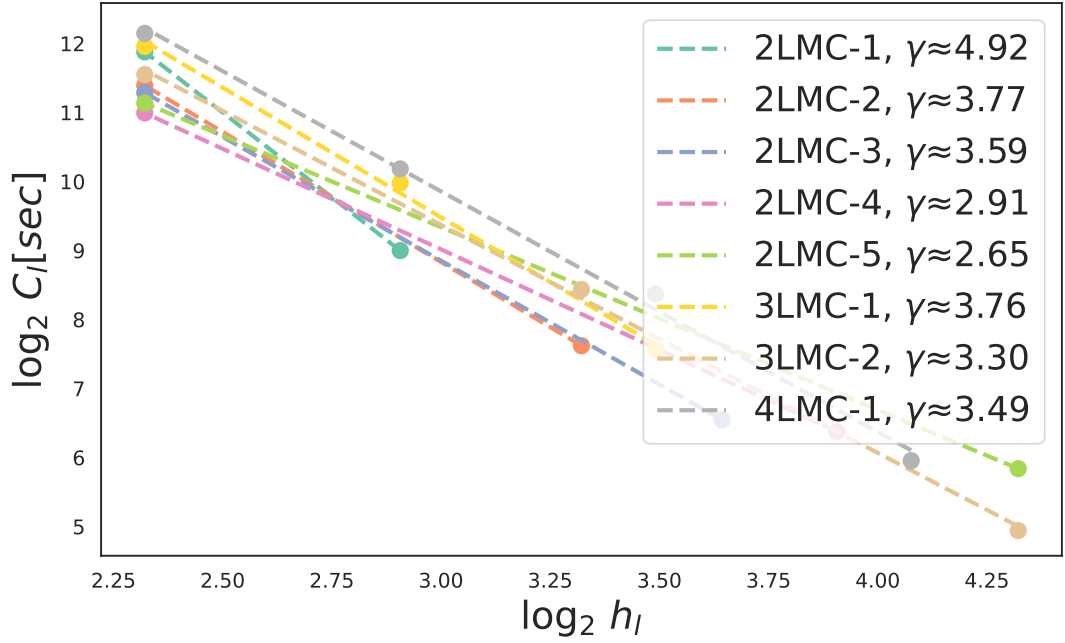


Figure 8.7: Level computational cost C_l for the Anisotropy problem, plotted against the mesh step h_l on a logarithmic scale. The fitted slope parameter γ from MLMC Theorem 4.2.1 describes the growth of C_l with level.

8.3.3 Performance Assessment of 3LMC-2

The performance of the three-level MLMC estimator 3LMC-2 for the Anisotropy problem is assessed using the same comparative framework introduced in Section 8.2.3. 3LMC-2 is evaluated against a variant employing CPU-based surrogate inference (3LMC-2-CPU) and a configuration relying on numerical homogenization without a surrogate (3LMC-2-NH). As in the Constraint problem, computational costs are reused across configurations, except for the homogenization component.

The computational costs required to meet the target variance V_t are summarized in Table 8.7. The same qualitative trends observed for the Constraint problem persist. However, due to the relatively small number of samples on levels $l > 1$, the higher cost of numerical homogenization does not lead to a proportionally large increase in the total computational cost C . CPU-based surrogate inference results in only a marginal increase in total cost (approximately 3%) compared to GPU-based inference, whereas replacing the surrogate with numerical homogenization leads to a substantial cost increase (approximately 74%). Despite this, all investigated configurations remain significantly more efficient than the corresponding single-level Monte Carlo estimator. Even the most expensive configuration, 3LMC-2-NH, accounts for only about 3% of the computational cost of the standard MC method ($C = 24,317,334$).

Table 8.7: Computational cost of 3LMC-2 variants for the Anisotropy problem.

	N	C_l	C
3LMC-2	[11,731; 80; 14]	[31; 345; 2,989]	428,651
3LMC-2-CPU	[11,741; 80; 14]	[24; 427; 3,196]	442,875
3LMC-2-NH	[12,039; 80; 14]	[31; 2,481; 12,379]	744,995

The runtime breakdown averaged over 20 samples (Figure 8.8) is very similar to that observed for the Constraint problem (see Figure 8.4). However, simulation runs account for a larger fraction of the total runtime in this case, which further reduces the relative impact of accelerated homogenization on the total computational cost.

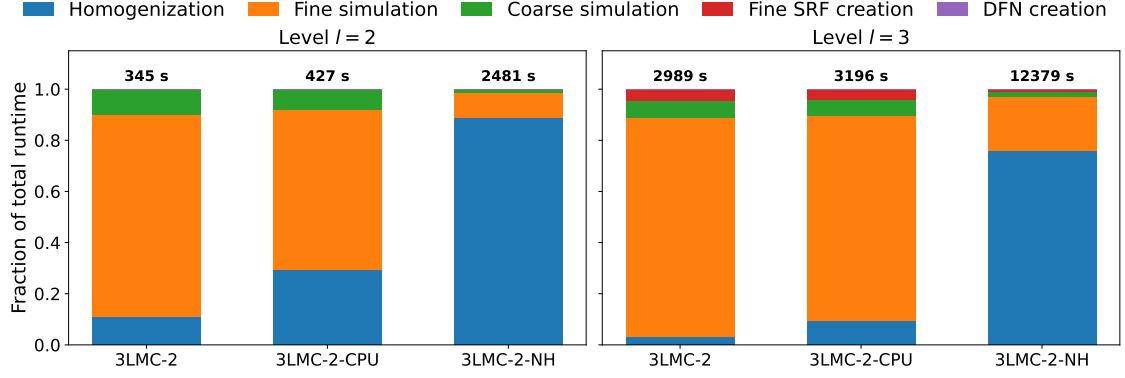


Figure 8.8: Normalized runtime breakdown for the Anisotropy problem using three-level MLMC estimators. For levels $l \in \{2, 3\}$, stacked bars show the fractional contribution of the main computational components. 3LMC-2 (GPU-based surrogate inference), 3LMC-2-CPU (CPU-based surrogate inference), and 3LMC-2-NH (run without surrogates) are compared. Absolute total runtimes for each configuration are annotated above the bars.

The level-wise variance and cost behavior, together with the corresponding MLMC Theorem parameters, are shown in Figure 8.9. In contrast to the relationship between the decay rates β and γ observed for the Constraint problem (see Figure 8.5), here it is $\beta > \gamma$ even for 3LMC-2-NH, enabling potential improvements of efficiency by adding new coarse levels also for the case with no surrogate.

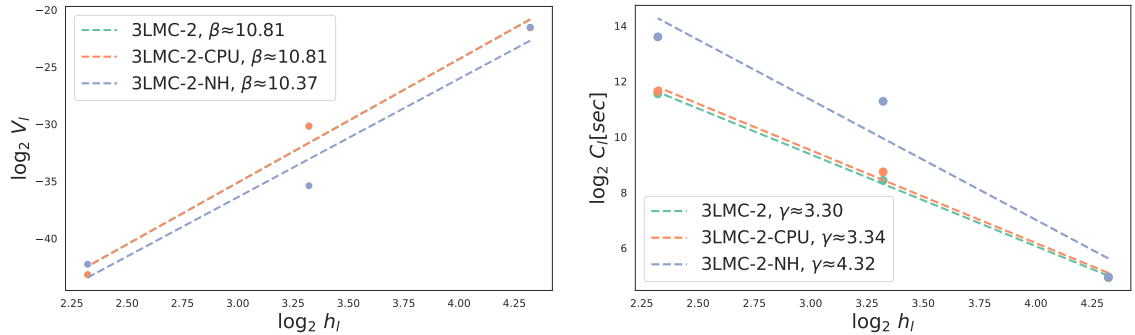


Figure 8.9: Variance (left) and computational cost (right) per level of 3LMC for the Anisotropy problem. The level is determined by the mesh step h_l . Parameters β and γ correspond to the MLMC Theorem.

This chapter presented the results obtained for the MLMC methods applied to DFM models. Two-, three-, and four-level Monte Carlo methods were examined and compared against the single-level MC baseline. For the Constraint problem, MLMC reduced the computational cost to approximately 6% of the standard MC cost, and to approximately 1.8% for the Anisotropy problem, without compromising the estimated statistics. Notably, substantial cost reductions were also achieved without surrogate models: to about 23% for the Constraint problem and to about 3% for the Anisotropy problem. Overall, in settings where most computational effort is concentrated on coarse levels and only a limited number of samples are required on the finest level, the homogenization step does not impede the construction of

an efficient MLMC estimator. However, for the proposed MLMC–DFM procedure based on approach (A), the requirement to construct a population of equivalent tensors $\mathcal{B}_l$ of a prescribed size artificially increases the computational cost. This aspect requires further investigation to determine whether the cost associated with generating $\mathcal{B}_l$ can be reduced. In addition, limitations on admissible differences between mesh steps at consecutive levels were identified as necessary to avoid biased estimators. These constraints should be examined further, together with the influence of correlation length on coarse levels, to better understand their impact on MLMC estimator accuracy and efficiency.

9 Conclusions

This dissertation presented a comprehensive framework for applying the multi-level Monte Carlo (MLMC) method to three-dimensional discrete fracture–matrix (DFM) models. The framework was enabled by numerical homogenization, which provided the essential mechanism for transitioning between resolution levels. The primary focus of this work was to significantly accelerate the homogenization step by means of deep learning–based surrogate models and to integrate these surrogates into MLMC estimators for efficient uncertainty quantification in fractured porous medium.

The surrogates replaced a computationally expensive finite element method-based simulator, directly predicting the equivalent hydraulic conductivity tensor, $\mathbf{K}^{eq}$, from voxelized hydraulic conductivities of DFM blocks. Three training datasets were generated, each corresponding to a fracture-to-matrix hydraulic conductivity ratio $K_f/K_m \in \{1 \times 10^3, 1 \times 10^5, 1 \times 10^7\}$. The surrogate architecture was specifically designed for this task, with large-scale training performed on high-performance computing clusters. The trained surrogates achieved high predictive accuracy on test samples, with NRMSE < 0.22 for all tensor components.

The generalization capability of the surrogates was thoroughly examined across varying matrix parameters and fracture network configurations. Accuracy remained stable within the training data distribution, with only moderate degradation observed under extreme conditions, such as substantially increased fracture densities. From a computational perspective, the surrogates provided substantial acceleration. Surrogate-based upscaling executed on a GPU achieved speedups of approximately $100\times$ compared to conventional numerical homogenization.

To assess the influence of surrogate accuracy at the macroscale, the upscaled conductivity tensors obtained from numerical homogenization and from the surrogates were incorporated into two macroscale hydrological test cases: the Constraint and Anisotropy problems. For both problems, the quantities of interest exhibited only minor differences between surrogate- and homogenization-based upscaling, even in scenarios where macroscale behavior was strongly influenced by the upscaled conductivities. These results confirm that surrogate errors in homogenization do not propagate in a detrimental manner to the macroscale quantities of interest considered.

Building on these findings, the trained surrogates were integrated into MLMC estimators within an MLMC-DFM software framework. Several MLMC hierarchies were systematically evaluated for both macroscale problems. The surrogate-accelerated MLMC approach consistently delivered substantial reductions in compu-

tational cost compared to the standard single-level Monte Carlo approach, without loss of estimator accuracy. For the Constraint problem, the most efficient configuration required approximately 6% of the single-level Monte Carlo cost, while for the Anisotropy problem, the cost was reduced to approximately 1.8%. The observed variance-cost behaviour confirms that specific three-level hierarchies employing surrogate-based homogenization can be highly efficient, even when inference is performed on CPUs rather than GPUs.

Overall, this dissertation demonstrated that deep learning-based surrogates for numerical homogenization, when embedded within an MLMC framework and supported by a configurable MLMC software implementation, constitute an effective strategy for uncertainty quantification in fractured porous medium. The combination of strong surrogate generalization, significant computational acceleration, and robust MLMC implementation represents a key contribution of this work. In addition to these methodological and computational results, the complete framework was implemented as open-source software and made publicly available, thereby supporting reproducibility and reuse. Furthermore, key results related to surrogate-based numerical homogenization have been published in peer-reviewed journals, providing independent validation of the proposed approaches.

Future research directions include extending surrogate applicability to broader parameter ranges, improving robustness under extreme fracture densities and homogenization block sizes, and developing adaptive MLMC strategies that automatically select the number of levels and their resolutions based on variance and cost estimates. Such extensions would further enhance the practicality and efficiency of MLMC methods for complex hydrogeological problems.

Bibliography

- [1] *Geological Disposal Facilities for Radioactive Waste*. IAEA Safety Standards Series: Specific Safety Guide No. SSG-14. International Atomic Energy Agency, Vienna, 2011.
- [2] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, and X. Zheng. Tensorflow: A system for large-scale machine learning. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pages 265–283, 2016.
- [3] Pierre M. Adler and Jean-François Thovert. *Fractures and Fracture Networks*. Springer Netherlands, Dordrecht, 1999.
- [4] M. Ahmed, Z. Zhao, and Y. Liu. Forecasting groundwater quality using deep learning time-series techniques for arid regions. *Sustainability*, 15(8):6529, 2023.
- [5] Naif Alqahtani, Fatimah Alzubaidi, Ryan T. Armstrong, Pawel Swietojanski, and Peyman Mostaghimi. Machine learning for predicting properties of porous media from 2d x-ray images. *Journal of Petroleum Science and Engineering*, 184:106514, 2020.
- [6] Laith Alzubaidi, Jinglan Zhang, Amjad J. Humaidi, Ayad Al-Dujaili, Ye Duan, Omran Al-Shamma, J. Santamaría, Mohammed A. Fadhel, Muthana Al-Amidie, and Laith Farhan. Review of deep learning. *Journal of Big Data*, 8(1), 2021.
- [7] Mary P Anderson, William W Woessner, and Randall J Hunt. *Applied groundwater modeling: simulation of flow and advective transport*. Academic press, second edition, 2015.
- [8] Nikolai Andrianov. Upscaling of two-phase discrete fracture simulations using a convolutional neural network. *Computational Geosciences*, 26:1237 – 1259, 2022.
- [9] M. J. Asher. *Surrogate Groundwater Models*. PhD thesis, Mathematical Science Institute, ANU College of Science, The Australian National University, 2021.

- [10] M. J. Asher, B. F. W. Croke, A. J. Jakeman, and L. J. M. Peeters. A review of surrogate models and their application to groundwater modeling. *Water Resources Research*, 51(8):5957–5973, 2015.
- [11] Jean-Louis Auriault, Claude Boutin, and Christian Geindreau. *Homogenization of Coupled Phenomena in Heterogenous Media*. ISTE, London, UK, 2009.
- [12] Siroos Azizmohammadi and Stephan K. Matthäi. Is the permeability of naturally fractured rocks scale dependent? *Water Resources Research*, 53(9):8041–8063, 2017.
- [13] Husam Musa Baalousha. Using Monte Carlo simulation to estimate natural groundwater recharge in Qatar. *Modeling Earth Systems and Environment*, 2:1–7, 2016.
- [14] David Banks and Nick Robins. *An introduction to groundwater in crystalline bedrock*. Geological Survey of Norway, Trondheim, Norway, 2002.
- [15] Andrew R. Barron and Chyong-Hwa Sheu. Approximation of Density Functions by Sequences of Exponential Families. *The Annals of Statistics*, 19(3):1347–1369, 1991.
- [16] Inga Berre, Florian Doster, and Eirik Keilegavlen. Flow in fractured porous media. *Transport in Porous Media*, 130(1):215–236, 2019.
- [17] Stefano Berrone, Claudio Canuto, Sandra Pieraccini, and Stefano Scialò. Uncertainty quantification in discrete fracture network models: Stochastic geometry. *Water Resources Research*, 54(2):1338–1352, 2018.
- [18] Stefano Berrone, Sandra Pieraccini, and Stefano Scialò. On Simulations of Discrete Fracture Network Flows with an Optimization-Based Extended Finite Element Method. *SIAM Journal on Scientific Computing*, 35(2):A908–A935, 2013.
- [19] Claudio Bierig and Alexey Chernov. Approximation of probability density functions by the Multilevel Monte Carlo Maximum Entropy method. *Journal of Computational Physics*, 314:661–681, 2016.
- [20] Onur Bilgin, Thomas Vergutz, and Siamak Mehrkanon. Gcn-ffnn: A two-stream deep model for learning solution to partial differential equations. *Neurocomputing*, 511:131–141, 2022.
- [21] I. I. Bogdanov, V. V. Mourzenko, J.-F. Thovert, and P. M. Adler. Effective permeability of fractured porous media in steady state flow. *Water Resources Research*, 39(1), 2003.
- [22] I. I. Bogdanov, V. V. Mourzenko, J.-F. Thovert, and P. M. Adler. Effective permeability of fractured porous media with power-law distribution of fracture sizes. *Phys. Rev. E*, 76:036309, Sep 2007.

- [23] E. Bonnet, O. Bour, N. E. Odling, P. Davy, I. Main, P. Cowie, and B. Berkowitz. Scaling of fracture systems in geological media. *Reviews of Geophysics*, 39(3):347–383, 2001.
- [24] Olivier Bour and Philippe Davy. Connectivity of random fault networks following a power law fault length distribution. *Water Resources Research*, 33(7):1567–1583, 1997.
- [25] Jan Březina and Jan Stebel. Analysis of model error for a continuum-fracture model of porous media flow. In *High Performance Computing in Science and Engineering*, pages 152–160, Cham, 2016. Springer.
- [26] Jan Březina, Jan Stebel, Pavel Exner, and Jan Hybš. Flow123d. <http://flow123d.github.io>, repository: <http://github.com/flow123d/flow123d>, 2011–2025.
- [27] Jan Březina and Martin Špetlík. MLMC Python library. <https://pypi.org/project/mlmc/>, 2025.
- [28] Martin D. Buhmann. *Radial Basis Functions: Theory and Implementations*. Cambridge University Press, 2003.
- [29] Jan Březina, Jiří Kopal, Radek Srb, Jana Ehlerová, and Jiří Hnídek. bgem: B-spline geometry modeling package. <https://github.com/GeoMop/bgem>, 2025.
- [30] Baris Caglar, Guillaume Broggi, Muhammad A. Ali, Laurent Orgéas, and Véronique Michaud. Deep learning accelerated prediction of the permeability of fibrous microstructures. *Composites Part A: Applied Science and Manufacturing*, 158:106973, 2022.
- [31] Chen Cai, Nikolaos Vlassis, Lucas Magee, Ran Ma, Zeyu Xiong, Bahador Bahmani, Teng-Fong Wong, Yusu Wang, and WaiChing Sun. Equivariant geometric learning for digital rock physics: estimating formation factor and effective permeability tensors from morse graph. *International Journal for Multiscale Computational Engineering*, 21(5):1–24, 2023.
- [32] Tao Chen, Christoph Clauser, Gabriele Marquart, Karen Willbrand, and Darius Mottaghy. A new upscaling method for fractured porous media. *Advances in Water Resources*, 80:60–68, 2015.
- [33] Yingyi Chen, Lihua Song, Yeqi Liu, Ling Yang, and Daoliang Li. A review of the artificial neural network models for water quality prediction. *Applied Sciences*, 10(17), 2020.
- [34] K. A. Cliffe, M. B. Giles, R. Scheichl, and A. L. Teckentrup. Multilevel Monte Carlo methods and applications to elliptic pdes with random coefficients. *Computing and Visualization in Science*, 14(1):3–15, 2011.

- [35] Tomasz Danel, Przemysław Spurek, Jacek Tabor, Marek Śmieja, Łukasz Struski, Agnieszka Słowik, and Łukasz Maziarka. Spatial graph convolutional networks. In *Neural Information Processing*, pages 668–675, Cham, 2020. Springer.
- [36] J.-R. de Dreuzy, Y. Méheust, and G. Pichot. Influence of fracture scale heterogeneity on the flow properties of three-dimensional discrete fracture networks (DFN). *Journal of Geophysical Research: Solid Earth*, 117(B11), 2012.
- [37] Michaël Defferrard, Xavier Bresson, and Pierre Vandergheynst. Convolutional neural networks on graphs with fast localized spectral filtering. In *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc., 2016.
- [38] C. L. Farmer. Upscaling: a review. *International Journal for Numerical Methods in Fluids*, 40(1-2):63–78, 2002.
- [39] P. E. Farrell, M. B. Giles, M. Croci, T. Roy, and C. Beentjes. MLMC software project (Python). <https://bitbucket.org/pefarrell/pymmlmc/src/master/>, 2020.
- [40] Carlos A.S. Ferreira, Teeratrorn Kadeethum, Nikolaos Bouklas, and Hamidreza M. Nick. A framework for upscaling and modelling fluid flow for discrete fractures using conditional generative adversarial networks. *Advances in Water Resources*, 166:104264, 2022.
- [41] Alexander Forrester, Andras Sobester, and Andy Keane. *Engineering Design via Surrogate Modelling: A Practical Guide*. John Wiley & Sons, 2008.
- [42] Michael B. Giles. Multilevel Monte Carlo path simulation. *Operations Research*, 56(3):607–617, 2008.
- [43] Michael B. Giles. Multilevel Monte Carlo methods. *Acta Numerica*, 24:259–328, 2015.
- [44] Michael B. Giles, Tigran Nagapetyan, and Klaus Ritter. Multilevel Monte Carlo approximation of distribution functions and densities. *SIAM/ASA Journal on Uncertainty Quantification*, 3(1):267–295, 2015.
- [45] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, Cambridge, Massachusetts, 2016. <http://www.deeplearningbook.org>.
- [46] Daniele Grattarola and Cesare Alippi. Graph neural networks in tensor-flow and keras with spektral. *IEEE Computational Intelligence Magazine*, 16(1):99–106, feb 2021.
- [47] N. Guezgouz, D. Boutoutaou, and A. Hani. Prediction of groundwater flow in shallow aquifers using artificial neural networks in the northern basins of Algeria. *Journal of Water and Climate Change*, 12(4):1220–1228, 07 2020.

- [48] Teklu Hadgu, Satish Karra, Elena Kalinina, Natalia Makedonska, Jeffrey D. Hyman, Katherine Klise, Hari S. Viswanathan, and Yifeng Wang. A comparative study of discrete fracture network and equivalent continuum models for simulating flow and transport in the far field of a hypothetical nuclear waste repository in crystalline host rock. *Journal of Hydrology*, 553:59–70, 2017.
- [49] William L. Hamilton. Graph representation learning. *Synthesis Lectures on Artificial Intelligence and Machine Learning*, 14(3):1–159.
- [50] William L. Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. *ArXiv*, abs/1706.02216, 2017.
- [51] Marwah Sattar Hanoon, Ali Najah Ahmed, Chow Ming Fai, Ahmed H. Birima, Arif Razzaq, Mohsen Sherif, Ahmed Sefelnasr, and Ahmed El-Shafie. Application of artificial intelligence models for modeling water quality in groundwater. *Water, Air, & Soil Pollution*, 232(10), 2021.
- [52] N. J. Hardebol, C. Maier, H. Nick, S. Geiger, G. Bertotti, and H. Boro. Multiscale fracture network characterization and impact on flow: A case study on the latemar carbonate platform. *Journal of Geophysical Research: Solid Earth*, 120(12):8197–8222, 2015.
- [53] Xupeng He, Ryan Santoso, Marwa Alsinan, Hyung Kwak, and Hussein Hoteit. Constructing Dual-Porosity Models from High-Resolution Discrete-Fracture Models Using Deep Neural Networks. SPE Reservoir Simulation Conference, 10 2021.
- [54] C. Heaney, Yuling Li, O. Matar, and C. Pain. Applying convolutional neural networks to data on unstructured meshes with space-filling curves. *ArXiv*, abs/2011.14820, 2020.
- [55] Stefan Heinrich. Multilevel Monte Carlo methods. In Svetozar Margenov, Jerzy Waśniewski, and Plamen Yalamov, editors, *Large-Scale Scientific Computing*, pages 58–67, Berlin, Heidelberg, 2001. Springer.
- [56] Jin Hong and Jie Liu. Rapid estimation of permeability from digital rock using 3d convolutional neural network. *Computational Geosciences*, 24(4):1523–1539, 2020.
- [57] A. L. James and C. M. Oldenburg. Linear and Monte Carlo uncertainty analysis for subsurface contaminant transport simulation. *Water Resources Research*, 33(11):2495–2508, 1997.
- [58] Warner James, Luke Morrill, and Juan Barrientos. Multi-Level Monte Carlo with Python (MLMCPy). <https://software.nasa.gov/software/LAR-19515-1>, 2018.

- [59] Steven Joyce, Trevor Simpson, Lee Hartley, David Applegate, Jaap Hoek, Peter Jackson, David Swan, Niko Marsic, and Sven Follin. Groundwater flow modelling of periods with temperate climate conditions – Forsmark. SKB R-09-20, Svensk Kärnbränslehantering AB:315.
- [60] Paul F. Juckem and Michael N. Fienen. Simulation of the probabilistic plume extent for a potential replacement wastewater-infiltration lagoon, and probabilistic contributing areas for supply wells for the town of lac du flambeau, vilas county, wisconsin. Open-File Report 2020–1032, U.S. Geological Survey, 2020.
- [61] Paul F. Juckem, Michael N. Fienen, and Randall J. Hunt. Simulation of groundwater flow and interaction of groundwater and surface water on the Lac du Flambeau reservation, Wisconsin. Scientific Investigations Report 2014–5020, U.S. Geological Survey, 2014.
- [62] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *ArXiv*, abs/1412.6980, 2014.
- [63] Maximilian O. Kottwitz, Anton A. Popov, Steffen Abe, and Boris J. P. Kaus. Investigating the effects of intersection flow localization in equivalent-continuum-based upscaling of flow in discrete fracture networks. *Solid Earth*, 12(10):2235–2254, 2021.
- [64] N. Koudina, R. Gonzalez Garcia, J.-F. Thovert, and P. M. Adler. Permeability of three-dimensional fracture networks. *Physical Review E*, 57(4):4466–4479, 1998.
- [65] Slawomir Koziel and Leifur Leifsson. *Surrogate-Based Modeling and Optimization*. Springer, New York, NY, 2013.
- [66] Prashant Kumar, Peiyao Luo, Francisco J. Gaspar, and Cornelis W. Oosterlee. A multigrid multilevel Monte Carlo method for transport in the darcy-stokes system. *Journal of Computational Physics*, 371:382–408, 2018.
- [67] Eric Laloy, Romain Hérault, Diederik Jacques, and Niklas Linde. Training-image based geostatistical inversion using a spatial generative adversarial neural network. *Water Resources Research*, 54(1):381–406, 2018.
- [68] P. S. Lang, A. Paluszny, and R. W. Zimmerman. Permeability tensor of three-dimensional fractured porous rock and a comparison to trace map predictions. *Journal of Geophysical Research: Solid Earth*, 119(8):6288–6307, 2014.
- [69] S. H. Lee, M. F. Lough, and C. L. Jensen. Hierarchical modeling of flow in naturally fractured formations with multiple length scales. *Water Resources Research*, 37(3):443–455, 2001.

- [70] Mingliang Liu, Rasool Ahmad, Wei Cai, and Tapan Mukerji. Hierarchical homogenization with deep-learning-based surrogate model for rapid estimation of effective permeability from digital rocks. *Journal of Geophysical Research: Solid Earth*, 128(2):e2022JB025378, 2023.
- [71] Richeng Liu, Bo Li, Yujing Jiang, and Na Huang. Review: Mathematical expressions for estimating equivalent permeability of rock fracture networks. *Hydrogeology Journal*, 24(7):1623–1649, 2016.
- [72] Tianyuan Liu, Yunzhu Li, Qi Jing, Yonghui Xie, and Di Zhang. Supervised learning method for the physical field reconstruction in a nanofluid heat transfer problem. *International Journal of Heat and Mass Transfer*, 165:120684, 2021.
- [73] J. C. S. Long, J. S. Remer, C. R. Wilson, and P. A. Witherspoon. Porous media equivalents for networks of discontinuous fractures. *Water Resources Research*, 18(3):645–658, 1982.
- [74] Jean Marçais and Jean-Raynald de Dreuzy. Prospective interest of deep learning for hydrological inference. *Groundwater*, 55(5):688–692, 2017.
- [75] Yinquan Meng, Jianguo Jiang, Jichun Wu, and Dong Wang. Transformer-based deep learning models for predicting permeability of porous media. *Advances in Water Resources*, 179:104520, 2023.
- [76] Jan Mohring, René Milk, Adrian Ngo, Ole Klein, Oleg Iliev, Mario Ohlberger, and Peter Bastian. Uncertainty quantification for porous media flow using multilevel Monte Carlo. In *Large-Scale Scientific Computing*, pages 145–152, Cham, 2015. Springer.
- [77] V. V. Mourzenko, J.-F. Thovert, and P. M. Adler. Percolation of three-dimensional fracture networks with power-law size distribution. *Physical Review E*, 72(3), 2005.
- [78] Kevin P. Murphy. *Probabilistic Machine Learning: An introduction*. MIT Press, 2022.
- [79] Florian Müller, Patrick Jenny, and Daniel W. Meyer. Multilevel Monte Carlo for two phase flow and buckley–leverett transport in random heterogeneous porous media. *Journal of Computational Physics*, 250:685–702, 2013.
- [80] Sebastian Müller and Lennart Schöler. GSTools. <https://github.com/GeoStat-Framework/GSTools>, 2019.
- [81] Giannis Nikolentzos, Siyun Wang, Johannes Lutzeyer, and Michalis Vazirgiannis. Graph neural machine: A new model for learning with tabular data. *ArXiv*, abs/2402.02862, 2024.

- [82] Francis Ogoke, Kazem Meidani, Amirreza Hashemi, and Amir Barati Farmani. Graph convolutional networks applied to unstructured flow field data. *Machine Learning: Science and Technology*, 2(4):045020, 2021.
- [83] Antonio Ortega, Pascal Frossard, Jelena Kovačević, José M. F. Moura, and Pierre Vandergheynst. Graph signal processing: Overview, challenges, and applications. *Proceedings of the IEEE*, 106(5):808–828, 2018.
- [84] Mayur Pal, Pijus Makauskas, and Shruti Malik. Upscaling porous media using neural networks: A deep learning approach to homogenization and averaging. *Processes*, 11(2), 2023.
- [85] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, and W. Zaremba. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS 2019)*, volume 32, pages 8026–8037, 2019.
- [86] Hao Peng, An Liu, Jingcheng Huang, Lingxin Cao, Jikai Liu, and Lin Lu. Ph-net: Parallelepiped microstructure homogenization via 3d convolutional neural networks. *Additive Manufacturing*, 60:103237, 2022.
- [87] Sandra Pieraccini. Uncertainty quantification analysis in discrete fracture network flow simulations. *GEM - International Journal on Geomathematics*, 11:1–21, 2020.
- [88] Prajit Ramachandran, Barret Zoph, and Quoc Le. Swish: a self-gated activation function. *ArXiv*, abs/1710.05941, 2017.
- [89] Chengping Rao and Yang Liu. Three-dimensional convolutional neural network (3d-cnn) for heterogeneous material homogenization. *Computational Materials Science*, 184:109850, 2020.
- [90] Saman Razavi, Bryan A. Tolson, and Donald H. Burn. Review of surrogate modeling in water resources. *Water Resources Research*, 48(7), 2012.
- [91] Ph. Renard and G. de Marsily. Calculating equivalent permeability: a review. *Advances in Water Resources*, 20(5):253–278, 1997.
- [92] Philippe Renard and Rachid Ababou. Equivalent permeability tensor of heterogeneous media: Upscaling methods and criteria (review and analyses). *Geosciences*, 12(7), 2022.
- [93] Ingvar Rhén, Torbjörn Forsmark, Lee Hartley, Peter Jackson, David Roberts, Dave Swan, and Björn Gylling. Hydrogeological conceptualisation and parameterization, site descriptive modelling sdm-site laxemar. SKB R-08-78:745, 2008.
- [94] T. D. Robinson, M. S. Eldred, K. E. Willcox, and R. Haimes. Surrogate-based optimization using multifidelity models with variable parameterization and corrected space mapping. *AIAA Journal*, 46(11):2814–2822, 2008.

- [95] Muhammad Sahimi. *Flow and Transport in Porous Media and Fractured Rock*. Wiley, Germany, second edition, 2011.
- [96] Xavier Sanchez-Vila, Alberto Guadagnini, and Jesus Carrera. Representative hydraulic conductivities in saturated groundwater flow. *Reviews of Geophysics*, 44(3), 2006.
- [97] T. H. Sandve, I. Berre, and J. M. Nordbotten. An efficient multi-point flux approximation method for Discrete Fracture–Matrix simulations. *Journal of Computational Physics*, 231(9):3784–3800, 2012.
- [98] Javier E. Santos, Duo Xu, Honggeun Jo, Christopher Landry, Masa Prodanovic, and Michael J. Pyrcz. Poreflow-net: A 3d convolutional neural network to predict fluid flow through porous media. *Advances in Water Resources*, 138:103539, 2020.
- [99] Chaopeng Shen. A transdisciplinary review of deep learning research and its relevance for water resources scientists. *Water Resources Research*, 54(11):8558–8593, 2018.
- [100] Sergei Stepanov, Denis Spiridonov, and Tina Mai. Prediction of numerical homogenization using deep learning for the richards equation. *Journal of Computational and Applied Mathematics*, 424:114980, 2023.
- [101] L. Sun and J.-X. Wang. Surrogate modeling for fluid flow: A comparison of machine learning techniques. *Computer Methods in Applied Mechanics and Engineering*, 361:112732, 2020.
- [102] C. Taccari, F. Rizzo, and M. Lombardo. Cost-effective groundwater flow modeling using deep neural operators (deeponet). *ArXiv*, abs/2304.12299, 2023.
- [103] Richard J. Trudeau. *Introduction to graph theory*. Dover Publications, New York, 1st edition, 1993.
- [104] Maria Vasilyeva and Aleksey Tyrylgina. Machine learning for accelerating macroscopic parameters prediction for poroelasticity problem in stochastic media. *Computers & Mathematics with Applications*, 84:185–202, 2021.
- [105] Jonathan Viquerat and Elie Hachem. A supervised neural network for drag prediction of arbitrary 2d shapes in laminar flows at low reynolds number. *Computers & Fluids*, 210:104645, 2020.
- [106] Minjie Wang, Da Zheng, Zihao Ye, Quan Gan, Mufei Li, Xiang Song, Jinjing Zhou, Chao Ma, Lingfan Yu, Yu Gai, Tianjun Xiao, Tong He, George Karypis, Jinyang Li, and Zheng Zhang. Deep graph library: A graph-centric, highly-performant package for graph neural networks. *ArXiv*, abs/1909.01315, 2019.
- [107] Yanji Wang, Hangyu Li, Jianchun Xu, Shuyang Liu, and Xiaopu Wang. Machine learning assisted relative permeability upscaling for uncertainty quantification. *Energy*, 245:123284, 2022.

- [108] Ze Zhou Wang, Changlin Xiao, Siang Huat Goh, and Min-Xuan Deng. Metamodel-based reliability analysis in spatially variable soils using convolutional neural networks. *Journal of Geotechnical and Geoenvironmental Engineering*, 147(3), 2021.
- [109] Xian-Huan Wen and J.Jaime Gómez-Hernández. Upscaling hydraulic conductivities in heterogeneous media: An overview. *Journal of Hydrology*, 183(1):ix–xxxii, 1996.
- [110] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S. Yu. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32(1):4–24, 2021.
- [111] Mengfei Xu, S. Song, Xuxiang Sun, and Weiwei Zhang. Ucn: A convolutional strategy on unstructured mesh. *ArXiv*, abs/2101.05207, 2021.
- [112] Heesung Yoon, Seong-Chun Jun, Yunjung Hyun, Gwang-Ok Bae, and Kang-Kun Lee. A comparative study of artificial neural networks and support vector machines for predicting groundwater levels in a coastal aquifer. *Journal of Hydrology*, 396(1):128–138, 2011.
- [113] Hongkyu Yoon, David B. Hart, and Sean A. McKenna. Parameter estimation and predictive uncertainty in stochastic inverse modeling of groundwater flow: Comparing null-space Monte Carlo and multiple starting point methods. *Water Resources Research*, 49, 2013.
- [114] Xiayang Yu, Tao Cui, J. Sreekanth, Stephane Mangeon, Rebecca Doble, Pei Xin, David Rassam, and Mat Gilfedder. Deep learning emulators for groundwater contaminant transport modelling. *Journal of Hydrology*, 590:125351, 2020.
- [115] Aston Zhang, Zachary C. Lipton, Mu Li, and Alexander J. Smola. *Dive into Deep Learning*. 2020. <https://d2l.ai>.
- [116] Dongxiao Zhang. *Stochastic Methods for Flow in Porous Media*. Elsevier, USA, 1. edition, 2002.
- [117] Ding-Xuan Zhou. Universality of deep convolutional neural networks. *Applied and Computational Harmonic Analysis*, 48(2):787–794, 2020.
- [118] Martin Špetlík. MLMC-DFM. <https://github.com/martinspetlik/MLMC-DFM>, 2025.
- [119] Martin Špetlík, Jan Březina, and Eric Laloy. Deep learning surrogate for predicting hydraulic conductivity tensors from stochastic discrete fracture-matrix models. *Computational Geosciences*, October 2024.
- [120] Martin Špetlík and Jan Březina. Groundwater flow meta-model for multilevel Monte Carlo methods. In *CEUR Workshop Proceedings*, volume 2962, pages 104–113, 2021.

- [121] Martin Špetlík and Jan Březina. Groundwater contaminant transport solved by Monte Carlo methods accelerated by deep learning meta-model. *Applied Sciences*, 12(15), 2022.
- [122] Martin Špetlík and Jan Březina. Convolutional surrogate for 3D discrete fracture–matrix tensor upscaling. *Computers & Geosciences*, 209, 2026.